République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Carthage

École Nationale d'Ingénieurs de Carthage

Département de Génie Informatique

Projet de Fin d'Études

Présenté par

Oulimata SALL

Pour l'obtention du

Diplôme National d'Ingénieur en Informatique

Détection d'équipements par intelligence artificielle

Climatiseurs, équipements informatiques, audiovisuels et électroménagers

avec Vision-RAG, n8n et application mobile Flutter

Au sein de

Sous la supervision de :

Encadrante au sein de l'organisme d'accueil : Mme Ben Noamen Mariem

Encadrante académique : Mme Fourati Myriam

Année universitaire 2025/2026

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui m'ont accompagnée durant la réalisation de ce projet de fin d'études.

Je remercie tout particulièrement **Mme Fourati Myriam**, mon encadrante académique à l'**École Nationale d'Ingénieurs de Carthage**, pour son suivi, ses conseils et ses orientations tout au long de ce travail.

Mes remerciements vont également à **Mme Ben Noamen Mariem**, mon encadrante au sein de **SFM Technologies**, pour son accompagnement, sa disponibilité et ses remarques constructives.

J'adresse aussi mes remerciements aux membres du jury pour l'intérêt porté à ce travail et pour le temps consacré à son évaluation.

Je remercie l'ensemble du personnel de **l'École Nationale d'Ingénieurs de Carthage**, en particulier le **Département de Génie Informatique**, pour la qualité de la formation dispensée.

Enfin, j'adresse mes sincères remerciements à ma famille et à mes proches pour leur soutien constant durant ces années d'études. Une pensée particulière va à ma mère, qui n'a cessé de m'encourager tout au long de ce parcours.

Résumé

Ce projet de fin d'études, réalisé au sein de **SFM Technologies**, s'intéresse à la détection automatique d'équipements électriques à partir de courtes vidéos filmées par smartphone, dans le cadre d'audits énergétiques de bâtiments tertiaires.

Le travail réalisé consiste à concevoir une chaîne de traitement combinant la sélection de keyframes par filtre Laplacien (OpenCV), la détection *zero-shot* par **YOLO-World**, le suivi et la déduplication par **ByteTrack**, l'analyse visuelle par **Gemini 2.5 Flash**, l'enrichissement par **RAG** avec **Qdrant**, puis la validation des résultats à l'aide de **Pydantic V2**.

La solution produit des fiches **JSON Equipment** structurées regroupant notamment la désignation, la marque, l'état de fonctionnement et les principales spécifications techniques. Sur 60 keyframes comprenant 59 objets annotés, le détecteur atteint un mAP@50 de 74,9 %, une précision micro de 63,0 % et un rappel micro de 78,0 %. Les quatre vidéos de validation sont traitées en 21,4 secondes en moyenne côté client. Les résultats confirment la faisabilité du prototype, tout en montrant des limites de surdétection, de couverture du corpus RAG et de validation métier des fiches générées.

Mots-clés : vision par ordinateur, LLM multimodal, YOLO-World, Gemini, RAG, Qdrant, CRISP-DM, audit énergétique, FastAPI, Flutter.

Abstract

This final-year project, carried out at **SFM Technologies**, deals with the automatic detection of electrical equipment from short smartphone videos, in the context of energy audits for commercial buildings.

The proposed processing chain combines keyframe selection using a Laplacian filter, **YOLO-World** zero-shot detection, tracking and deduplication with **ByteTrack**, visual analysis using **Gemini 2.5 Flash**, **RAG**-based enrichment with **Qdrant**, and result validation through **Pydantic V2**.

The system generates structured **JSON Equipment** records including the equipment name, brand, operating status and main technical specifications. On 60 keyframes containing 59 manually annotated objects, the detector achieves a 74.9% mAP@50, 63.0% micro-precision and 78.0% micro-recall. The four validation videos are processed in an average client-side time of 21.4 seconds. These results confirm the feasibility of the prototype while highlighting limitations related to false positives, RAG corpus coverage and the business validation of generated records.

Keywords : computer vision, multimodal LLM, YOLO-World, Gemini, RAG, Qdrant, CRISP-DM, energy audit, FastAPI, Flutter.

Table des matières

Remerciements i

Résumé ii

Liste des Abréviations xi

Introduction générale 1

1 Cadre du projet et analyse des besoins 3
1.1 Présentation de l'organisme d'accueil . . . 3
1.1.1 Présentation générale de SFM Technologies . . . 3
1.1.2 Mission, vision et valeurs . . . 3
1.1.3 Domaines d'activité . . . 4
1.1.4 Département d'accueil . . . 4
1.2 Contexte métier et étude de l'existant . . . 5
1.2.1 Les audits énergétiques des bâtiments tertiaires . . . 5
1.2.2 Processus actuel chez SFM Technologies . . . 6
1.2.3 Analyse critique de l'existant . . . 7
1.3 Problématique et solution proposée . . . 8
1.3.1 Positionnement du problème . . . 8
1.3.2 Problématique . . . 8
1.3.3 Solution proposée . . . 9
1.4 Méthodologie adoptée . . . 11
1.4.1 Présentation de la méthodologie CRISP-DM . . . 11
1.4.2 Application de CRISP-DM au projet . . . 11
1.4.3 Organisation du projet . . . 12
1.5 Analyse des besoins . . . 12
1.5.1 Besoins fonctionnels . . . 12
1.5.2 Besoins non fonctionnels . . . 13
1.5.3 Contraintes techniques . . . 15
1.5.4 Acteurs du système . . . 15
1.5.5 Cas d'utilisation . . . 15
1.5.6 Matrice de traçabilité des besoins . . . 16
1.6 Analyse des risques . . . 17

2 État de l'art et choix technologiques 20
2.1 Vision par ordinateur pour la détection d'équipements . . . 20
2.1.1 Vision par ordinateur . . . 20
2.1.2 Détection d'objets . . . 21
2.1.3 Détection supervisée . . . 21
2.1.4 Détection open-vocabulary . . . 21
2.1.5 Principales approches existantes . . . 23

2.1.6 Étude comparative . . . 24
2.1.7 Choix de YOLOv8x-Worldv2 . . . 26
2.2 Suivi multi-objets dans les séquences vidéo . . . 26
2.2.1 Principe du suivi multi-objets . . . 27
2.2.2 SORT . . . 27
2.2.3 DeepSORT . . . 28
2.2.4 ByteTrack . . . 28
2.2.5 Étude comparative . . . 28
2.2.6 Choix de ByteTrack . . . 29
2.3 Intelligence artificielle multimodale . . . 29
2.3.1 Les grands modèles de langage (LLM) . . . 29
2.3.2 Les modèles vision-langage (VLM) . . . 30
2.3.3 GPT-4 Vision . . . 31
2.3.4 Gemini . . . 31
2.3.5 Claude . . . 32
2.3.6 LLaVA . . . 32
2.3.7 Étude comparative . . . 32
2.3.8 Choix de Gemini . . . 33
2.4 Génération augmentée par récupération . . . 34
2.4.1 Limites des modèles de langage . . . 34
2.4.2 Principe du RAG . . . 34
2.4.3 Embeddings . . . 35
2.4.4 Bases de données vectorielles . . . 35
2.4.5 Pinecone . . . 36
2.4.6 Chroma . . . 36
2.4.7 Weaviate . . . 36
2.4.8 FAISS . . . 36
2.4.9 Qdrant . . . 36
2.4.10 Étude comparative . . . 37
2.4.11 Choix de Qdrant . . . 37
2.5 Technologies de développement et de déploiement . . . 38
2.5.1 Flutter . . . 38
2.5.2 FastAPI . . . 38
2.5.3 n8n . . . 38
2.5.4 Docker comme perspective de déploiement . . . 39
2.5.5 Choix technologiques et perspective de déploiement . . . 39

3 Compréhension et préparation des données 40
3.1 Collecte des données . . . 40
3.1.1 Sources des données . . . 40
3.1.2 Collecte des séquences vidéo . . . 41
3.1.3 Collecte des données documentaires . . . 42
3.2 Compréhension des données brutes . . . 43
3.2.1 Profil exploratoire des vidéos . . . 43
3.2.2 Profil exploratoire des données documentaires . . . 43
3.2.3 Limites de représentativité . . . 43
3.3 Filtrage et nettoyage des données visuelles . . . 43
3.3.1 Prétraitement des vidéos . . . 44
3.3.2 Filtrage par qualité — variance du Laplacien . . . 44
3.3.3 Sélection de keyframes et extraction de crops . . . 45

3.3.4 Organisation des données visuelles . . . 46
3.3.5 Constitution de la vérité terrain . . . 47
3.4 Préparation de la base documentaire . . . 48
3.4.1 Structure des fiches techniques . . . 48
3.4.2 Nettoyage et normalisation . . . 49
3.4.3 Organisation des documents . . . 50
3.5 Vectorisation des données . . . 51
3.5.1 Génération des embeddings textuels . . . 51
3.5.2 Génération des embeddings visuels . . . 52
3.5.3 Stratégie de recherche multimodale . . . 53
3.5.4 Stockage dans Qdrant . . . 56
3.6 Synthèse et bilan de la préparation des données . . . 57
3.6.1 Statistiques du corpus . . . 57
3.6.2 Données finales retenues . . . 59

4 Conception et architecture du système 61
4.1 Vue fonctionnelle du système . . . 61
4.2 Architecture globale du système . . . 61
4.2.1 Présentation des composants . . . 61
4.2.2 Schéma d'architecture globale . . . 62
4.3 Conception détaillée des modules . . . 62
4.3.1 Module de traitement vidéo . . . 62
4.3.2 Module de détection . . . 63
4.3.3 Module RAG (`RAGService`) . . . 65
4.3.4 Module Gemini (`GeminiService`) . . . 65
4.4 Diagrammes dynamiques . . . 66
4.4.1 Diagramme de séquence global . . . 66
4.5 Diagrammes statiques . . . 67
4.5.1 Diagramme de classes métier . . . 67
4.5.2 Diagramme de composants . . . 67
4.6 Modélisation des données . . . 67
4.6.1 Architecture des données . . . 68
4.6.2 Modèle Equipment (Pydantic V2) . . . 68
4.6.3 Structure des réponses API . . . 69
4.7 Diagramme de déploiement . . . 69
4.7.1 Vue logique — interactions entre composants . . . 69
4.7.2 Vue physique et ressources matérielles . . . 70
4.8 Glossaire des termes techniques . . . 71

5 Résultats expérimentaux et validation 72
5.1 Protocole expérimental . . . 72
5.1.1 Objectifs de l'évaluation . . . 72
5.1.2 Niveaux d'évaluation . . . 72
5.1.3 Jeu de vidéos de test . . . 73
5.1.4 Critères d'évaluation . . . 73
5.1.5 Méthode de mesure . . . 74
5.1.6 Statut des mesures . . . 74
5.2 Environnement expérimental . . . 74
5.2.1 Configuration matérielle . . . 75
5.2.2 Configuration logicielle . . . 75

5.3 Évaluation du pipeline Vision-RAG . . . 75
5.3.1 Extraction des keyframes . . . 75
5.3.2 Détection YOLO-World . . . 77
5.3.3 Déduplication ByteTrack . . . 79
5.3.4 Recherche contextuelle RAG . . . 80
5.3.5 Génération des fiches Equipment . . . 80
5.3.6 Conformité structurelle avec Pydantic . . . 81
5.4 Évaluation des performances . . . 81
5.4.1 Temps par étape . . . 82
5.4.2 Temps global par vidéo . . . 82
5.4.3 Analyse des performances . . . 82
5.4.4 Test de robustesse et effet du quota externe . . . 82
5.5 Validation fonctionnelle . . . 83
5.5.1 Résultats par vidéo . . . 84
5.5.2 Tests des interfaces et cas limites . . . 84
5.5.3 Validation des fiches par le technicien . . . 85
5.6 Impact du module RAG . . . 85
5.6.1 Résultats sans RAG . . . 85
5.6.2 Résultats avec RAG . . . 85
5.6.3 Analyse comparative . . . 86
5.7 Discussion . . . 87
5.7.1 Forces du système . . . 87
5.7.2 Limites observées . . . 87
5.7.3 Perspectives d'amélioration . . . 87

6 Mise en œuvre et déploiement du prototype 88
6.1 Périmètre fonctionnel du prototype . . . 88
6.2 Environnement d'implémentation . . . 89
6.3 Implémentation du backend FastAPI . . . 90
6.3.1 Organisation des services . . . 90
6.3.2 Traitement d'une requête d'analyse . . . 91
6.3.3 Validation et gestion des erreurs . . . 92
6.4 Application mobile Flutter . . . 92
6.5 Automatisation avec n8n et persistance Supabase . . . 93
6.5.1 Workflow post-analyse . . . 93
6.5.2 Modèle de persistance . . . 93
6.6 Installation et exécution du prototype . . . 95
6.6.1 Démarrage du backend . . . 95
6.6.2 Démarrage de n8n . . . 95
6.6.3 Lancement de l'application Android . . . 95
6.7 Vérification opérationnelle du déploiement . . . 95
6.8 Incidents rencontrés et stabilisation . . . 96
6.9 Limites du déploiement actuel . . . 97
6.10 Perspective de mise en production . . . 98

Conclusion générale et perspectives 100

Bibliographie 102

A Éléments techniques complémentaires 103
A.1 Correspondance des vidéos expérimentales . . . 103

A.2 Procédure de reproduction 103
A.3 Checklist de validation d'une exécution 103

Table des figures

1.1 Architecture réseau interne de SFM Technologies (source : SFM Technologies) 4
1.2 Vue fonctionnelle de la solution proposée 10
1.3 Les six phases de la méthodologie CRISP-DM (adapté de [1]) 11
1.4 Diagramme de cas d'utilisation du système Vision-RAG 16

2.1 Illustration conceptuelle de la détection open-vocabulary, réalisée dans le cadre du projet à partir des principes décrits dans [2] 22
2.2 Illustration conceptuelle du fonctionnement de YOLO-World, réalisée dans le cadre du projet à partir de [3] 25
2.3 Illustration conceptuelle du suivi multi-objets, réalisée dans le cadre du projet à partir des principes décrits dans [4] 27
2.4 Architecture RAG appliquée au pipeline Vision-RAG SFM 35

3.1 Stratégie de recherche multimodale à trois niveaux 54
3.2 Réduction progressive du volume de données à chaque étape du pipeline 58

4.1 Architecture logicielle du système Vision-RAG 63
4.2 Diagramme d'activités du module de traitement vidéo 64
4.3 Diagramme de séquence du pipeline Vision-RAG complet. Les durées expérimentales sont présentées au chapitre suivant. 66
4.4 Diagramme de classes métier du backend Vision-RAG 67
4.5 Diagramme des composants du pipeline Vision-RAG et des services externes . 68
4.6 Vue logique de l'environnement de développement et de validation 70

5.1 Exemples de keyframes annotées constituant la vérité terrain 76
5.2 Comparaison qualitative de la vérité terrain et des prédictions : vrai positif, faux positif et faux négatif 79

6.1 Enchaînement opérationnel d'une analyse vidéo 91
6.2 Parcours principal dans l'application mobile 92
6.3 Capture ou sélection d'une vidéo dans l'application Flutter 93
6.4 Analyse, résultat et suivi des sessions dans l'application mobile 94
6.5 Workflow n8n mis en œuvre après l'analyse 94

Liste des tableaux

1.1 Besoins fonctionnels du système . . . 13
1.2 Besoins non fonctionnels . . . 14
1.3 Acteurs du système . . . 15
1.4 Description des cas d'utilisation . . . 16
1.5 Matrice de traçabilité entre besoins fonctionnels et cas d'utilisation . . . 17
1.6 Principaux risques du projet et mesures de maîtrise . . . 18

2.1 Comparaison des principales approches de détection d'objets . . . 25
2.2 Comparaison des méthodes de suivi multi-objets . . . 29
2.3 Comparaison des modèles multimodaux . . . 33
2.4 Comparaison des bases de données vectorielles . . . 37
2.5 Comparaison des technologies applicatives envisagées . . . 38
2.6 Technologies du prototype et perspective de déploiement . . . 39

3.1 Sources de données du projet . . . 40
3.2 Corpus de vidéos collectées . . . 41
3.3 Distinction entre catégories, équipements et crops . . . 41
3.4 Éléments documentaires collectés avant indexation . . . 42
3.5 Impact du seuil Laplacien sur la sélection des frames . . . 45
3.6 Résultats de la sélection de keyframes par vidéo . . . 46
3.7 Réduction des détections par ByteTrack sur l'ensemble du corpus . . . 46
3.8 Règles d'annotation du corpus d'évaluation . . . 48
3.9 Structure d'une fiche technique dans la base RAG . . . 48
3.10 Contenu du catalogue manuel de dix fiches techniques . . . 50
3.11 Caractéristiques du modèle all-MiniLM-L6-v2 . . . 51
3.12 Caractéristiques du modèle CLIP ViT-B/32 . . . 52
3.13 Résultats de la recherche multimodale par crop sur le corpus complet . . . 55
3.14 Structure du payload des deux collections Qdrant . . . 57
3.15 Bilan du pipeline de préparation des données visuelles . . . 58
3.16 État réel des données documentaires disponibles lors de la dernière exécution . . . 58
3.17 Empreinte mémoire des composantes de préparation . . . 59

4.1 Composants principaux du système Vision-RAG . . . 62
4.2 Correspondance entre les classes YOLO-World et les catégories SFM . . . 64
4.3 Mécanisme de nouvelle tentative après une erreur HTTP 429 . . . 66
4.4 Justification des trois mécanismes de stockage . . . 68
4.5 Champs du modèle Equipment et contraintes . . . 69
4.6 Champs de la réponse `POST /api/analyze-video` . . . 69
4.7 Ressources mémoire estimées par composant . . . 70
4.8 Glossaire des termes du pipeline Vision-RAG . . . 71

5.1 Niveaux d'évaluation du prototype . . . 73
5.2 Description du jeu de vidéos utilisé pour l'évaluation . . . 73

5.3 Critères retenus pour l'évaluation du prototype 74
5.4 Configuration matérielle utilisée pendant les tests 75
5.5 Configuration logicielle utilisée pendant les tests 75
5.6 Résultats de l'extraction et de la sélection des keyframes 76
5.7 Métriques de détection YOLO-World 77
5.8 Résultats de détection par type d'équipement 78
5.9 Métriques YOLO-World par classe représentée 78
5.10 Effet de la déduplication sur le nombre de crops retenus 79
5.11 Métrique de suivi ByteTrack 80
5.12 Qualité des informations retrouvées par le module RAG 80
5.13 Évaluation des champs des fiches `Equipment` 81
5.14 Résultats de la validation automatique des fiches 81
5.15 Temps moyen de traitement par étape du pipeline 82
5.16 Temps global de traitement par vidéo 82
5.17 Résultats du premier test séquentiel de robustesse 83
5.18 Résultats de la répétition du test de robustesse le 9 août 2026 83
5.19 Validation fonctionnelle sur le corpus réel 84
5.20 Tests fonctionnels complémentaires du backend 84
5.21 Résultats de validation humaine des fiches générées 85
5.22 Temps des trois exécutions avec et sans contexte RAG 86
5.23 Comparaison agrégée des fiches générées avec et sans contexte RAG 86

6.1 État d'implémentation des besoins fonctionnels 88
6.2 Matrice de conformité des besoins non fonctionnels 89
6.3 Environnement logiciel du prototype déployé 90
6.4 Services du backend et responsabilités associées 91
6.5 Écrans principaux de l'application Flutter 92
6.6 Vérifications opérationnelles réalisées 96
6.7 Principaux incidents corrigés pendant l'intégration 97
6.8 Limites du prototype et évolutions nécessaires 98

A.1 Correspondance entre les identifiants et les fichiers vidéo 103
A.2 Checklist minimale de validation expérimentale 104

Liste des Abréviations

ANN	Approximate Nearest Neighbors
API	Application Programming Interface
BTU	British Thermal Unit
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CRISP-DM	Cross Industry Standard Process for Data Mining
DTO	Data Transfer Object
ENICarthage	École Nationale d'Ingénieurs de Carthage
GPU	Graphics Processing Unit
GMAO	Gestion de Maintenance Assistée par Ordinateur
HNSW	Hierarchical Navigable Small World
HOTA	Higher Order Tracking Accuracy
IA	Intelligence Artificielle
IDF1	Identity F1 Score
JSON	JavaScript Object Notation
LLM	Large Language Model
mAP	mean Average Precision
ML	Machine Learning
MOTA	Multiple Object Tracking Accuracy
OCR	Optical Character Recognition
PFE	Projet de Fin d'Études
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SSOT	Single Source of Truth
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
VLM	Vision-Language Model
YOLO	You Only Look Once

Introduction générale

La transition énergétique pousse aujourd'hui les entreprises et les organismes publics à mieux suivre leur consommation électrique. Dans les bâtiments tertiaires — bureaux, hôtels, centres commerciaux, établissements d'enseignement ou entrepôts —, cette consommation dépend en grande partie des équipements installés et de leur mode d'utilisation. Les audits énergétiques permettent justement d'établir un état des lieux, d'identifier les postes les plus consommateurs et de proposer des actions d'amélioration adaptées.

SFM Technologies, entreprise spécialisée dans les réseaux de télécommunications, les technologies de l'information et les solutions d'ingénierie, accompagne ses clients dans ce type de mission. Lors d'un audit, l'identification des équipements électriques et la collecte de leurs caractéristiques constituent une étape importante. Elle est encore souvent réalisée manuellement par les techniciens, qui parcourent les locaux, prennent des notes, relèvent les informations visibles sur les appareils et complètent ensuite les documents nécessaires au rapport. Cette méthode reste efficace, mais elle demande du temps et dépend fortement de l'expérience de chaque intervenant.

Les progrès récents de la vision par ordinateur et des modèles multimodaux ouvrent des perspectives intéressantes pour assister ce travail de terrain. Des modèles comme **YOLO-World** peuvent détecter plusieurs catégories d'objets sans nécessiter un entraînement spécifique pour chaque classe. De leur côté, des modèles tels que **Gemini** permettent d'analyser une image en tenant compte d'un contexte textuel. En complément, une base vectorielle comme **Qdrant** peut servir à retrouver des informations techniques proches de l'équipement observé, dans une logique de génération augmentée par récupération (*Retrieval-Augmented Generation – RAG*).

C'est dans ce cadre que s'inscrit ce projet de fin d'études. L'objectif est de concevoir une solution capable d'exploiter une courte vidéo prise avec un smartphone, d'y repérer les équipements électriques présents, puis de produire des fiches structurées pouvant être utilisées dans un audit énergétique. La solution proposée associe vision par ordinateur, suivi d'objets, analyse multimodale et recherche d'informations techniques, avec une application mobile, un backend de traitement et des mécanismes d'automatisation.

La conduite du projet s'appuie sur la méthodologie **CRISP-DM** (*Cross Industry Standard Process for Data Mining*), largement adoptée dans les projets d'intelligence artificielle. Cette démarche structure le travail autour de plusieurs phases successives : compréhension du métier, compréhension des données, préparation des données, modélisation, évaluation et déploiement. Sur le plan du développement logiciel, une approche agile inspirée de **Scrum** a été retenue afin de favoriser une réalisation progressive des différentes composantes du système.

Le présent rapport est organisé en **six chapitres** :

— Le **chapitre 1** présente le contexte général du projet, l'organisme d'accueil, l'étude de l'existant ainsi que l'analyse des besoins fonctionnels et non fonctionnels ayant conduit à la définition de la solution proposée.

— Le **chapitre 2** expose l'état de l'art relatif aux technologies mobilisées dans ce travail, notamment la détection d'objets, les modèles vision-langage, le suivi d'objets, les systèmes de génération augmentée par récupération et les bases de données vectorielles. Une étude comparative des solutions existantes y est également présentée afin de justifier les choix technologiques retenus.

— Le **chapitre 3** est consacré à la compréhension et à la préparation des données. Il décrit les différentes étapes de collecte, de prétraitement et d'organisation des données utilisées pour la conception et l'évaluation du système, ainsi que la constitution de la base documentaire exploitée par le mécanisme RAG.

— Le **chapitre 4** détaille la conception de la solution proposée. Il présente l'architecture globale du système, les différents composants logiciels, les interactions entre les modules ainsi que les diagrammes de conception associés.
— Le **chapitre 5** présente les expérimentations réalisées et l'évaluation du système. Il précise le protocole de test, les vidéos utilisées, les résultats obtenus ainsi que les principales limites observées.
— Le **chapitre 6** décrit le déploiement de la solution. Il présente l'intégration du backend, de l'application mobile Flutter, des workflows d'automatisation n8n et des services externes mobilisés.

Le rapport s'achève par une **conclusion générale** récapitulant les principales contributions du projet et proposant plusieurs pistes d'évolution pour enrichir et étendre la solution développée.

Chapitre 1

Cadre du projet et analyse des besoins

Introduction

Ce chapitre a pour objectif de présenter le contexte général dans lequel s'inscrit ce projet de fin d'études. Après une présentation de l'organisme d'accueil SFM Technologies, nous étudierons le contexte métier associé aux audits énergétiques ainsi que les pratiques actuellement mises en œuvre pour l'identification des équipements électriques. L'analyse critique de l'existant permettra ensuite de mettre en évidence les limites rencontrées et de formuler la problématique du projet. Enfin, nous présenterons la solution proposée ainsi que les besoins fonctionnels et non fonctionnels auxquels elle devra répondre, avant d'identifier les principaux risques susceptibles d'affecter sa réalisation ou son utilisation.

1.1 Présentation de l'organisme d'accueil

Avant d'aborder les aspects techniques du projet, il convient de présenter l'environnement professionnel dans lequel celui-ci a été réalisé.

1.1.1 Présentation générale de SFM Technologies

Services for Fixed and Mobile Telecommunications Network and Systems (SFM Technologies) est un cabinet d'ingénierie et d'expertise spécialisé dans les réseaux de télécommunications et les technologies de l'information. Créée en 1995 par un groupe d'ingénieurs et de consultants, l'entreprise accompagne les opérateurs de télécommunications, les organismes de régulation ainsi que les institutions publiques dans la conception, l'optimisation et la transformation de leurs infrastructures numériques.

Membre de l'ITU-D (International Telecommunication Union – Development Sector), SFM Technologies s'est progressivement imposée comme un acteur reconnu dans le domaine des télécommunications en Afrique. Le groupe est aujourd'hui présent dans plus de vingt-quatre pays répartis sur quatre continents : l'Afrique, l'Europe, l'Asie et l'Amérique du Nord.

Grâce à son expertise multidisciplinaire, l'entreprise intervient aussi bien dans les domaines du conseil stratégique que dans ceux de l'ingénierie, de la recherche et du développement technologique.

1.1.2 Mission, vision et valeurs

La mission de SFM Technologies consiste à contribuer à l'amélioration de la connectivité numérique, à accompagner la transformation digitale de ses clients et à promouvoir la modernisation des processus métiers. L'entreprise ambitionne de devenir un acteur majeur de l'évolution digitale à l'échelle internationale.

Afin d'atteindre ces objectifs, SFM Technologies s'appuie sur plusieurs valeurs fondamentales :

— le respect ;
— l'innovation ;
— l'intelligence collective ;
— la motivation ;
— la valorisation des compétences.

Ces valeurs guident l'ensemble des activités de l'entreprise et favorisent une culture orientée vers l'excellence, l'innovation et l'amélioration continue.

1.1.3 Domaines d'activité

Les activités de SFM Technologies s'articulent autour de quatre axes principaux.

1. **Expertise technique**

   SFM accompagne les opérateurs et autorités de régulation dans l'audit des réseaux de télécommunications, la planification et l'optimisation des infrastructures, l'analyse du trafic ainsi que la gestion du spectre de fréquences.

2. **Conseil stratégique**

   L'entreprise réalise des études et missions de conseil portant sur la réglementation, la transformation numérique, la migration vers de nouvelles technologies de communication et la gestion des ressources stratégiques du secteur des télécommunications.

3. **Recherche et Développement**

   À travers son laboratoire d'innovation SFM Lab, l'entreprise mène des travaux de recherche dans plusieurs domaines tels que l'analyse de la qualité de service, les modèles de propagation, les outils de mesure terrain et l'intelligence artificielle appliquée.

4. **Formation**

   SFM organise régulièrement des formations spécialisées destinées aux professionnels du secteur afin de favoriser le transfert de compétences et l'adoption des nouvelles technologies.

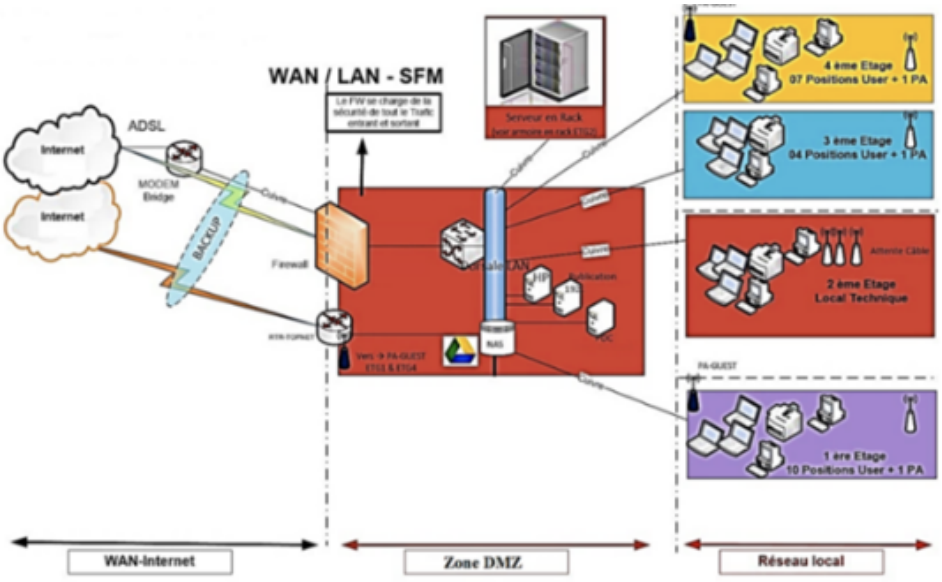


FIGURE 1.1 – Architecture réseau interne de SFM Technologies (source : SFM Technologies)

1.1.4 Département d'accueil

Le présent projet a été réalisé au sein de **SFM Lab**, le centre de recherche et développement de SFM Technologies. Cette structure a pour mission de concevoir des solutions innovantes répondant aux besoins des clients de l'entreprise dans les domaines des télécommunications, des technologies de l'information et de la transformation numérique.

SFM Lab constitue un espace dédié à l'innovation, à la recherche appliquée et au développement de nouvelles solutions technologiques. Les activités qui y sont menées couvrent notamment

la conception de plateformes logicielles, l'analyse de données, l'automatisation des processus métiers ainsi que l'intégration de technologies émergentes dans les solutions proposées par l'entreprise.

Dans le cadre de sa stratégie d'innovation continue, SFM Lab encourage la réalisation de projets de recherche et de développement permettant d'explorer de nouvelles approches technologiques et d'apporter des solutions adaptées aux problématiques rencontrées par ses clients.

C'est au sein de cet environnement orienté vers l'innovation et la recherche appliquée qu'a été réalisé le présent projet de fin d'études.

Après avoir présenté l'entreprise et son environnement d'activité, il est nécessaire de s'intéresser au contexte métier ayant motivé la réalisation de ce projet.

1.2 Contexte métier et étude de l'existant

Cette section présente le cadre métier du projet ainsi que les méthodes actuellement utilisées pour la réalisation des audits énergétiques.

1.2.1 Les audits énergétiques des bâtiments tertiaires

Définition et objectifs

Un audit énergétique est une démarche structurée visant à analyser les usages et les consommations d'énergie d'un bâtiment, d'un procédé ou d'une organisation, puis à identifier des actions d'amélioration de la performance énergétique. La norme ISO 50002-1 :2025 définit les exigences générales et les principes applicables à cette démarche, tandis que la norme ISO 50002-2 :2025 précise leur application aux bâtiments [5, 6].

Dans le secteur tertiaire — bureaux, administrations, hôtels, commerces ou entrepôts — l'audit permet de relier les consommations observées aux usages réels du site. Il poursuit notamment trois objectifs :

- **établir un état des lieux** en recensant les équipements et les principaux usages énergétiques du bâtiment ;
- **caractériser les consommations** à partir des puissances, des durées d'utilisation, des mesures disponibles et des conditions d'exploitation ;
- **identifier des possibilités d'amélioration** telles que le réglage, la maintenance, le remplacement d'un équipement ou l'adaptation de son mode d'utilisation.

L'enjeu est significatif : selon l'Agence internationale de l'énergie, les bâtiments représentent environ 30 % de la demande énergétique mondiale. Les bâtiments commerciaux et publics concentrent approximativement 30 % de la demande propre au secteur des bâtiments [7]. L'amélioration de leur efficacité constitue donc un levier important de maîtrise de la consommation énergétique.

Processus général d'un audit énergétique

Le déroulement précis dépend du niveau de détail recherché et des informations disponibles. Dans le cadre de ce projet, le processus général est synthétisé en cinq phases complémentaires :

1. **préparation de la mission** : définition du périmètre, collecte des plans, factures, historiques et contrats de maintenance, puis organisation de la visite ;
2. **visite du site** : inspection des locaux, observation des conditions d'exploitation et repérage des équipements consommateurs ;
3. **collecte des données techniques** : relevé des informations visibles ou mesurables, notamment la catégorie, la marque, la puissance et le mode d'utilisation ;

4. **analyse énergétique** : rapprochement des relevés, des mesures et des durées d'usage afin d'estimer les consommations et de repérer les postes prioritaires ;
5. **rapport et recommandations** : présentation des constats, des hypothèses, des actions proposées et, lorsque les données le permettent, de leur impact énergétique estimé.

Les étapes de visite et de collecte exigent une attention particulière, car elles conditionnent la qualité des calculs réalisés par la suite. Leur charge augmente avec la superficie du site, le nombre de locaux et la diversité des équipements à inventorier.

Importance de l'inventaire des équipements

L'inventaire constitue le socle factuel de l'analyse énergétique. Une omission, une mauvaise catégorie ou une puissance incorrecte peut se répercuter sur l'estimation des consommations et sur la pertinence des recommandations.

Selon les objectifs de la mission et la disponibilité des plaques signalétiques, un inventaire peut inclure la désignation, la catégorie, la marque, le modèle, la puissance nominale, la tension d'alimentation, le mode de fonctionnement et la localisation. Pour un climatiseur, la capacité de refroidissement peut également être exprimée en BTU/h. Certaines informations, comme le numéro de série ou le modèle exact, ne sont toutefois pas toujours lisibles ou nécessaires à la première estimation énergétique.

Dans ce rapport, les informations produites par le prototype sont regroupées dans une fiche structurée `Equipment`. Son périmètre est volontairement plus restreint que celui d'un inventaire patrimonial exhaustif : il se concentre sur les champs utiles au traitement développé et signale les résultats incertains à l'aide de `needsReview`. La définition technique complète de ce modèle est présentée au chapitre 4.

1.2.2 Processus actuel chez SFM Technologies

Le processus de référence étudié au cours du projet repose principalement sur une collecte réalisée par un technicien pendant la visite du bâtiment. Les pratiques peuvent varier selon le client, le type de site et le niveau de détail attendu ; la description suivante correspond au scénario métier retenu pour concevoir et évaluer le prototype.

Collecte des données sur le terrain

Le technicien parcourt les différents espaces du site — bureaux, salles techniques, cuisines, entrepôts ou couloirs — et consigne les équipements visibles dans un formulaire papier ou numérique. Il doit couvrir méthodiquement les locaux tout en évitant les omissions et les doubles comptages.

Lorsque le site comporte de nombreux locaux, cette opération peut mobiliser plusieurs heures ou plusieurs journées. Elle doit en outre être réalisée sans perturber les occupants ni interrompre le fonctionnement normal des équipements.

Identification des équipements

Pour chaque appareil repéré, le technicien recherche les informations visibles sur la façade ou sur la plaque signalétique. Il peut photographier l'étiquette constructeur et relever la marque, le modèle, la puissance ou toute autre caractéristique utile à la mission. Les équipements rencontrés peuvent être, par exemple, des climatiseurs, des ordinateurs, des téléviseurs, des réfrigérateurs, des fours à micro-ondes ou des commutateurs réseau (*switches*).

L'identification devient plus incertaine lorsque l'étiquette est absente, dégradée, mal éclairée ou difficilement accessible. Le technicien doit alors s'appuyer sur les informations partielles disponibles et sur son expérience, en indiquant les hypothèses qui nécessitent une vérification ultérieure.

Production des fiches techniques

Les relevés sont ensuite regroupés dans des fiches techniques ou des tableaux d'inventaire. Selon les outils utilisés pendant la mission, la saisie peut être effectuée directement sur le terrain ou reprise au bureau à partir des notes et des photographies collectées.

Ces fiches servent de base à l'estimation des consommations, à la comparaison des équipements et à la préparation des recommandations. Elles peuvent aussi être exportées vers les systèmes d'information du client ou vers un logiciel de gestion de maintenance assistée par ordinateur (GMAO). Leur exploitation suppose toutefois que les désignations, les unités et les champs obligatoires soient suffisamment homogènes.

1.2.3 Analyse critique de l'existant

L'étude du processus actuellement utilisé permet d'identifier plusieurs limites susceptibles d'affecter l'efficacité et la qualité des audits.

Processus chronophage

Le recensement manuel exige de parcourir l'ensemble des locaux, d'observer chaque appareil, de rechercher son étiquette et de reporter les informations dans le support de collecte. La charge augmente rapidement avec le nombre d'équipements, les difficultés d'accès et le niveau de détail demandé. À cette visite s'ajoute souvent un travail de classement des photographies et de retranscription au bureau.

Cette mobilisation réduit le temps disponible pour les tâches à plus forte valeur ajoutée, notamment l'analyse des usages, la vérification des hypothèses et la formulation des recommandations.

Risques d'erreurs humaines

La saisie manuelle expose le processus à plusieurs catégories d'erreurs :

— les **erreurs de lecture**, lorsque la plaque signalétique est peu visible ou comporte des références proches ;
— les **erreurs de saisie**, notamment sur les valeurs numériques et les unités ;
— les **omissions**, lorsqu'un équipement est peu accessible ou dissimulé dans la scène ;
— les **doubles comptages**, lorsqu'un même appareil est observé depuis plusieurs positions ou à différents moments de la visite ;
— les **estimations non vérifiées**, lorsque les caractéristiques sont déduites sans document constructeur suffisamment fiable.

Individuellement limitées, ces erreurs peuvent se cumuler à l'échelle d'un site et affecter la cohérence de l'inventaire final.

Manque de standardisation

En l'absence de règles de saisie uniformes, les désignations, les unités et le niveau de détail peuvent varier d'un opérateur ou d'une mission à l'autre. Un même appareil peut ainsi être décrit par plusieurs termes, tandis que les valeurs de puissance peuvent être exprimées en watts ou en kilowatts.

Cette hétérogénéité complique la consolidation de plusieurs inventaires, la comparaison entre sites et l'automatisation des traitements ultérieurs. Elle justifie l'utilisation d'un modèle de données contrôlé et de listes de valeurs autorisées.

Dépendance à l'expertise des techniciens

La qualité des relevés dépend en partie de l'expérience du technicien et de sa connaissance des familles d'équipements. Une personne expérimentée reconnaît plus rapidement les indices

visuels pertinents et identifie les informations à vérifier. Une personne moins familière avec certains appareils peut employer une catégorie trop générale ou omettre une caractéristique importante.

L'objectif du système proposé n'est pas de supprimer cette expertise, mais de la soutenir par une première détection, une fiche préstructurée et un signalement explicite des informations incertaines.

Difficulté d'exploitation des données collectées

Les informations collectées dans des notes libres, des photographies ou des tableurs hétérogènes nécessitent fréquemment une phase de nettoyage avant leur utilisation. Les désignations doivent être harmonisées, les unités converties, les doublons supprimés et les champs manquants identifiés.

Ce travail limite l'intégration directe des résultats dans les systèmes d'information, les outils d'analyse énergétique ou les logiciels de GMAO. La production immédiate d'objets JSON conformes à un schéma métier constitue donc un apport important de la solution proposée.

Les limites observées montrent que les méthodes actuellement utilisées atteignent rapidement leurs limites lorsque le nombre d'équipements à analyser devient important. L'évolution récente des technologies d'intelligence artificielle ouvre cependant de nouvelles perspectives pour automatiser certaines étapes de ce processus.

1.3 Problématique et solution proposée

Cette section vise à formaliser le problème traité dans le cadre du projet et à présenter l'approche retenue pour y répondre.

1.3.1 Positionnement du problème

Bien que les procédures actuellement utilisées permettent de réaliser les audits énergétiques, elles reposent essentiellement sur des tâches manuelles nécessitant du temps et une expertise importante. Dans un contexte où les entreprises recherchent davantage d'efficacité et de traçabilité, l'automatisation de ces opérations apparaît comme une évolution naturelle.

1.3.2 Problématique

La problématique abordée dans ce projet peut être formulée comme suit :

> Comment concevoir une solution intelligente capable d'identifier automatiquement les équipements électriques présents dans un bâtiment tertiaire à partir d'une simple séquence vidéo, tout en produisant des fiches techniques conformes aux exigences métier de SFM Technologies ?

Cette problématique soulève plusieurs défis techniques :
— la détection automatique des équipements présents dans une scène ;
— le suivi cohérent des équipements au sein d'une séquence vidéo ;
— la génération automatique d'informations techniques exploitables ;
— l'exploitation des connaissances métier de l'entreprise ;
— l'intégration de l'ensemble du processus dans une application accessible aux techniciens sur le terrain.

1.3.3 Solution proposée

La solution proposée prend la forme d'un assistant mobile de recensement des équipements. Son objectif n'est pas de remplacer le technicien, mais de réduire le temps consacré aux tâches répétitives : sélectionner les images utiles, repérer les appareils visibles, éviter de compter plusieurs fois le même objet et préparer une première version de la fiche technique. Le technicien conserve la responsabilité de vérifier les informations avant leur archivage ou leur utilisation dans un rapport d'audit.

Principe général

Le point de départ est une courte vidéo enregistrée avec un smartphone pendant la visite d'un local. Cette vidéo constitue une entrée plus naturelle qu'une série de photographies imposées : le technicien peut parcourir la pièce et filmer les équipements sans interrompre systématiquement son inspection. Le backend transforme ensuite cette séquence en un nombre limité d'images exploitables, puis applique successivement les traitements de vision, de recherche documentaire et de génération structurée.

La réponse retournée n'est pas un simple texte libre. Pour chaque équipement retenu, le système construit une fiche `Equipment` comprenant notamment la désignation, la marque lorsqu'elle est visible, l'état de fonctionnement estimé et les principales caractéristiques techniques disponibles. Les champs peu fiables ou incomplets sont signalés afin d'attirer l'attention de l'utilisateur.

Chaîne de traitement fonctionnelle

Le traitement est organisé en sept étapes complémentaires :

1. **acquisition de la vidéo** : l'application Flutter permet de filmer la scène ou de sélectionner une vidéo existante, puis de la transmettre au backend avec la localisation de l'audit ;
2. **sélection des keyframes** : OpenCV élimine les images trop floues et limite la redondance afin de ne conserver qu'un ensemble représentatif de la séquence ;
3. **détection des équipements** : YOLO-World localise les objets correspondant au vocabulaire métier configuré sans nécessiter un nouvel entraînement pour chaque terme ajouté ;
4. **suivi et déduplication** : ByteTrack associe les détections successives et sélectionne un crop représentatif par trajectoire, ce qui réduit le risque de produire plusieurs fiches pour un même appareil ;
5. **recherche contextuelle** : Qdrant récupère les documents les plus proches du crop ou de sa description afin de fournir à Gemini un contexte issu de la base de connaissances ;
6. **génération et validation** : Gemini analyse les crops et le contexte documentaire, puis Pydantic vérifie que la réponse respecte la structure et les règles du modèle `Equipment` ;
7. **validation métier et archivage** : les fiches sont affichées dans l'application pour confirmation ou correction. Après validation, le workflow n8n peut assurer l'archivage dans Supabase et la production d'un rapport récapitulatif.

La figure 1.2 résume ce parcours. Elle présente la logique métier générale ; l'architecture logicielle et les échanges entre services seront détaillés au chapitre 4.

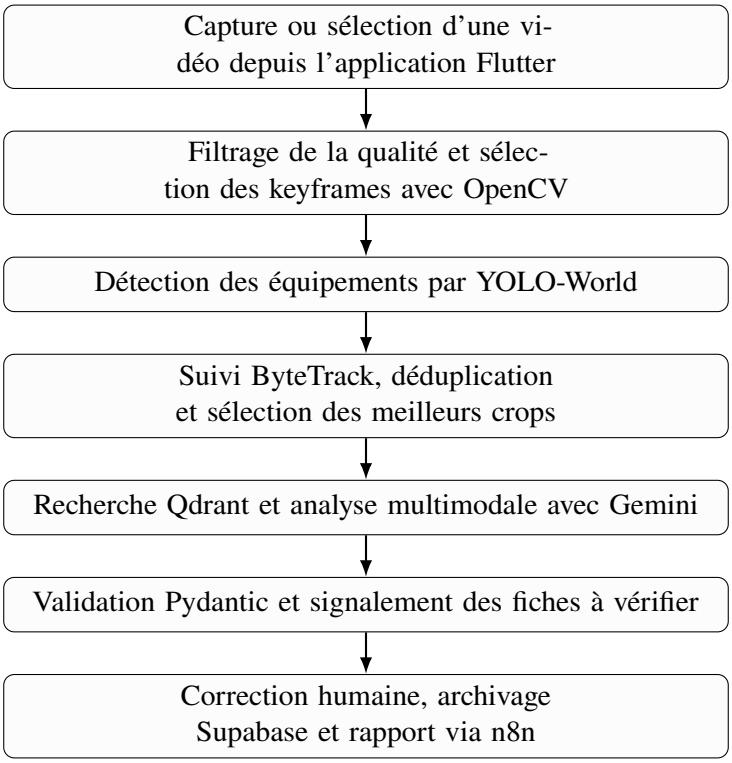


FIGURE 1.2 – Vue fonctionnelle de la solution proposée

Rôle des technologies retenues

Chaque technologie répond à une responsabilité précise dans cette chaîne :

- **YOLO-World** assure la détection *open-vocabulary* des équipements à partir de prompts textuels configurables [3] ;
- **ByteTrack** maintient la continuité des objets entre les keyframes et contribue à la suppression des doublons [8] ;
- **Gemini** interprète les images recadrées et génère les valeurs nécessaires à la fiche structurée [9] ;
- **Qdrant** fournit la recherche vectorielle utilisée par le mécanisme RAG pour rapprocher les observations des documents métier [10] ;
- **Flutter** fournit une interface mobile adaptée à la capture, à la consultation et à la correction des résultats [11] ;
- **n8n** prend en charge les opérations postérieures à l'analyse, notamment la persistance et la génération du rapport [12].

Apports attendus et périmètre

La solution vise trois apports principaux. Le premier est un gain de temps sur le terrain et lors de la préparation du rapport. Le deuxième est une meilleure homogénéité des fiches, puisque les résultats suivent un schéma commun au backend, à l'application et à la base de données. Le troisième est une meilleure traçabilité : les informations proposées peuvent être relues, corrigées et archivées au lieu de rester dispersées entre des notes et des photographies.

Le périmètre reste toutefois celui d'un prototype d'assistance. La détection peut manquer un objet ou produire une mauvaise catégorie, tandis que les informations générées par le modèle multimodal peuvent être incomplètes. La validation humaine constitue donc une étape normale du fonctionnement et non un traitement exceptionnel réservé aux erreurs. L'export vers la GMAO, la notification mobile complète et les mécanismes de sécurité d'un déploiement public relèvent des évolutions ultérieures.

La section suivante précise les besoins auxquels cette solution doit répondre afin de satisfaire les attentes métier identifiées.

1.4 Méthodologie adoptée

La réalisation d'un projet d'intelligence artificielle nécessite l'adoption d'une démarche méthodologique rigoureuse permettant d'assurer la cohérence des différentes phases du projet, depuis l'identification du besoin métier jusqu'au déploiement de la solution. Dans ce cadre, le présent projet s'appuie sur la méthodologie **CRISP-DM** (*Cross Industry Standard Process for Data Mining*) [1], largement reconnue comme une référence dans la conduite des projets de science des données et d'intelligence artificielle.

1.4.1 Présentation de la méthodologie CRISP-DM

CRISP-DM est une méthodologie itérative qui structure le développement d'un projet d'intelligence artificielle en six phases principales :

1. Compréhension du métier (*Business Understanding*) ;
2. Compréhension des données (*Data Understanding*) ;
3. Préparation des données (*Data Preparation*) ;
4. Modélisation (*Modeling*) ;
5. Évaluation (*Evaluation*) ;
6. Déploiement (*Deployment*).

Cette approche favorise une meilleure prise en compte des besoins métiers tout en garantissant une démarche structurée pour la conception et l'évaluation des solutions basées sur l'intelligence artificielle.

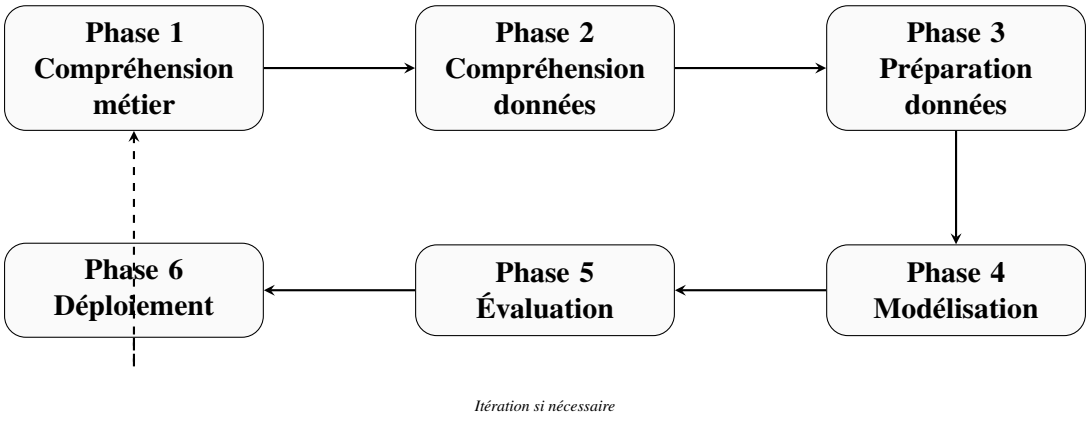


FIGURE 1.3 – Les six phases de la méthodologie CRISP-DM (adapté de [1])

1.4.2 Application de CRISP-DM au projet

Dans le cadre de ce projet, les différentes phases de CRISP-DM ont été adaptées aux spécificités de la problématique traitée.

La phase de compréhension du métier a permis d'analyser le processus d'audit énergétique utilisé par SFM Technologies et d'identifier les limitations liées à l'identification manuelle des équipements électriques.

La compréhension des données a d'abord consisté à décrire les vidéos et les documents collectés, à examiner leur diversité et à relever les principaux problèmes de qualité. La préparation

a ensuite porté sur le contrôle des fichiers, le filtrage des images floues, la sélection des keyframes, le nettoyage documentaire et la constitution de la base utilisée par le mécanisme de génération augmentée par récupération (RAG). Ces deux phases sont présentées dans un même chapitre afin de montrer directement comment les constats exploratoires ont motivé les traitements appliqués, tout en restant distinguées dans son plan.

La phase de modélisation a été adaptée au caractère intégrateur du projet. Aucun modèle n'a été entraîné à partir de zéro : cette phase a consisté à configurer et à assembler des modèles pré-entraînés, puis à concevoir leurs interactions avec le backend et la base vectorielle.

La phase d'évaluation a porté sur l'analyse des performances de la solution développée à travers différentes métriques adaptées aux tâches de détection, de suivi et de génération d'informations.

Enfin, la phase de déploiement concerne l'intégration de la solution dans une architecture opérationnelle comprenant une application mobile, un backend de traitement et des mécanismes d'automatisation.

1.4.3 Organisation du projet

En complément de la méthodologie CRISP-DM, une approche agile inspirée de **Scrum** a été adoptée pour l'organisation du travail. Le projet a été découpé en plusieurs itérations permettant de développer, tester et améliorer progressivement les différentes composantes du système.

Cette organisation a favorisé une meilleure prise en compte des retours de l'encadrement ainsi qu'une validation progressive des fonctionnalités développées.

Après avoir présenté la méthodologie adoptée pour la conduite du projet, la section suivante identifie les besoins fonctionnels et non fonctionnels auxquels la solution proposée devra répondre.

1.5 Analyse des besoins

Cette section présente les besoins fonctionnels et non fonctionnels qui serviront de référence pour la conception et l'évaluation de la solution.

1.5.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit fournir aux utilisateurs. Ci-dessous un tableau décrivant ces besoins retenus pour ce projet.

TABLE 1.1 – Besoins fonctionnels du système

ID	Description
BF1	Permettre au technicien de capturer ou sélectionner une vidéo depuis l'application mobile.
BF2	Transmettre la vidéo au serveur d'analyse de manière sécurisée.
BF3	Extraire automatiquement les images représentatives (keyframes) de la vidéo.
BF4	Détecter automatiquement les équipements électriques présents dans les images à l'aide d'un modèle de vision artificielle.
BF5	Assurer le suivi multi-objets afin d'éviter les doublons lorsqu'un même équipement apparaît sur plusieurs images.
BF6	Identifier les équipements détectés à l'aide d'une base de connaissances métier.
BF7	Générer automatiquement une fiche technique structurée pour chaque équipement identifié.
BF8	Permettre la validation et la modification manuelle des fiches générées.
BF9	Archiver les fiches validées dans la base de données centrale.
BF10	Notifier l'utilisateur à la fin du traitement.
BF11	Produire automatiquement un rapport récapitulatif exploitable dans le cadre d'un audit énergétique.
BF12	Exporter les données vers les systèmes métiers de SFM Technologies.

1.5.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité et de performance auxquelles le système doit satisfaire.

TABLE 1.2 – Besoins non fonctionnels

ID	Exigence	Critère de validation
BNF1	Performance de détection	Atteindre un mAP@50 supérieur ou égal à 0,70 sur un sous-ensemble de frames annotées manuellement avec des boîtes englobantes de référence. Le nombre de frames et les règles d'annotation doivent être consignés dans le protocole expérimental.
BNF2	Performance de suivi	Atteindre un score IDF1 supérieur à 70% sur des séquences pour lesquelles l'identité de chaque équipement est annotée manuellement sur les frames évaluées.
BNF3	Temps de réponse	Temps total d'analyse inférieur à 60 secondes pour une vidéo de moins de 30 secondes.
BNF4	Fiabilité du backend	Terminer avec succès au moins 95% d'une série d'au moins 20 requêtes d'analyse consécutives, sans arrêt ni redémarrage du backend. Les échecs liés à un quota externe doivent être distingués des erreurs internes.
BNF5	Maintenabilité	Implémenter les six étapes principales du pipeline dans des modules ou services distincts. L'exigence est satisfaite si les six éléments de la checklist d'architecture sont validés : traitement vidéo, détection, suivi, RAG, analyse multimodale et validation, chacun avec une responsabilité et une interface publique identifiées.
BNF6	Sécurité	Toutes les communications entre l'application mobile et le backend doivent être chiffrées via HTTPS.
BNF7	Robustesse d'exécution	Traiter au moins 10 demandes successives sans redémarrage du système, chaque demande devant atteindre un état final de succès ou d'échec explicitement remonté à l'utilisateur.
BNF8	Utilisabilité	Permettre l'accès aux résultats en trois actions principales au maximum : sélectionner ou capturer la vidéo, lancer l'analyse, puis consulter les résultats. La validation est accessible directement depuis l'écran de résultat.

Les seuils de performance retenus s'appuient sur les métriques couramment utilisées dans la littérature scientifique pour l'évaluation des systèmes de détection et de suivi d'objets. Leur calcul nécessite cependant une vérité terrain annotée manuellement. Le protocole de constitution de ce sous-ensemble annoté est donc présenté avec la préparation des données, puis exploité dans

le chapitre d'évaluation.

Le mAP@50 constitue la métrique de référence pour les tâches de détection d'objets et est notamment utilisé dans le benchmark COCO (Common Objects in Context) [13].

Le score IDF1 est quant à lui largement employé dans les compétitions MOTChallenge pour évaluer la cohérence du suivi multi-objets [14].

Le temps maximal d'analyse a été défini en concertation avec les techniciens de SFM Technologies afin de garantir une utilisation acceptable sur le terrain.

1.5.3 Contraintes techniques

Le développement de la solution est soumis à plusieurs contraintes :

— CT1 : utilisation obligatoire de smartphones Android pour la collecte terrain ;
— CT2 : recours à des API cloud pour les modèles multimodaux ;
— CT3 : compatibilité avec le référentiel Equipment de SFM Technologies ;
— CT4 : exécution sur une machine standard sans GPU dédié ;
— CT5 : respect des quotas imposés par l'API Gemini ;
— CT6 : stockage local de la base vectorielle Qdrant.

1.5.4 Acteurs du système

Le système met en interaction plusieurs acteurs.

TABLE 1.3 – Acteurs du système

Acteur	Rôle
Technicien	Capture les vidéos et consulte les résultats.
Administrateur	Consulte les données archivées, exporte les fiches et génère les rapports nécessaires au suivi des audits.

1.5.5 Cas d'utilisation

Afin de mieux représenter les interactions entre les acteurs et le système, sept cas d'utilisation principaux ont été retenus. Les traitements techniques tels que l'extraction des keyframes, la détection, le suivi, la recherche RAG et la génération de la fiche ne sont pas des objectifs autonomes d'un acteur : ils sont exécutés par le système dans le cadre du lancement d'une analyse.

TABLE 1.4 – Description des cas d'utilisation

ID	Cas d'utilisation	Description
UC1	S'authentifier	Ouvrir une session sécurisée dans l'application.
UC2	Capturer ou sélectionner une vidéo	Préparer la vidéo qui sera transmise au backend.
UC3	Lancer une analyse	Envoyer la vidéo et déclencher l'ensemble du pipeline Vision-RAG.
UC4	Consulter les résultats	Afficher les équipements détectés et les fiches générées.
UC5	Valider ou corriger les fiches	Confirmer les informations proposées ou modifier les champs incorrects.
UC6	Exporter les données	Transmettre les fiches validées vers les systèmes métiers.
UC7	Générer un rapport	Produire un récapitulatif des équipements validés pour l'audit.

L'authentification constitue une précondition pour les cas d'utilisation UC2 à UC7. Le technicien réalise le parcours complet de collecte et de validation. L'administrateur peut consulter les données archivées, les exporter et générer les rapports nécessaires au suivi des audits.

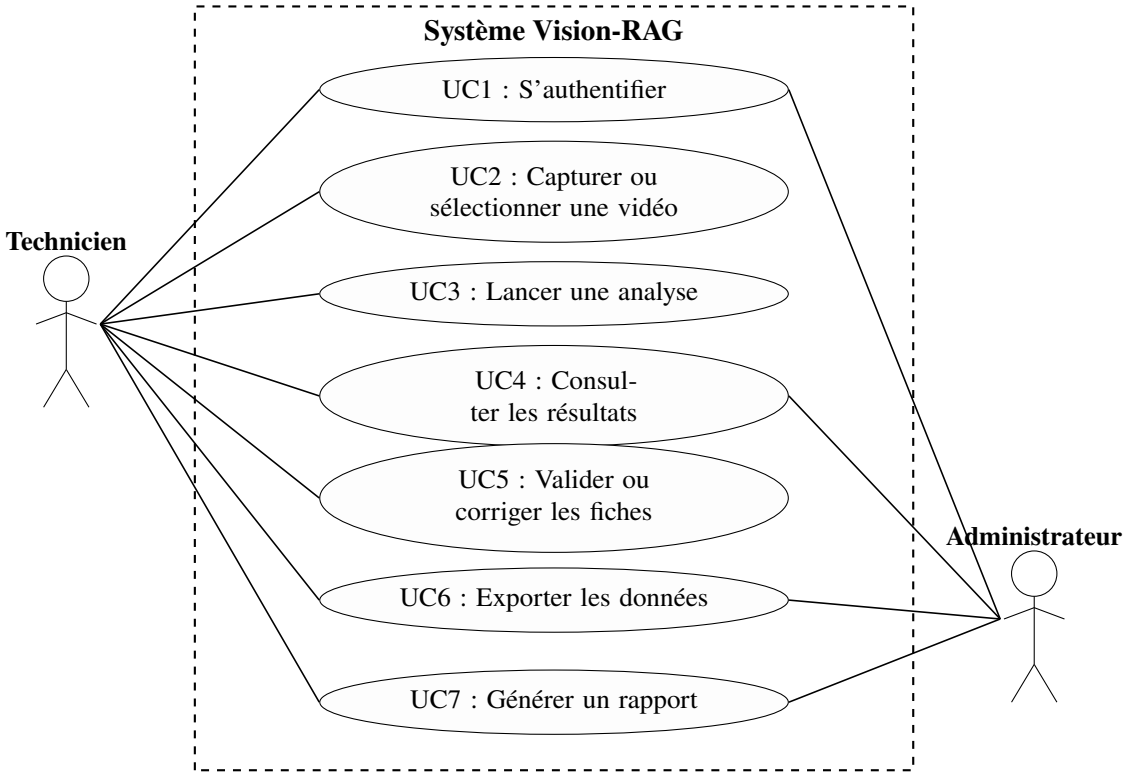


FIGURE 1.4 – Diagramme de cas d'utilisation du système Vision-RAG

1.5.6 Matrice de traçabilité des besoins

La matrice de traçabilité relie chaque besoin fonctionnel à un cas d'utilisation ou à un sous-processus interne. Elle permet de vérifier qu'aucun besoin n'est isolé de l'analyse fonctionnelle.

TABLE 1.5 – Matrice de traçabilité entre besoins fonctionnels et cas d'utilisation

Besoin	**UC associé**	**Justification**
BF1	UC2	Capture ou sélection de la vidéo par le technicien.
BF2	UC3	Transmission sécurisée lors du lancement de l'analyse.
BF3	UC3	Extraction interne des keyframes après réception de la vidéo.
BF4	UC3	Détection interne des équipements sur les keyframes retenues.
BF5	UC3	Suivi et déduplication internes des équipements détectés.
BF6	UC3	Recherche interne d'informations dans la base de connaissances.
BF7	UC3	Génération interne d'une fiche structurée par équipement.
BF8	UC5	Validation et correction manuelle des fiches.
BF9	UC5	Archivage réalisé après validation de la fiche.
BF10	UC3	Notification envoyée à la fin du traitement.
BF11	UC7	Production du rapport récapitulatif.
BF12	UC6	Export vers les systèmes métiers de SFM Technologies.

1.6 Analyse des risques

L'identification des risques complète l'analyse des besoins. Elle permet de repérer, dès le cadrage, les événements susceptibles de réduire la qualité des résultats, de retarder le traitement ou de limiter l'utilisation du prototype. L'appréciation retenue est qualitative : la probabilité et l'impact sont classés selon trois niveaux — faible, moyen ou élevé — à partir des conditions d'utilisation prévues et des observations effectuées pendant le développement.

Le tableau 1.6 regroupe les principaux risques techniques, fonctionnels et organisationnels du projet. Les mesures indiquées correspondent soit à des mécanismes déjà intégrés au prototype, soit à des actions nécessaires avant une mise en production.

TABLE 1.6 – Principaux risques du projet et mesures de maîtrise

Risque	Probabilité	Impact	Mesures de maîtrise
Vidéo floue, sombre ou trop rapide	Élevée	Élevé	Filtrage par netteté, sélection de keyframes et consignes de prise de vue ; possibilité de refaire la capture.
Équipement manqué ou mal classé	Élevée	Élevé	Prompts métier, seuils configurables, évaluation sur annotations manuelles et confirmation par le technicien.
Réponse Gemini incomplète ou incorrecte	Moyenne	Élevé	Prompt structuré, contexte RAG, reprises sur erreur, validation Pydantic et signalement par `needsReview`.
Base documentaire insuffisante ou obsolète	Élevée	Moyen	Utilisation de sources vérifiées, conservation de la provenance et enrichissement progressif par des documents constructeurs.
Indisponibilité ou quota d'un service cloud	Moyenne	Élevé	Gestion des délais et des erreurs, nombre d'appels limité, journalisation et étude d'un mode dégradé pour les fonctions non essentielles.
Temps de traitement trop long sur CPU	Moyenne	Moyen	Réduction du nombre de keyframes, traitement par lots et possibilité d'utiliser un modèle plus léger ou une machine équipée d'un GPU.
Indisponibilité de n8n ou de Supabase	Moyenne	Moyen	Webhook non bloquant pour l'analyse principale, reprise ultérieure des opérations d'archivage et contrôle explicite de leur état.
Divulgation d'images ou de données d'audit	Moyenne	Élevé	Limitation du prototype au réseau local. Avant la production : suppression automatique des fichiers temporaires, HTTPS, authentification et contrôle des accès.
Corpus de validation peu représentatif	Élevée	Élevé	Présenter les résultats comme une validation initiale et élargir les essais à plusieurs bâtiments, appareils et conditions de capture.

Les risques les plus critiques concernent la qualité des vidéos, les erreurs de détection et la représentativité du corpus. Ils ont un effet direct sur la fiabilité des fiches produites. Le maintien d'une validation humaine est donc la principale mesure de sécurité fonctionnelle du prototype. Les risques liés aux services cloud et à la sécurité deviennent prioritaires lors du passage à une utilisation régulière ou à un hébergement accessible depuis Internet.

Cette analyse doit être mise à jour au fur et à mesure de l'élargissement du corpus et des essais terrain. Un risque dont la fréquence augmente ou dont les conséquences deviennent plus importantes devra conduire à une adaptation de l'architecture, du protocole de capture ou des règles de validation.

Conclusion

Dans ce chapitre, nous avons présenté l'environnement professionnel dans lequel s'inscrit ce projet ainsi que le contexte métier associé aux audits énergétiques. L'étude de l'existant a permis de mettre en évidence plusieurs limites liées au caractère manuel de l'identification et de la caractérisation des équipements électriques. Ces constats ont conduit à la formulation d'une problématique centrée sur l'automatisation de ce processus grâce aux technologies d'intelligence artificielle.

Une solution reposant sur la détection d'objets, le suivi multi-objets, l'intelligence artificielle multimodale et les systèmes de génération augmentée par récupération a alors été proposée. Les besoins fonctionnels et non fonctionnels identifiés constituent désormais le cadre de référence pour la suite du projet. L'analyse des risques a enfin mis en évidence la nécessité de maîtriser la qualité des prises de vue, les erreurs de détection, la dépendance aux services externes et la protection des données, tout en maintenant le technicien dans la boucle de validation.

Afin de justifier les choix technologiques retenus et de positionner la solution proposée par rapport aux travaux existants, le chapitre suivant présente un état de l'art des principales approches utilisées dans les domaines concernés.

Chapitre 2

État de l'art et choix technologiques

Introduction

Après avoir présenté le contexte du projet ainsi que les besoins auxquels la solution doit répondre, il est nécessaire d'étudier les différentes approches technologiques susceptibles de résoudre les problématiques identifiées.

Ce chapitre présente les principaux concepts et technologies mobilisés dans le cadre de ce travail. L'étude porte notamment sur la détection d'objets, le suivi multi-objets, les modèles multimodaux, les systèmes de génération augmentée par récupération ainsi que les technologies utilisées pour le développement et le déploiement de la solution.

Une analyse comparative des différentes alternatives est également réalisée afin de justifier les choix technologiques retenus.

2.1 Vision par ordinateur pour la détection d'équipements

La première étape du système proposé consiste à identifier automatiquement les équipements électriques présents dans une vidéo capturée lors d'un audit énergétique. Cette tâche relève du domaine de la vision par ordinateur, discipline de l'intelligence artificielle qui permet aux machines d'interpréter et de comprendre les informations contenues dans les images et les vidéos.

Grâce aux progrès récents de l'apprentissage profond, les systèmes de vision par ordinateur atteignent aujourd'hui des niveaux de performance remarquables dans des tâches telles que la classification, la détection d'objets, la segmentation d'images ou encore le suivi d'objets. Dans le cadre de ce projet, l'objectif est d'identifier automatiquement différents équipements électriques à partir d'une simple vidéo filmée par un technicien.

Cette section présente les principaux concepts liés à la détection d'objets, les différentes approches existantes ainsi qu'une étude comparative des modèles susceptibles de répondre aux besoins du projet.

2.1.1 Vision par ordinateur

La vision par ordinateur (*Computer Vision*) est une branche de l'intelligence artificielle qui vise à permettre aux systèmes informatiques d'acquérir, traiter et interpréter des informations visuelles provenant du monde réel [15].

Inspirée du système visuel humain, elle cherche à reproduire des capacités cognitives telles que la reconnaissance d'objets, l'identification de formes, la compréhension de scènes ou encore l'analyse de mouvements.

Les premières approches reposaient principalement sur des techniques de traitement d'images et d'extraction de caractéristiques conçues manuellement (*hand-crafted features*), telles que les descripteurs SIFT (*Scale Invariant Feature Transform*) [16] ou SURF (*Speeded-Up Robust Features*) [17].

L'émergence des réseaux de neurones convolutifs (*Convolutional Neural Networks – CNN*) a profondément transformé ce domaine en permettant l'apprentissage automatique de représentations visuelles directement à partir des données [18].

Aujourd'hui, la vision par ordinateur couvre plusieurs tâches fondamentales :

— la classification d'images ;

— la localisation d'objets ;

— la détection d'objets ;
— la segmentation d'images ;
— le suivi d'objets ;
— la compréhension de scènes.

Parmi ces tâches, la détection d'objets constitue le cœur du système développé dans ce projet.

2.1.2 Détection d'objets

La détection d'objets (*Object Detection*) consiste à identifier et localiser simultanément les objets présents dans une image ou une vidéo [19].

Contrairement à la classification d'images, qui attribue une unique étiquette à une image entière, la détection d'objets doit répondre à deux questions :

1. Quels objets sont présents dans l'image ?
2. Où se trouvent-ils ?

Pour chaque objet détecté, le modèle produit généralement :
— une classe ;
— une boîte englobante (*bounding box*) ;
— un score de confiance.

Les performances des détecteurs sont généralement évaluées à l'aide de la métrique *Mean Average Precision* (mAP), qui mesure simultanément la qualité de la classification et de la localisation des objets [20].

Les approches modernes de détection peuvent être regroupées en deux grandes catégories :
— les détecteurs à deux étapes (*Two-Stage Detectors*) ;
— les détecteurs à une étape (*One-Stage Detectors*).

Les détecteurs à deux étapes privilégient généralement la précision, tandis que les détecteurs à une étape privilégient la vitesse d'exécution, ce qui les rend particulièrement adaptés aux applications temps réel.

2.1.3 Détection supervisée

La majorité des systèmes de détection d'objets reposent sur l'apprentissage supervisé. Dans cette approche, le modèle est entraîné à partir d'un ensemble d'images annotées manuellement, dans lesquelles chaque objet est associé à une classe et à une boîte englobante.

Cette méthode a permis le développement de détecteurs extrêmement performants sur des jeux de données de référence tels que PASCAL VOC [20], COCO [13] ou LVIS [21].

Cependant, la détection supervisée présente plusieurs limitations.

Premièrement, elle nécessite la constitution d'un jeu de données annoté, dont la création représente un coût important en temps et en ressources humaines.

Deuxièmement, les performances du modèle restent fortement dépendantes des classes présentes dans les données d'entraînement.

Enfin, l'ajout d'une nouvelle catégorie d'objet nécessite généralement une nouvelle phase de collecte de données et de réentraînement.

Dans le cadre du présent projet, la diversité des équipements électriques rencontrés lors des audits énergétiques rend difficile la constitution d'un jeu de données suffisamment représentatif pour couvrir l'ensemble des cas possibles.

2.1.4 Détection open-vocabulary

La détection *open-vocabulary* (OVD – *Open-Vocabulary Detection*) constitue une évolution récente de la détection d'objets traditionnelle. Contrairement aux approches supervisées qui ne

peuvent reconnaître que les catégories présentes dans les données d'entraînement, les modèles open-vocabulary sont capables de détecter des objets décrits sous forme textuelle, même si ces catégories n'ont jamais été observées lors de l'apprentissage [2].

Cette capacité repose sur l'utilisation de modèles vision-langage entraînés sur de très grands volumes de données image-texte provenant du Web. Ces modèles apprennent à projeter simultanément les images et les descriptions textuelles dans un espace vectoriel commun, permettant ainsi d'établir des correspondances sémantiques entre les deux modalités.

Parmi les travaux les plus influents figure CLIP (*Contrastive Language-Image Pretraining*) proposé par OpenAI [22]. CLIP a démontré qu'un modèle entraîné sur plusieurs centaines de millions de paires image-texte pouvait reconnaître de nouveaux concepts à partir de simples descriptions textuelles, sans nécessiter de réentraînement.

Le principe général de la détection open-vocabulary est illustré à la figure 2.1.

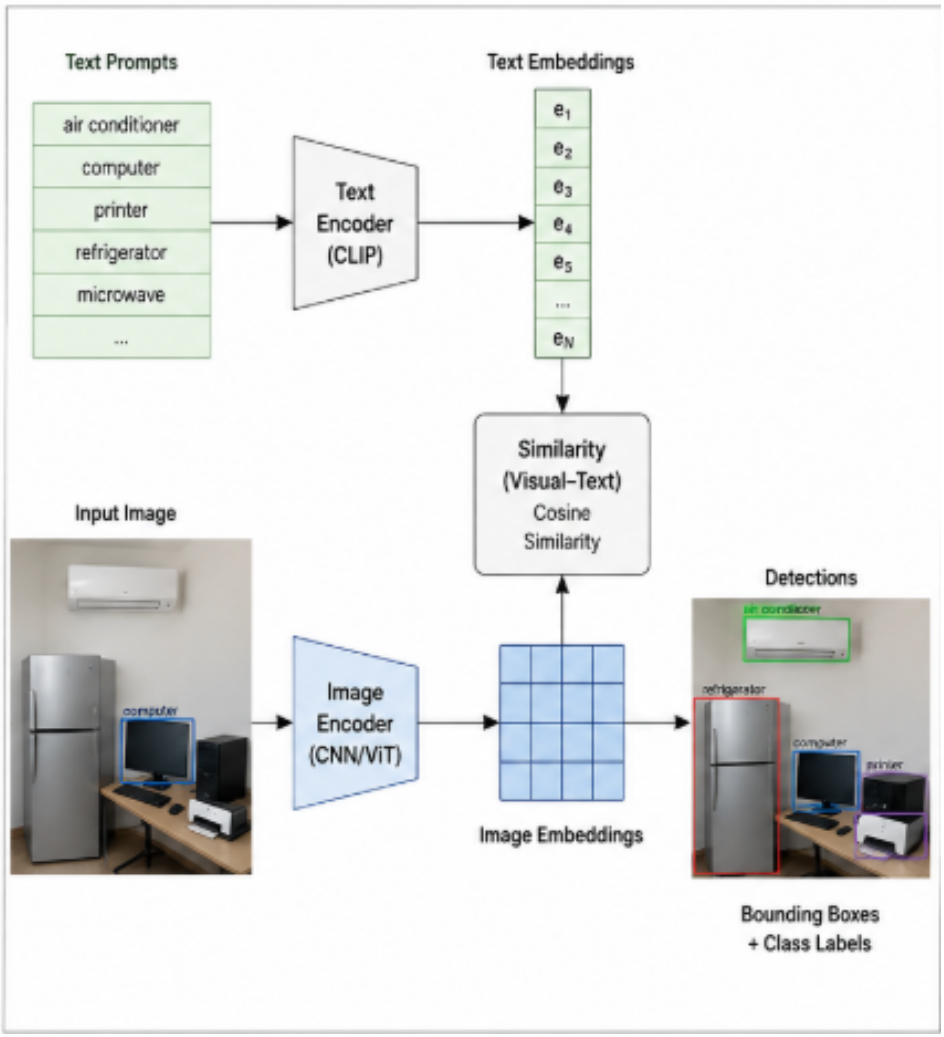


FIGURE 2.1 – Illustration conceptuelle de la détection open-vocabulary, réalisée dans le cadre du projet à partir des principes décrits dans [2]

Dans le contexte du présent projet, cette approche présente un intérêt particulier. En effet, les équipements électriques rencontrés lors des audits énergétiques peuvent varier d'un site à l'autre et inclure des catégories rarement présentes dans les jeux de données publics. L'utilisation d'un détecteur open-vocabulary permet ainsi de définir les objets recherchés sous forme de prompts textuels tels que :

— *air conditioner* ;
— *desktop computer* ;

— *server rack* ;
— *printer* ;
— *refrigerator* ;
— *UPS*.

Cette flexibilité réduit considérablement les besoins en annotation de données tout en facilitant l'adaptation du système à de nouveaux types d'équipements.

2.1.5 Principales approches existantes

De nombreuses architectures ont été proposées au cours des dernières années pour la détection d'objets. Cette section présente les modèles les plus représentatifs ainsi que leur évolution vers les approches open-vocabulary.

Famille R-CNN

La famille R-CNN (*Region-based Convolutional Neural Networks*) a marqué une étape importante dans l'histoire de la détection d'objets.

Le modèle R-CNN, proposé par Girshick et al. en 2014 [23], repose sur une approche en deux étapes. Une première phase génère des régions candidates susceptibles de contenir des objets, tandis qu'une seconde phase réalise leur classification à l'aide d'un réseau de neurones convolutif.

Bien que cette approche ait permis d'améliorer considérablement les performances de détection, elle présentait un coût computationnel élevé. Pour remédier à cette limitation, plusieurs variantes ont été proposées, notamment Fast R-CNN [24] puis Faster R-CNN [25].

Faster R-CNN introduit un réseau de proposition de régions (*Region Proposal Network*) permettant de réduire fortement le temps d'inférence tout en conservant une excellente précision.

Malgré leurs performances, les modèles de la famille R-CNN demeurent relativement coûteux en calcul et peu adaptés aux applications temps réel sur flux vidéo.

SSD

SSD (*Single Shot Detector*) a été proposé par Liu et al. [26] afin de répondre aux contraintes de rapidité des applications temps réel.

Contrairement aux détecteurs à deux étapes, SSD réalise simultanément la localisation et la classification des objets en une seule passe sur l'image.

L'utilisation de cartes de caractéristiques à différentes résolutions permet au modèle de détecter des objets de tailles variées tout en maintenant une vitesse d'exécution élevée.

SSD constitue ainsi l'une des premières approches de détection à une étape ayant démontré qu'il était possible d'obtenir un compromis satisfaisant entre précision et rapidité.

Famille YOLO

YOLO (*You Only Look Once*), introduit par Redmon et al. [27], constitue aujourd'hui l'une des familles de détecteurs les plus populaires.

Le principe fondamental de YOLO consiste à traiter la détection comme un problème de régression unique. Le modèle prédit directement les boîtes englobantes et les probabilités de classes en une seule passe sur l'image.

Les différentes générations de YOLO ont progressivement amélioré la précision, la robustesse et la vitesse d'exécution.

Parmi les versions les plus importantes figurent :
— YOLOv3 ;
— YOLOv5 ;
— YOLOv7 ;

— YOLOv8 ;
— YOLOv9 ;
— YOLOv10.

Grâce à leur excellente vitesse d'inférence et à leur facilité d'utilisation, les modèles YOLO sont largement utilisés dans les applications industrielles et embarquées.

Grounding DINO

Grounding DINO [28] est l'une des premières architectures open-vocabulary ayant obtenu des performances comparables aux détecteurs supervisés classiques.

Basé sur une architecture Transformer, Grounding DINO combine les capacités de localisation d'objets du modèle DINO avec les représentations textuelles issues de modèles vision-langage.

Le modèle permet de détecter des objets à partir de descriptions textuelles détaillées et obtient d'excellentes performances sur les benchmarks open-vocabulary.

Cependant, cette précision élevée s'accompagne d'un coût computationnel important qui limite son utilisation dans les applications nécessitant un traitement temps réel.

YOLO-World

YOLO-World, proposé par Cheng et al. en 2024 [3], représente une évolution majeure de la famille YOLO vers la détection open-vocabulary.

Contrairement aux versions traditionnelles de YOLO qui nécessitent un apprentissage supervisé sur un ensemble de classes prédéfinies, YOLO-World intègre directement des représentations textuelles dans son processus de détection. Cette approche permet de détecter des objets à partir de simples descriptions textuelles sans nécessiter de réentraînement spécifique.

L'architecture repose sur un mécanisme de fusion entre les caractéristiques visuelles extraites de l'image et les représentations textuelles obtenues à partir d'un encodeur de langage. Cette fusion est réalisée au sein d'un module nommé *RepVL-PAN* (*Re-parameterizable Vision-Language Path Aggregation Network*), chargé d'aligner les informations visuelles et textuelles dans un espace de représentation commun.

Le principe général de fonctionnement de YOLO-World est illustré à la figure 2.2.

Cette architecture permet notamment :
— la détection d'objets inconnus du jeu d'entraînement ;
— l'ajout dynamique de nouvelles catégories par simple prompt ;
— le maintien de performances compatibles avec les applications temps réel ;
— une intégration naturelle dans les pipelines industriels.

Ces caractéristiques rendent YOLO-World particulièrement adapté aux contextes où les catégories d'objets évoluent fréquemment ou ne peuvent pas être exhaustivement annotées.

YOLO-World v2 — Afin d'améliorer davantage les performances de la première version, les auteurs ont proposé YOLO-World v2, qui introduit plusieurs optimisations portant sur l'alignement vision-langage, la qualité des représentations textuelles et l'efficacité de l'inférence.

Cette seconde génération améliore notamment :
— la précision de détection en mode open-vocabulary ;
— la robustesse face aux variations d'apparence des objets ;
— la généralisation vers des catégories rarement observées ;
— l'efficacité du traitement sur des matériels disposant de ressources limitées.

Dans le cadre de ce projet, la variante **YOLOv8x-Worldv2** a été retenue en raison de son excellent compromis entre précision, généralisation et vitesse d'exécution.

2.1.6 Étude comparative

Le tableau 2.1 synthétise les principales caractéristiques des approches étudiées.

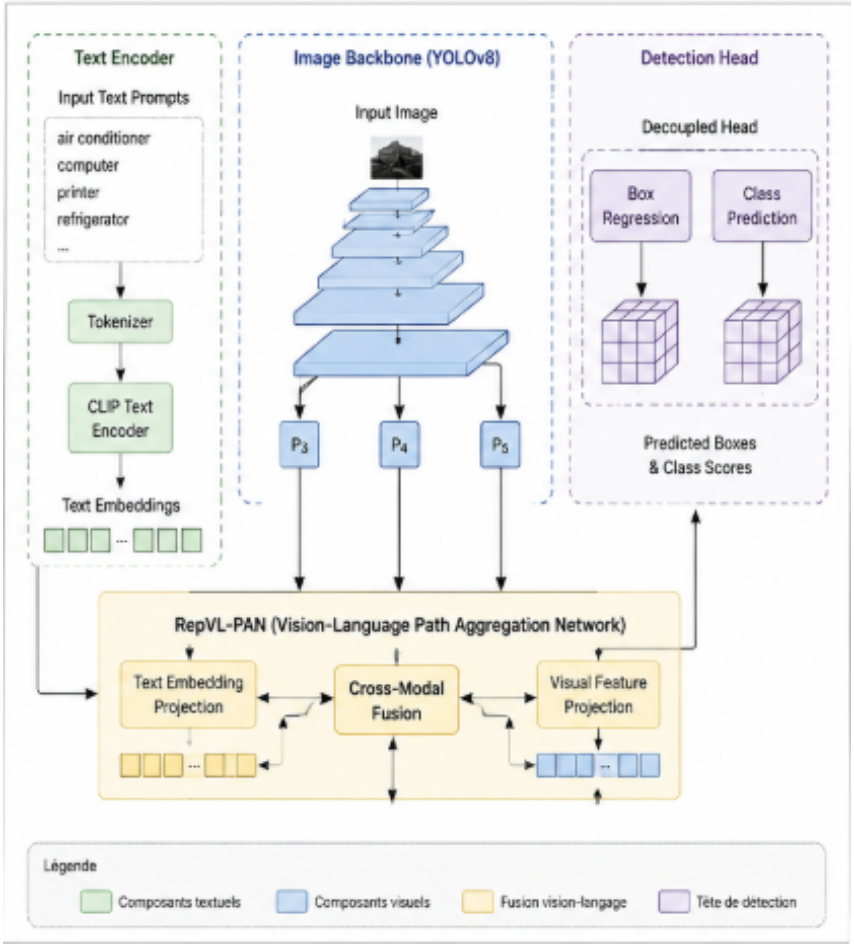


FIGURE 2.2 – Illustration conceptuelle du fonctionnement de YOLO-World, réalisée dans le cadre du projet à partir de [3]

TABLE 2.1 – Comparaison des principales approches de détection d'objets

Approche	**Temps réel**	**Vocabulaire ouvert**	**Résultat publié**	**Adaptation au projet**
Faster R-CNN	Non	Non	73,2 % mAP sur PASCAL VOC 2007 avec VGG-16 [25]	Faible : réentraînement nécessaire
SSD300	Oui	Non	74,3 % mAP sur PASCAL VOC 2007 à 59 FPS sur Titan X [26]	Faible : classes fermées
YOLOv8x	Oui	Non	53,9 % $AP_{50:95}$ sur COCO, entrée 640 px [29]	Moyenne : rapide mais réentraînement requis
Grounding DINO	Partiel	Oui	52,5 % AP sur COCO en transfert *zero-shot* [28]	Élevée : bonne ouverture, inférence plus lourde
YOLO-World	Oui	Oui	35,4 % AP sur LVIS en *zero-shot* et 52 FPS sur V100 [3]	Très élevée : vocabulaire configurable et temps réel

Les valeurs ci-dessus sont celles rapportées par les auteurs ou les documentations officielles. Elles ne constituent pas un classement direct, car les jeux de données, les protocoles et les matériels diffèrent. Elles remplacent néanmoins les qualificatifs généraux par des repères quantifiés et montrent que le choix de YOLO-World repose surtout sur le compromis entre détection *zero-shot* et vitesse d'inférence.

2.1.7 Choix de YOLOv8x-Worldv2

L'analyse des différentes approches met en évidence qu'aucun modèle supervisé classique ne répond pleinement aux contraintes du projet.

Les détecteurs traditionnels tels que Faster R-CNN, SSD ou YOLOv8 nécessitent la constitution d'un jeu de données annoté couvrant l'ensemble des équipements susceptibles d'être rencontrés lors des audits énergétiques. Une telle démarche représenterait un coût important en temps et en ressources humaines, tout en limitant la capacité du système à évoluer vers de nouvelles catégories d'équipements.

Les approches open-vocabulary apparaissent ainsi plus adaptées au contexte du projet. Parmi celles-ci, Grounding DINO offre d'excellentes performances mais présente un temps d'inférence relativement élevé, peu compatible avec un traitement efficace de séquences vidéo.

À l'inverse, YOLO-World combine les avantages des architectures YOLO temps réel avec la flexibilité offerte par la détection open-vocabulary. La version YOLOv8x-Worldv2 permet ainsi de détecter des équipements à partir de simples descriptions textuelles tout en conservant une vitesse d'exécution adaptée à une utilisation opérationnelle.

Les principales raisons ayant motivé ce choix sont les suivantes :

— absence de nécessité de constituer un jeu de données annoté ;
— ajout dynamique de nouvelles catégories par simple prompt ;
— compatibilité avec les contraintes temps réel ;
— robustesse face à la diversité des équipements rencontrés ;
— intégration native dans l'écosystème Ultralytics ;
— facilité de déploiement dans une architecture industrielle.

Pour ces raisons, YOLOv8x-Worldv2 constitue le détecteur retenu pour la première étape du système proposé.

Après avoir étudié les méthodes permettant d'identifier les équipements présents dans une vidéo, il convient désormais de s'intéresser aux techniques assurant le suivi cohérent de ces objets au cours du temps.

2.2 Suivi multi-objets dans les séquences vidéo

La détection d'objets permet d'identifier les équipements présents dans chaque image d'une vidéo. Cependant, lorsque plusieurs images sont analysées successivement, un même équipement peut être détecté plusieurs fois sous des positions légèrement différentes.

Afin d'éviter la duplication des équipements et de suivre leur évolution au cours du temps, il est nécessaire de mettre en œuvre un mécanisme de suivi d'objets (*Object Tracking*). Cette étape est essentielle dans le cadre du projet puisque les vidéos capturées lors des audits énergétiques contiennent généralement plusieurs vues d'un même équipement.

Cette section présente les principales approches de suivi multi-objets ainsi qu'une étude comparative permettant de justifier le choix de ByteTrack.

2.2.1 Principe du suivi multi-objets

Le suivi multi-objets (*Multi-Object Tracking – MOT*) consiste à maintenir l'identité d'un ensemble d'objets au cours d'une séquence vidéo [4].

Contrairement à la détection d'objets qui traite chaque image de manière indépendante, le suivi cherche à associer les détections successives correspondant à un même objet.

Pour chaque objet suivi, un identifiant unique (*Track ID*) est attribué et conservé tant que l'objet reste visible dans la scène.

La figure 2.3 illustre le principe général du suivi multi-objets.

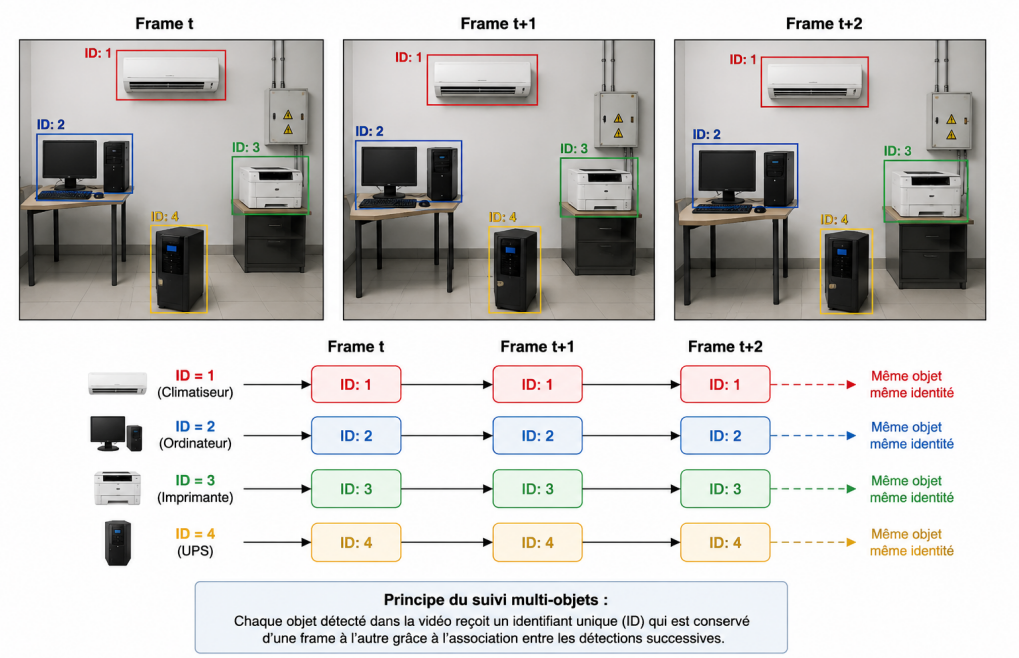


FIGURE 2.3 – Illustration conceptuelle du suivi multi-objets, réalisée dans le cadre du projet à partir des principes décrits dans [4]

Le suivi multi-objets repose généralement sur deux opérations principales :

1. la prédiction de la position future des objets ;
2. l'association entre les détections actuelles et les objets déjà suivis.

Les performances des algorithmes de suivi sont généralement évaluées à l'aide des métriques MOTChallenge telles que :

— MOTA (*Multiple Object Tracking Accuracy*) ;
— IDF1 (*Identity F1 Score*) ;
— HOTA (*Higher Order Tracking Accuracy*) ;
— nombre de changements d'identité (*ID Switches*).

2.2.2 SORT

SORT (*Simple Online and Realtime Tracking*) a été proposé par Bewley et al. en 2016 [4].

L'objectif principal de cette méthode est de fournir un algorithme de suivi simple, rapide et compatible avec les applications temps réel.

SORT repose sur deux composantes principales :

— un filtre de Kalman permettant de prédire la position future des objets ;
— l'algorithme hongrois (*Hungarian Algorithm*) utilisé pour associer les détections aux trajectoires existantes.

L'association est réalisée à partir de l'indice d'intersection sur union (IOU) entre les boîtes englobantes détectées et les boîtes prédites.

L'avantage principal de SORT réside dans sa simplicité et sa vitesse d'exécution. Les auteurs rapportent une vitesse supérieure à **260 images/seconde**, ce qui en fait l'un des trackers les plus rapides de la littérature.

Cependant, SORT présente une limitation importante : il ne tient compte que de la position géométrique des objets. En cas d'occlusion ou de croisement entre plusieurs objets, les changements d'identité sont fréquents.

2.2.3 DeepSORT

Afin d'améliorer la robustesse de SORT, Wojke et al. ont proposé DeepSORT en 2017 [30].

DeepSORT introduit une information supplémentaire appelée *apparence visuelle*. Pour chaque objet détecté, un réseau de neurones extrait un vecteur de caractéristiques permettant de décrire son apparence.

L'association entre les détections et les trajectoires existantes est alors réalisée en combinant :

— la proximité spatiale ;

— la similarité d'apparence.

Cette amélioration permet de réduire significativement les changements d'identité dans les scénarios complexes.

Sur le benchmark MOT16, DeepSORT atteint un score IDF1 supérieur à celui de SORT tout en conservant des performances temps réel satisfaisantes.

Néanmoins, l'extraction des descripteurs visuels augmente la complexité du système et le coût computationnel du suivi.

2.2.4 ByteTrack

ByteTrack a été proposé par Zhang et al. en 2022 [8]. Il constitue aujourd'hui l'une des méthodes de suivi multi-objets les plus performantes sur les benchmarks MOT.

Contrairement aux approches précédentes qui éliminent les détections ayant un faible score de confiance, ByteTrack exploite également ces détections dites *low-score detections*.

L'idée centrale consiste à réaliser l'association en deux étapes :

1. association des détections à forte confiance ;
2. association des détections à faible confiance restantes.

Cette stratégie permet de récupérer de nombreuses trajectoires perdues lors d'occlusions partielles ou de détections incertaines.

Les résultats publiés sur MOT17 montrent que ByteTrack atteint :

— **80,3 MOTA** ;

— **77,3 IDF1** ;

— **63,1 HOTA**.

Ces performances placent ByteTrack parmi les trackers temps réel les plus performants de sa génération.

Par ailleurs, ByteTrack s'intègre naturellement aux détecteurs de la famille YOLO, ce qui facilite son utilisation dans des pipelines de vision industrielle.

2.2.5 Étude comparative

Le tableau 2.2 compare les principales approches de suivi étudiées.

TABLE 2.2 – Comparaison des méthodes de suivi multi-objets

Méthode	**Temps réel**	**IDF1**	**Gestion des occlusions**	**Complexité**
SORT	Excellent	61,8	Faible	Faible
DeepSORT	Bonne	72,0	Bonne	Moyenne
ByteTrack	Excellente	77,3	Très bonne	Moyenne

2.2.6 Choix de ByteTrack

L'analyse comparative met en évidence plusieurs avantages de ByteTrack pour le contexte du projet.

Tout d'abord, la méthode offre un excellent compromis entre précision et vitesse d'exécution, ce qui constitue un critère essentiel pour le traitement de vidéos capturées sur le terrain.

Ensuite, sa stratégie d'exploitation des détections à faible confiance permet de réduire les pertes de trajectoires et les changements d'identité lorsque les équipements sont partiellement masqués ou observés sous différents angles.

Enfin, ByteTrack est aujourd'hui intégré nativement dans l'écosystème Ultralytics utilisé pour l'implémentation de YOLOv8x-Worldv2, facilitant ainsi son intégration dans le pipeline global du système.

Pour ces raisons, ByteTrack a été retenu comme mécanisme de suivi multi-objets dans le cadre de ce projet.

Après avoir étudié les mécanismes permettant de détecter et de suivre les équipements dans une vidéo, il convient désormais d'examiner les technologies capables d'interpréter les informations visuelles collectées et de générer automatiquement des descriptions techniques structurées.

L'identification et le suivi des équipements constituent une première étape. Cependant, la génération automatique de fiches techniques nécessite des capacités avancées de compréhension visuelle et textuelle.

2.3 Intelligence artificielle multimodale

L'identification et le suivi des équipements constituent une première étape du processus d'analyse. Toutefois, la finalité du système ne se limite pas à détecter des objets dans une vidéo. Il doit également être capable d'interpréter les informations visuelles observées et de produire automatiquement des fiches techniques exploitables par les équipes d'audit énergétique.

Cette capacité nécessite l'utilisation de modèles capables de raisonner simultanément sur plusieurs modalités d'information, notamment les images et le texte. Les avancées récentes dans le domaine des grands modèles de langage et des modèles vision-langage ont permis l'émergence de systèmes capables d'analyser une image, de comprendre son contenu et de générer des descriptions détaillées en langage naturel.

Cette section présente les principaux concepts liés aux modèles multimodaux ainsi qu'une étude comparative des solutions actuellement disponibles.

2.3.1 Les grands modèles de langage (LLM)

Les grands modèles de langage (*Large Language Models – LLM*) constituent l'une des avancées majeures de l'intelligence artificielle au cours des dernières années. Ces modèles reposent généralement sur l'architecture Transformer introduite par Vaswani et al. [31], qui a profondément transformé le domaine du traitement automatique du langage naturel.

Un LLM est un réseau de neurones entraîné sur d'immenses volumes de données textuelles provenant de livres, d'articles, de sites Web et d'autres ressources numériques. Grâce à cet apprentissage à grande échelle, le modèle acquiert des connaissances générales sur de nombreux domaines et développe des capacités avancées de compréhension et de génération de texte.

Le principe fondamental consiste à prédire le mot suivant dans une séquence de texte. Bien que cette tâche paraisse simple, elle permet au modèle d'apprendre des relations complexes entre les mots, les phrases et les concepts. À grande échelle, cette approche conduit à l'émergence de capacités telles que le raisonnement, la synthèse d'informations, la traduction ou encore la génération de contenu.

Parmi les modèles les plus connus figurent GPT (*Generative Pre-trained Transformer*) développé par OpenAI, Gemini développé par Google, Claude développé par Anthropic ainsi que LLaMA proposé par Meta.

Les LLM modernes possèdent plusieurs capacités particulièrement intéressantes pour les applications professionnelles :

— compréhension du langage naturel ;
— génération de textes structurés ;
— synthèse et reformulation d'informations ;
— extraction d'informations à partir de documents ;
— raisonnement sur des données complexes ;
— assistance à la prise de décision.

Dans le cadre du présent projet, les modèles de langage jouent un rôle important dans la génération automatique de descriptions techniques et de fiches d'équipements à partir des informations extraites lors de l'analyse vidéo.

Cependant, les LLM traditionnels présentent une limitation majeure : ils ne peuvent traiter directement que des données textuelles. Or, les audits énergétiques reposent principalement sur l'analyse d'images et de vidéos. Cette contrainte a conduit au développement d'une nouvelle catégorie de modèles capables de combiner plusieurs modalités d'information : les modèles vision-langage.

2.3.2 Les modèles vision-langage (VLM)

Les modèles vision-langage (*Vision-Language Models – VLM*) constituent une évolution des grands modèles de langage permettant le traitement conjoint de données visuelles et textuelles [32].

Alors que les LLM traditionnels sont conçus pour manipuler uniquement du texte, les VLM sont capables d'intégrer des informations provenant d'images, de vidéos ou d'autres sources visuelles afin de produire des réponses plus riches et plus contextualisées.

Le développement de ces modèles résulte de la convergence entre deux domaines majeurs de l'intelligence artificielle : la vision par ordinateur et le traitement automatique du langage naturel.

De manière générale, un modèle vision-langage repose sur trois composants principaux :

— un encodeur visuel chargé d'extraire les caractéristiques des images ;
— un modèle de langage responsable de l'interprétation et de la génération de texte ;
— un mécanisme d'alignement permettant de relier les représentations visuelles et textuelles.

Grâce à cette architecture, les VLM peuvent réaliser de nombreuses tâches telles que :

— la description automatique d'images ;
— la réponse à des questions visuelles ;
— la reconnaissance d'objets et de scènes ;
— l'extraction d'informations à partir de documents ;
— l'analyse de graphiques et de schémas ;
— la compréhension de vidéos.

Les performances des VLM ont considérablement progressé ces dernières années grâce à l'apparition de modèles multimodaux de grande taille capables d'exploiter des milliards de paramètres et d'immenses volumes de données image-texte.

Dans le cadre du présent projet, ces modèles jouent un rôle central. Après la détection et le suivi des équipements électriques, ils permettent d'analyser les images représentatives de chaque équipement, d'identifier leurs caractéristiques visibles et de générer des descriptions techniques exploitables dans les rapports d'audit.

Parmi les modèles multimodaux les plus performants actuellement disponibles figurent GPT-4 Vision, Gemini, Claude et LLaVA.

2.3.3 GPT-4 Vision

GPT-4 Vision est la version multimodale du modèle GPT-4 développée par OpenAI [33]. Ce modèle étend les capacités des grands modèles de langage en leur permettant de traiter simultanément du texte et des images.

L'utilisateur peut soumettre une ou plusieurs images accompagnées d'instructions textuelles. Le modèle analyse alors le contenu visuel et génère une réponse adaptée à la demande formulée.

GPT-4 Vision est capable de réaliser diverses tâches telles que :

— la description détaillée d'images ;
— l'identification d'objets ;
— l'analyse de documents ;
— l'interprétation de graphiques ;
— l'assistance au diagnostic visuel.

L'un de ses principaux atouts réside dans sa capacité de raisonnement multimodal. Le modèle ne se limite pas à identifier les éléments présents dans une image mais peut également établir des relations entre ces éléments et produire des explications contextualisées.

Toutefois, GPT-4 Vision demeure un service propriétaire dont l'utilisation nécessite un accès aux API d'OpenAI ainsi qu'un coût d'exploitation pouvant devenir significatif dans les applications à grande échelle.

2.3.4 Gemini

Gemini est une famille de modèles multimodaux développée par Google DeepMind [34]. Contrairement à certaines approches reposant sur l'ajout d'un module visuel à un modèle de langage existant, Gemini a été conçu dès l'origine comme un modèle nativement multimodal.

Cette conception lui permet de traiter simultanément plusieurs types de données, notamment :

— le texte ;
— les images ;
— les vidéos ;
— l'audio ;
— le code informatique.

Les différentes versions de Gemini proposent plusieurs niveaux de performance adaptés à différents usages. Les modèles Gemini 1.5 puis Gemini 2.0 ont notamment démontré d'excellentes capacités de raisonnement multimodal et de compréhension de contextes complexes.

Parmi les principales fonctionnalités offertes par Gemini figurent :

— l'analyse d'images ;
— la compréhension de documents ;
— la génération de contenu structuré ;
— l'extraction d'informations ;
— la réponse à des questions multimodales ;
— le raisonnement sur de longues séquences contextuelles.

L'un des avantages majeurs de Gemini réside dans sa fenêtre de contexte particulièrement étendue, permettant d'analyser simultanément un grand nombre d'informations au sein d'une même requête.

Dans le cadre du présent projet, cette capacité est particulièrement intéressante puisqu'elle permet d'exploiter simultanément les données issues de la détection d'objets, les images représentatives des équipements ainsi que les informations récupérées par le système RAG.

Gemini constitue ainsi un candidat particulièrement pertinent pour la génération automatisée de fiches techniques d'équipements.

2.3.5 Claude

Claude est une famille de modèles développée par Anthropic [35] et conçue pour offrir des capacités avancées de raisonnement tout en mettant l'accent sur la sécurité et la fiabilité des réponses générées.

Les versions récentes de Claude intègrent également des capacités multimodales permettant l'analyse d'images et de documents.

Claude est reconnu pour :

— ses capacités de raisonnement avancées ;
— la qualité rédactionnelle de ses réponses ;
— sa gestion efficace des contextes volumineux ;
— son approche orientée sécurité.

Grâce à sa grande fenêtre de contexte, Claude est particulièrement adapté aux tâches nécessitant l'analyse simultanée de nombreux documents ou de grandes quantités d'informations.

Cependant, son intégration dans certains environnements industriels reste parfois moins répandue que celle des solutions proposées par Google ou OpenAI.

2.3.6 LLaVA

LLaVA (*Large Language and Vision Assistant*) est un modèle vision-langage open source [36] développé dans le but de rendre les capacités multimodales accessibles à la communauté scientifique.

L'architecture de LLaVA repose généralement sur la combinaison d'un encodeur visuel et d'un grand modèle de langage open source tels que Vicuna ou LLaMA.

LLaVA permet notamment :

— la description d'images ;
— l'identification d'objets ;
— la réponse à des questions visuelles ;
— l'analyse de scènes.

L'un de ses principaux avantages réside dans son caractère open source, qui facilite son déploiement local et sa personnalisation.

Toutefois, malgré ses bonnes performances, LLaVA demeure généralement moins performant que les modèles propriétaires de dernière génération tels que GPT-4 Vision, Gemini ou Claude sur les tâches de raisonnement multimodal complexes.

Il constitue néanmoins une alternative intéressante lorsque les contraintes de confidentialité ou de coût imposent l'utilisation de solutions entièrement hébergées localement.

2.3.7 Étude comparative

Les modèles multimodaux étudiés présentent chacun des avantages et des limitations. Le choix d'un modèle dépend généralement de plusieurs critères tels que les performances de

compréhension visuelle, la qualité du raisonnement, la disponibilité d'une API, le coût d'utilisation et la facilité d'intégration dans une application industrielle.

Le tableau 2.3 synthétise les principales caractéristiques des modèles étudiés.

TABLE 2.3 – Comparaison des modèles multimodaux

Modèle	**Entrées multiples**	**API**	**Repère publié**	**Tarif API***
GPT-4o	Oui	Oui	69,1 % sur MMMU [35]	2,50 / 15 USD
Gemini 2.5 Flash	Oui	Oui	Évaluation propre au chapitre 5	0,30 / 2,50 USD
Claude 3.5 Sonnet	Oui	Oui	68,3 % sur MMMU [35]	3 / 15 USD
LLaVA 1.5	Oui	Variable	Résultats sur 11 benchmarks [36]	Calcul local, sans coût API

* *Tarifs indicatifs en dollars par million de tokens d'entrée / de sortie, consultés le 22 juillet 2026 [37, 38, 39]. Les tarifs et la disponibilité des anciennes versions peuvent évoluer. Les scores MMMU cités servent de repères et ne remplacent pas l'évaluation du cas d'usage menée au chapitre 5.*

L'analyse montre que les modèles propriétaires dominent actuellement les benchmarks multimodaux grâce à leurs capacités avancées de compréhension visuelle et de raisonnement. Les modèles open source tels que LLaVA offrent davantage de flexibilité mais restent généralement en retrait en termes de performances globales.

Les solutions GPT-4 Vision, Gemini et Claude apparaissent comme les alternatives les plus pertinentes pour une utilisation dans un système d'audit énergétique automatisé.

2.3.8 Choix de Gemini

À l'issue de l'étude comparative, le modèle Gemini a été retenu pour la composante multimodale du système.

Plusieurs raisons ont motivé ce choix.

Tout d'abord, Gemini offre d'excellentes capacités de compréhension visuelle. Le modèle est capable d'analyser simultanément plusieurs images représentant un même équipement sous différents angles, ce qui constitue un avantage important dans le contexte des audits énergétiques.

Ensuite, Gemini dispose de solides capacités de raisonnement lui permettant non seulement d'identifier les équipements observés, mais également d'en déduire certaines caractéristiques techniques à partir du contexte visuel fourni.

Par ailleurs, l'API proposée par Google facilite son intégration dans une architecture applicative moderne. Le modèle peut être invoqué directement depuis le backend afin de générer automatiquement des descriptions techniques structurées.

Le coût est également compatible avec un prototype manipulant quelques images par analyse. Au 22 juillet 2026, le tarif standard publié pour Gemini 2.5 Flash est de 0,30 USD par million de tokens d'entrée multimodaux et de 2,50 USD par million de tokens de sortie, avec un palier gratuit pour les essais [38]. Cette donnée chiffrée précise le compromis économique évoqué dans le tableau comparatif.

Enfin, son intégration avec les écosystèmes d'intelligence artificielle modernes ainsi que son support des traitements multimodaux à grande échelle en font une solution adaptée aux

contraintes du projet.

Les principales raisons ayant conduit au choix de Gemini sont les suivantes :

— excellente compréhension multimodale ;
— capacités avancées de raisonnement ;
— prise en charge simultanée du texte et des images ;
— API simple à intégrer ;
— bon compromis entre performances et coût ;
— évolutivité adaptée aux besoins futurs du projet.

Pour ces raisons, Gemini constitue le modèle multimodal retenu pour l'analyse des équipements détectés et la génération automatique des fiches techniques.

Malgré leurs performances, les modèles multimodaux restent limités par les connaissances apprises lors de leur entraînement. Afin d'enrichir leurs réponses avec des informations techniques externes et actualisées, il est nécessaire de recourir aux mécanismes de génération augmentée par récupération.

2.4 Génération augmentée par récupération

Les systèmes de génération augmentée par récupération (*Retrieval-Augmented Generation*) permettent d'enrichir les réponses des modèles de langage grâce à des connaissances externes.

2.4.1 Limites des modèles de langage

Les modèles de langage, aussi performants soient-ils, présentent deux limites fondamentales qui les rendent insuffisants seuls pour ce projet :

1. **Connaissance figée dans le temps** : les LLM et VLM sont entraînés sur des corpus collectés jusqu'à une date de coupure. Toute information postérieure — nouveau modèle d'équipement, mise à jour de spécifications constructeur — leur est inconnue ;
2. **Hallucinations** : en l'absence de données factuelles dans leur contexte, les modèles peuvent générer des valeurs plausibles mais incorrectes. Dans un contexte industriel d'inventaire d'équipements, une puissance ou une tension erronée peut conduire à des décisions énergétiques incorrectes. Des études récentes estiment le taux d'hallucination des LLM entre **15 % et 27 %** sur des tâches factuelles [40].

La génération augmentée par récupération (*Retrieval-Augmented Generation*, RAG) a été proposée par Lewis et al. [41] pour pallier ces deux limitations en ancrant les réponses du modèle sur des données externes vérifiées.

2.4.2 Principe du RAG

Le RAG combine deux composantes :

1. un **module de récupération** (*retriever*) qui recherche, dans une base de connaissances externe, les documents les plus pertinents par rapport à la requête courante ;
2. un **module de génération** (*generator*) qui produit la réponse finale en tenant compte à la fois de la requête et des documents récupérés.

Dans ce projet, le RAG est positionné **avant** l'appel à Gemini : pour chaque crop d'équipement, le système recherche dans la base Qdrant les fiches techniques les plus similaires, puis injecte le contexte récupéré directement dans le prompt. Ceci ancre la réponse de Gemini sur des données réelles avant toute génération, réduisant significativement le risque d'hallucination sur les spécifications techniques.

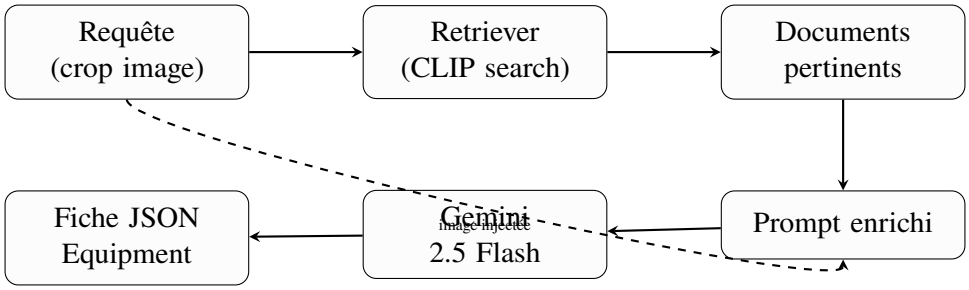


FIGURE 2.4 – Architecture RAG appliquée au pipeline Vision-RAG SFM

2.4.3 Embeddings

La récupération des documents pertinents repose sur une recherche par similarité dans un espace vectoriel. Chaque document et chaque requête sont préalablement transformés en vecteurs numériques denses appelés **embeddings**, qui capturent la sémantique du contenu dans un espace de haute dimension.

Deux types d'embeddings sont utilisés dans ce projet :

***Embeddings textuels.* —** Le modèle **sentence-transformers/all-MiniLM-L6-v2** [42] est utilisé pour encoder les descriptions textuelles des fiches techniques. Ce modèle produit des vecteurs de **384 dimensions** et offre un excellent compromis entre performance et légèreté : il occupe seulement **90 MB** en mémoire et s'exécute efficacement sur CPU, avec un temps d'encodage inférieur à **10 millisecondes** par phrase [42].

***Embeddings visuels.* —** Le modèle **CLIP** [22] (*Contrastive Language-Image Pre-training*) de OpenAI est utilisé pour encoder les images des équipements. CLIP produit des vecteurs de **512 dimensions** (variante ViT-B/32) alignés dans le même espace que les embeddings textuels, permettant une recherche multimodale : une image peut être comparée à des descriptions textuelles et vice versa. Cette propriété est exploitée dans le mécanisme de repli du RAG : lorsqu'aucune image similaire n'est trouvée dans la base, le système identifie la classe la plus proche via une comparaison texte-image CLIP.

2.4.4 Bases de données vectorielles

Les systèmes RAG nécessitent un mécanisme permettant de stocker et de rechercher efficacement les embeddings produits à partir des documents.

Les bases de données vectorielles répondent à ce besoin en offrant des structures d'indexation spécialement conçues pour la recherche de similarité dans des espaces de grande dimension.

Contrairement aux bases de données relationnelles classiques qui effectuent des recherches exactes, les bases vectorielles utilisent des algorithmes de recherche approximative du plus proche voisin (*Approximate Nearest Neighbor – ANN*) afin de retrouver rapidement les éléments les plus similaires à une requête.

Ces technologies constituent aujourd'hui un composant essentiel des architectures RAG modernes. Elles permettent de gérer des collections contenant plusieurs milliers, voire plusieurs millions de vecteurs tout en conservant des temps de réponse compatibles avec les applications temps réel.

Dans le cadre de ce projet, la base vectorielle joue un rôle central dans le pipeline Vision-RAG. Elle stocke les embeddings associés aux fiches techniques des équipements ainsi que les représentations visuelles extraites des images de référence. Lorsqu'un nouvel équipement est détecté, le système recherche les éléments les plus proches dans cet espace vectoriel afin de récupérer les informations techniques pertinentes qui seront ensuite transmises au modèle Gemini

pour la génération de la fiche descriptive.

2.4.5 Pinecone

Pinecone est une base de données vectorielle entièrement gérée dans le cloud (*fully managed*), proposée sous forme de service SaaS [43]. Elle offre une mise à l'échelle automatique, une haute disponibilité et des temps de requête inférieurs à la milliseconde sur des collections de milliards de vecteurs.

Pinecone est particulièrement adaptée aux applications de production à grande échelle. Cependant, son utilisation implique :

— une **dépendance à un service cloud externe**, avec des données hébergées sur les serveurs de Pinecone ;
— un **coût** dès que le volume de données dépasse le plan gratuit (1 index, 100 000 vecteurs) ;
— une **latence réseau** incompatible avec un usage local sur machine de développement sans connexion stable.

2.4.6 Chroma

Chroma est une base de données vectorielle open source, légère et conçue spécifiquement pour les applications LLM [44]. Elle s'intègre nativement avec LangChain et LlamaIndex, et peut fonctionner en mémoire ou avec persistance sur disque.

Chroma est simple à utiliser et ne requiert pas de configuration serveur pour les petits volumes de données. Cependant, elle présente des **limitations de performance** sur des collections dépassant quelques centaines de milliers de vecteurs, et son implémentation de la persistance a été sujette à des problèmes de stabilité dans les versions antérieures à 0.4.

2.4.7 Weaviate

Weaviate est une base de données vectorielle open source dotée d'un moteur de recherche hybride combinant recherche vectorielle et recherche par mots-clés (*BM25*) [45]. Elle intègre nativement des modules d'embedding (text2vec, img2vec) et expose une API GraphQL.

Weaviate offre des fonctionnalités avancées mais requiert le déploiement d'un serveur Docker, ce qui introduit une complexité d'infrastructure inadaptée au contexte de développement de ce projet. De plus, sa consommation mémoire au démarrage est significative (> **500 MB**), incompatible avec les contraintes RAM disponibles.

2.4.8 FAISS

FAISS est une bibliothèque open source de recherche de similarité et de clustering de vecteurs développée par Meta [46]. Elle fonctionne localement, propose plusieurs types d'index et peut exploiter le CPU ou le GPU. Elle ne fournit toutefois pas, à elle seule, les services d'une base de données complète : la persistance, les métadonnées, les filtres métier et les mises à jour doivent être gérés par l'application. Cette charge de développement supplémentaire a conduit à privilégier Qdrant pour le prototype.

2.4.9 Qdrant

Qdrant [47] est une base de données vectorielle open source développée en Rust, conçue pour la production et optimisée pour les performances sur CPU. Elle propose :

— un **mode en mémoire** sans serveur, permettant une utilisation locale sans configuration ni déploiement Docker ;

— un **mode persistant** avec stockage sur disque pour les déploiements en production ;
— une **API Python native** (client officiel `qdrant-client`) simple et bien documentée ;
— le support de **plusieurs types de vecteurs** par point, permettant de stocker simultanément des embeddings textuels et visuels dans la même collection ;
— des **filtres sur les métadonnées** permettant de restreindre la recherche à des sous-ensembles de la base (par désignation, marque, etc.).

En termes de performances, Qdrant utilise l'algorithme **HNSW** (*Hierarchical Navigable Small World*) pour la recherche ANN, atteignant un rappel de **99 %** avec un temps de requête médian inférieur à **1 milliseconde** sur une collection de 1 million de vecteurs sur CPU [47].

2.4.10 Étude comparative

Le tableau 2.4 compare les principales bases vectorielles étudiées.

TABLE 2.4 – Comparaison des bases de données vectorielles

Solution	Open Source	Sans serveur	Multi-vecteurs	RAM min.	Mode local
Pinecone	Non	Oui (cloud)	Non	N/A	Non
Chroma	Oui	Oui	Non	< 100 MB	Oui
Weaviate	Oui	Non	Oui	> 500 MB	Partiel
FAISS [46]	Oui	Oui	Non	< 100 MB	Oui
Qdrant [47]	**Oui**	**Oui**	**Oui**	**< 50 MB**	**Oui**

2.4.11 Choix de Qdrant

Le choix de **Qdrant** comme base de données vectorielle est justifié par la convergence de quatre critères déterminants :

1. **Mode en mémoire sans configuration serveur** : Qdrant peut être instancié en quelques lignes de code sans déploiement Docker ni service externe, ce qui simplifie considérablement le développement local ;
2. **Souplesse de modélisation vectorielle** : Qdrant prend en charge les vecteurs nommés et plusieurs vecteurs par point. Le prototype n'exploite pas directement cette possibilité : il utilise deux collections, car MiniLM et CLIP produisent des dimensions et des espaces sémantiques incompatibles. Ce support reste pertinent pour une évolution future vers des points multimodaux unifiés ;
3. **Empreinte mémoire réduite** : avec une consommation mémoire très faible au démarrage, Qdrant reste compatible avec les contraintes matérielles du projet tout en maintenant d'excellentes performances ;
4. **Performances de recherche élevées** : grâce à l'algorithme HNSW, Qdrant permet d'effectuer des recherches de similarité extrêmement rapides tout en conservant un excellent niveau de précision.

Par ailleurs, Qdrant s'intègre naturellement avec les bibliothèques Python utilisées dans le projet, notamment FastAPI, Sentence-Transformers ainsi que les différents composants du pipeline RAG développés pour l'analyse des équipements électriques.

Ces caractéristiques font de Qdrant une solution particulièrement adaptée aux besoins du système proposé, tant en phase de développement qu'en perspective d'un déploiement opérationnel.

Après avoir étudié les technologies d'intelligence artificielle utilisées dans le système, il convient de présenter les outils ayant servi au développement de la solution.

2.5 Technologies de développement et de déploiement

Au-delà des composants d'intelligence artificielle, la réalisation du système nécessite plusieurs technologies assurant le développement de l'application mobile, l'implémentation du backend, l'orchestration des traitements ainsi que le déploiement des différents services.

Le tableau 2.5 présente les alternatives examinées avant de détailler les outils retenus.

TABLE 2.5 – Comparaison des technologies applicatives envisagées

Besoin	Alternatives examinées	Décision
Application mobile	Flutter, React Native, Kotlin natif	Flutter : code unique, composants de capture vidéo et cohérence Android/iOS
API Python	FastAPI, Flask, Django REST Framework	FastAPI : validation typée, documentation OpenAPI et intégration directe de Pydantic
Automatisation	n8n, Apache Airflow, Prefect	n8n : workflows événementiels simples et connecteurs adaptés aux webhooks du prototype
Conditionnement	Exécution locale, Docker	Exécution locale retenue pour le prototype ; Docker conservé comme perspective de production

2.5.1 Flutter

Flutter est un framework open source développé par Google pour la création d'applications multiplateformes à partir d'un code source unique.

Basé sur le langage Dart, Flutter permet de développer des applications mobiles performantes pour Android et iOS tout en offrant une interface utilisateur moderne et réactive.

Dans le cadre de ce projet, Flutter est utilisé pour développer l'application mobile destinée aux auditeurs énergétiques. Celle-ci permet notamment la capture de vidéos, la consultation des audits réalisés ainsi que la visualisation des fiches techniques générées par le système.

2.5.2 FastAPI

FastAPI est un framework web moderne développé en Python pour la création d'API REST performantes.

Il repose sur les annotations de type Python et offre une documentation automatique des services grâce aux standards OpenAPI et Swagger.

FastAPI a été retenu pour l'implémentation du backend du système. Il assure la gestion des requêtes provenant de l'application mobile, l'exécution du pipeline d'analyse vidéo ainsi que la communication avec les différents composants d'intelligence artificielle.

2.5.3 n8n

n8n est une plateforme open source d'automatisation de workflows.

Elle permet d'orchestrer différents services au sein d'un même pipeline grâce à une interface graphique basée sur des nœuds interconnectés.

Dans ce projet, n8n intervient après l'analyse réalisée par FastAPI. Il reçoit les fiches validées par webhook, déclenche leur persistance dans Supabase et peut orchestrer l'envoi de notifications.

Il ne pilote pas les étapes internes du pipeline Vision-RAG.

2.5.4 Docker comme perspective de déploiement

Docker est une plateforme de conteneurisation permettant de packager une application ainsi que l'ensemble de ses dépendances dans un environnement isolé appelé conteneur.

Cette approche garantit une exécution cohérente de l'application quel que soit l'environnement d'hébergement.

Docker n'est pas utilisé dans l'environnement actuel de développement et de validation. Il a été étudié comme solution de conditionnement pour une mise en production future, afin d'isoler les dépendances du backend et de rendre le déploiement reproductible.

2.5.5 Choix technologiques et perspective de déploiement

Le tableau 2.6 synthétise les technologies retenues pour le prototype et la perspective de conteneurisation.

TABLE 2.6 – Technologies du prototype et perspective de déploiement

Technologie	**Catégorie**	**Rôle principal**
Flutter	Développement mobile	Interface utilisateur et capture vidéo
FastAPI	Backend	API REST et orchestration des traitements
n8n	Automatisation	Gestion et orchestration des workflows
Docker	Perspective de déploiement	Non utilisé dans le prototype ; conteneurisation future des services

Ces technologies ont été sélectionnées en raison de leur compatibilité avec l'écosystème Python utilisé dans le projet, de leur facilité d'intégration ainsi que de leur capacité à supporter une architecture modulaire et évolutive.

Conclusion

Dans ce chapitre, nous avons étudié les principales technologies susceptibles de répondre aux besoins identifiés lors de l'analyse du projet.

Les analyses comparatives réalisées ont conduit au choix de **YOLO-World** pour la détection open-vocabulary des équipements, de **ByteTrack** pour le suivi multi-objets, de **Gemini** pour l'analyse multimodale et la génération d'informations, ainsi que de **Qdrant** pour l'implémentation du mécanisme de génération augmentée par récupération.

Les technologies **Flutter**, **FastAPI** et **n8n** ont également été retenues pour le développement du prototype. **Docker** reste une perspective de déploiement et ne fait pas partie de l'environnement actuellement validé.

Ces choix technologiques constituent le socle de l'architecture proposée. Le chapitre suivant présente la collecte, la préparation et l'organisation des données utilisées dans le cadre du projet.

Chapitre 3

Compréhension et préparation des données

Introduction

La qualité d'un système d'intelligence artificielle repose fondamentalement sur la qualité des données qui l'alimentent. Dans le cadre de ce projet, deux types de données sont nécessaires au fonctionnement du pipeline Vision-RAG : des **données visuelles** — séquences vidéo filmées dans les locaux de SFM Technologies — et des **données documentaires** — fiches techniques des équipements fournissant les spécifications non visibles dans les vidéos.

Ce chapitre couvre deux phases distinctes de CRISP-DM. La première consiste à comprendre les données collectées, à décrire leur diversité et à vérifier leur qualité. La seconde transforme ces constats en opérations de préparation : contrôle des fichiers, filtrage des images, sélection des keyframes, nettoyage documentaire, vectorisation et indexation dans Qdrant. Les deux phases sont réunies dans un même chapitre pour conserver le lien entre chaque problème observé et le traitement retenu, mais elles restent séparées dans le plan.

Dans la suite du rapport, les termes techniques *frame*, *keyframe* et *crop* sont conservés lorsqu'ils désignent précisément les objets manipulés par le code. Les noms de champs JSON, de classes Python et de paramètres sont également reproduits tels qu'ils apparaissent dans l'implémentation. Ce choix évite les traductions variables et facilite le rapprochement entre le texte, les journaux d'exécution et le code source.

3.1 Collecte des données

La collecte des données constitue la première étape du processus de préparation. Elle vise à rassembler l'ensemble des données nécessaires au fonctionnement et à la validation du système, en distinguant les données visuelles des données documentaires.

3.1.1 Sources des données

Les données utilisées dans ce projet proviennent de quatre sources complémentaires. Le tableau 3.1 synthétise ces différentes sources ainsi que leur rôle dans le pipeline.

TABLE 3.1 – Sources de données du projet

Source	Description	Type	Usage
Vidéos terrain	Séquences filmées dans les locaux de SFM Technologies avec un smartphone	Vidéo MP4	Détection et identification
Sites constructeurs	Pages produits officielles (Cisco, Samsung, Daikin, Dell, HP, Canon. . .)	HTML scrapé	Base RAG
Catalogue interne	Fiches techniques des équipements les plus fréquents chez les clients SFM, constituées manuellement	JSON	Base RAG
PDF constructeurs	Manuels d'installation et fiches techniques officielles téléchargées	PDF	Base RAG

3.1.2 Collecte des séquences vidéo

Les séquences vidéo ont été filmées dans les locaux de SFM Technologies à l'aide d'un smartphone Android, reproduisant au mieux les conditions réelles de réalisation d'un audit énergétique sur site. Le technicien se déplace dans la pièce en filmant les équipements présents, sans contrainte particulière sur la vitesse de déplacement ni sur l'angle de prise de vue.

Quatre vidéos de test ont été collectées afin de valider expérimentalement le pipeline proposé. Le tableau 3.2 présente les caractéristiques principales du corpus vidéo constitué.

TABLE 3.2 – Corpus de vidéos collectées

Vidéo	Durée	Frames	Résolution	Équipements filmés
ma_video.mp4	7,3 s	177	640×360	Ordinateur portable Dell, climatiseur Daikin, switch Cisco
ma_video1.mp4	20,2 s	601	360×640	Téléviseur LG
ma_video2.mp4	12,5 s	284	360×640	Réfrigérateur Samsung, four à micro-ondes, switch Cisco
ma_video3.mp4	12,1 s	323	640×360	Four à micro-ondes SILVERCREST, switch Cisco
Total	**52,1 s**	**1 385**	—	**6 catégories métier et 7 équipements physiques distincts**

† Bien que le switch Cisco soit visuellement présent dans `ma_video3`, il n'a pas généré de crop unique exploitable après filtrage ByteTrack : sa détection YOLO est instable (présente sur moins de deux frames consécutives), ne permettant pas de constituer une trajectoire fiable.

Il convient de distinguer les catégories métier, les équipements physiques filmés et les crops effectivement produits par le pipeline :

TABLE 3.3 – Distinction entre catégories, équipements et crops

Métrique	Valeur	Signification
Catégories métier observées	6	Ordinateur portable, climatiseur, switch réseau, téléviseur, réfrigérateur et four à micro-ondes
Équipements physiques distincts	7	Les deux fours à micro-ondes sont deux appareils différents, bien qu'ils appartiennent à la même catégorie métier
Crops uniques (ByteTrack)	7	Nombre de **crops finaux** produits par ByteTrack sur l'ensemble des 4 vidéos. Un crop correspond à un objet tracké dans une vidéo donnée. Le même équipement présent dans deux vidéos différentes génère deux crops distincts

Exemple illustratif

Les deux fours à micro-ondes filmés correspondent à deux équipements physiques, mais à une seule catégorie métier. À l'inverse, un même équipement visible dans plusieurs keyframes peut produire plusieurs détections avant d'être ramené à un seul crop par ByteTrack. Le nombre de catégories, le nombre d'appareils et le nombre de crops répondent donc à trois questions différentes.

Ces vidéos présentent des caractéristiques techniques représentatives des conditions réelles d'audit :

— **Format** : MP4 / H.264, standard smartphone Android et iOS ;
— **Résolution native** : 640×360 px à 1920×1080 px selon le mode de capture ;
— **Cadence** : 23 à 30 FPS selon la vidéo ;
— **Éclairage** : naturel et fluorescent, mixte ;
— **Stabilisation** : non garantie — flou de mouvement fréquent lors des déplacements.

Si les données visuelles constituent l'entrée principale du pipeline, elles restent insuffisantes pour produire des fiches techniques complètes. Il est donc nécessaire de collecter également des données documentaires décrivant les caractéristiques techniques des équipements.

3.1.3 Collecte des données documentaires

Les données documentaires alimentent la base RAG et permettent d'enrichir les fiches Equipment avec les spécifications techniques non visibles dans les vidéos. Trois mécanismes de collecte ont été mis en place :

***Catalogue manuel JSON.* —** Un catalogue de **10 fiches techniques** a été constitué manuellement pour les équipements les plus fréquents dans le parc SFM Technologies. Chaque fiche contient la désignation, la marque, le modèle, la puissance, la tension et — pour les climatiseurs — la capacité de refroidissement et la température de consigne.

***Web scraping automatique.* —** Un module de scraping Python (`scraping_service.py`) interroge automatiquement les sites constructeurs référencés pour extraire les spécifications techniques. Sur les huit sources configurées, deux ont retourné des résultats exploitables : **Samsung** (réfrigérateurs) et **Cisco** (switches réseau). Les autres sources ont retourné des erreurs HTTP 403 ou 404, reflétant les restrictions anti-scraping des grands constructeurs.

***Parsing de PDFs.* —** Un module de parsing PyMuPDF est configuré pour extraire le contenu textuel des fiches techniques PDF déposées dans le dossier `knowledge_base/pdfs/`. Aucune fiche technique au format PDF n'a été intégrée durant la phase d'expérimentation. Toutefois, le module de parsing reste opérationnel pour les déploiements futurs.

Le tableau suivant résume le bilan de la collecte documentaire :

TABLE 3.4 – Éléments documentaires collectés avant indexation

Source	Entrées	Images	Statut
Catalogue manuel JSON	10	0	**Disponible**
Web scraping — Cisco	1	3	**Disponible**
Web scraping — Samsung	1	0	**Disponible**
Web scraping — Daikin, LG, Dell, HP, Canon	0	0	Bloqué (403/404)
PDFs locaux	0	0	Module prêt
Total	**12**	**3**	—

Ce tableau décrit les éléments recueillis ou générés avant la construction de l'index. Il ne doit pas être confondu avec le contenu effectivement chargé par le backend. Lors de la dernière exécution, le point d'accès `/api/stats` a retourné 11 entrées textuelles et aucune image indexée. Une entrée collectée n'a donc pas été conservée dans l'index final. Les journaux disponibles ne permettent pas d'en identifier la cause avec suffisamment de certitude ; cet écart est signalé plutôt que justifié a posteriori. De même,

les trois images collectées existent sous la forme de six fichiers dupliqués sur disque, mais aucune n'est actuellement intégrée à Qdrant.

3.2 Compréhension des données brutes

Avant de définir les traitements de préparation, une exploration du corpus brut a été menée afin d'en caractériser le volume, la diversité et les défauts susceptibles d'influencer le pipeline. Cette étape ne cherche pas encore à améliorer les données : elle vise à justifier les choix effectués dans les sections suivantes.

3.2.1 Profil exploratoire des vidéos

Le corpus visuel comprend quatre séquences totalisant 52,1 secondes et 1 385 frames. Les durées varient de 7,3 à 20,2 secondes. Deux vidéos sont enregistrées au format paysage (640×360) et deux au format portrait (360×640), avec une cadence comprise entre 23 et 30 images par seconde. Cette hétérogénéité, bien que limitée, permet d'observer l'effet de l'orientation, de la durée et des mouvements de caméra sur la sélection des images.

L'examen visuel a fait ressortir trois difficultés principales :

— un flou de mouvement fréquent lorsque le smartphone se déplace ;
— une forte redondance entre des frames successives montrant le même équipement sous un angle proche ;
— des équipements parfois éloignés, partiellement occultés ou visibles pendant un nombre réduit de frames.

Ces observations motivent respectivement le filtrage par netteté, la sélection de keyframes par diversité et l'utilisation d'un mécanisme de suivi pour regrouper les détections d'un même objet.

3.2.2 Profil exploratoire des données documentaires

Les documents disponibles sont plus hétérogènes que les vidéos. Ils proviennent de fiches JSON internes, de pages web de constructeurs et de documents PDF. Les unités, les noms de champs et le niveau de détail varient selon les sources. Certaines pages sont également difficiles à collecter automatiquement en raison de restrictions HTTP.

L'exploration a mis en évidence des doublons potentiels sur le couple marque–modèle, des unités écrites sous plusieurs formes et des contenus HTML ou textuels inutiles pour la recherche. Ces constats justifient la normalisation des unités, la déduplication et le nettoyage présentés dans la section consacrée à la base documentaire.

3.2.3 Limites de représentativité

Le corpus reste un corpus de prototype : les quatre vidéos ont été filmées dans un environnement restreint et ne couvrent pas toutes les conditions rencontrées sur les sites clients. Les distributions par catégorie d'équipement sont déséquilibrées et certaines catégories ne sont représentées que par une occurrence. Les résultats obtenus sur ce corpus doivent donc être interprétés comme une validation expérimentale initiale, et non comme une estimation de performance généralisable à l'ensemble des audits énergétiques.

Une fois ces caractéristiques établies, les données peuvent être préparées afin de devenir exploitables par les différents modules du système.

3.3 Filtrage et nettoyage des données visuelles

Les séquences vidéo brutes ne peuvent pas être transmises directement au pipeline de détection. Une vidéo de 20 secondes capturée à 30 FPS contient environ 600 frames : analyser chacune d'elles avec

YOLO-World puis Gemini représenterait un coût computationnel et financier prohibitif. De plus, la majorité des frames sont soit floues en raison du mouvement de la caméra, soit redondantes car représentant le même équipement sous le même angle.

Cette section décrit les étapes de préparation permettant de réduire ce volume de données tout en conservant l'information pertinente.

3.3.1 Prétraitement des vidéos

Avant toute extraction de frames, chaque vidéo reçue est soumise à une étape de contrôle et de prétraitement visant à écarter les fichiers non exploitables et à normaliser les paramètres d'entrée du pipeline.

***Contrôle d'intégrité.* —** L'ouverture du fichier avec OpenCV constitue le premier contrôle. Le pipeline vérifie que le conteneur peut être lu, qu'au moins une frame est décodable et que la durée, la cadence et les dimensions retournées sont cohérentes. Un fichier vide, corrompu ou illisible est rejeté avant toute inférence. Le nom de fichier ne sert pas d'identifiant fonctionnel, ce qui évite qu'un doublon de nom écrase une vidéo déjà reçue.

***Vérification du format.* —** Le système accepte uniquement les vidéos au format **MP4 / H.264**, standard universel des smartphones Android et iOS. Tout autre format déclenche une erreur HTTP 422 avec un message explicite renvoyé à l'application Flutter.

***Redimensionnement avec conservation du ratio.* —** YOLO-World est appelé avec une taille d'inférence de **768×768 pixels**. Les frames ne sont pas étirées pour devenir carrées : le prétraitement applique un *letterboxing*. L'image est redimensionnée uniformément jusqu'à ce que son côté le plus long mesure 768 pixels, puis des bandes de remplissage sont ajoutées sur l'autre axe. Ainsi, une frame 640×360 devient 768×432 avant ajout du padding vertical, tandis qu'une frame 360×640 devient 432×768 avant ajout du padding horizontal. Le ratio d'aspect et la géométrie des équipements sont donc préservés.

Le choix de 768 pixels constitue un compromis entre :

- la **précision de détection** : une résolution trop faible (ex. : 320×320) dégrade la détection des petits équipements comme les switches réseau ;
- la **vitesse de traitement** : une résolution trop élevée (ex. : 1920×1080) augmente significativement le temps d'inférence YOLO sur CPU.

***Sauvegarde temporaire.* —** La vidéo est sauvegardée temporairement dans le dossier `uploads/` sous un identifiant unique généré par horodatage (`video_{timestamp}.mp4`). Elle est supprimée automatiquement à la fin de l'analyse.

***Orientation et métadonnées.* —** Les vidéos portrait et paysage sont conservées dans leur orientation décodée. Les coordonnées des boîtes sont calculées sur l'image transmise au détecteur, sans dépendre des métadonnées EXIF du smartphone. Cette précaution évite un décalage entre l'image affichée et les coordonnées utilisées pour extraire les crops.

Après normalisation des vidéos, une première étape de sélection est appliquée afin d'éliminer les frames de mauvaise qualité, notamment celles affectées par le flou de mouvement.

3.3.2 Filtrage par qualité — variance du Laplacien

La première étape de sélection des frames pertinentes repose sur un critère de **netteté**. Une frame floue, produite par le déplacement de la caméra, ne contient pas suffisamment de détails pour permettre une détection fiable des équipements.

***Mesure de la netteté.* —** La netteté d'une frame est quantifiée par la **variance du Laplacien**, un opérateur différentiel du second ordre qui mesure les variations d'intensité dans l'image [48]. Une image nette présente des contours marqués et donc une variance élevée ; une image floue présente des transitions douces et une variance faible.

$$\text{netteté}(I) = \text{Var}\left(\nabla^2 I\right) \quad \text{avec} \quad \nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

En pratique, le Laplacien est calculé par convolution de l'image en niveaux de gris avec le noyau discret :

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

***Calibration du seuil.* —** Le seuil de netteté τ a été calibré empiriquement sur le corpus de vidéos de test. Une valeur initiale de $\tau = 100$, couramment utilisée dans la littérature [48], a été abaissée à $\tau = 50$ après observation que certains équipements en arrière-plan — notamment le four à micro-ondes dans `ma_video3.mp4` — n'apparaissaient que dans des frames à variance comprise entre 50 et 100.

Afin d'évaluer l'impact du seuil de netteté sur la sélection des frames, une analyse comparative a été réalisée pour deux valeurs de seuil. Les résultats obtenus sont présentés dans le tableau 3.5.

Table 3.5 – Impact du seuil Laplacien sur la sélection des frames

Vidéo	**Frames totales**	$\tau = 100$	$\tau = 50$	**Gain**
ma_video.mp4	177	60	95	+58 %
ma_video1.mp4	601	150	200	+33 %
ma_video2.mp4	284	90	120	+33 %
ma_video3.mp4	323	100	130	+30 %
Total	**1 385**	**400**	**545**	**+36 %**

L'abaissement du seuil à $\tau = 50$ a permis de détecter le four à micro-ondes SILVERCREST dans `ma_video3.mp4` (score de confiance : 98 %), équipement manqué avec $\tau = 100$.

Le filtrage de netteté permet d'éliminer les frames floues. Cependant, un grand nombre de frames nettes restent redondantes. Une seconde phase de sélection est donc nécessaire pour conserver uniquement les frames les plus représentatives.

3.3.3 Sélection de keyframes et extraction de crops

Parmi les frames ayant passé le filtre de netteté, un second mécanisme élimine les frames redondantes et sélectionne les plus représentatives.

***Sélection de keyframes.* —** La sélection de keyframes repose sur un critère de **diversité par différence d'histogramme [49]**. Pour chaque frame candidate, son histogramme HSV est comparé à celui de la dernière keyframe retenue. Si la corrélation dépasse un seuil de similarité, la frame est considérée comme redondante et écartée. Ce mécanisme garantit que les keyframes retenues couvrent les différentes perspectives de la pièce filmée.

Le nombre maximal de keyframes est paramétrable (`max_keyframes`, défaut : 15). Ce paramètre constitue un levier de compromis entre :

- la **couverture** : un nombre élevé de keyframes réduit le risque de manquer un équipement ;
- la **performance** : chaque keyframe supplémentaire augmente le temps de traitement YOLO d'environ 0,5 seconde sur CPU.

TABLE 3.6 – Résultats de la sélection de keyframes par vidéo

Vidéo	Frames nettes	Keyframes	Réduction	Temps extraction
ma_video.mp4	95	15	84,2 %	1,5 s
ma_video1.mp4	200	15	92,5 %	4,8 s
ma_video2.mp4	120	15	87,5 %	1,7 s
ma_video3.mp4	130	15	88,5 %	2,3 s
Total	**545**	**60**	**89,0 %**	**10,3 s**

Extraction de crops via ByteTrack. — Après détection YOLO-World sur les keyframes, ByteTrack assigne un identifiant stable à chaque équipement détecté et sélectionne, pour chaque identifiant, la frame où l'équipement occupe la **surface maximale** dans l'image, selon une stratégie de sélection de la meilleure vue (*best-shot selection*) :

$$\text{best_shot}(k) = \arg\max_{f \in \mathcal{F}_k} \left[(x_2^{k,f} - x_1^{k,f}) \cdot (y_2^{k,f} - y_1^{k,f}) \right]$$

où $(x_1^{k,f}, y_1^{k,f}, x_2^{k,f}, y_2^{k,f})$ désignent les coordonnées de la boîte englobante de l'équipement k dans la frame f.

Le crop correspondant est extrait et sauvegardé au format JPEG dans le dossier `crops/` sous le nom `track_{id}.jpg`.

Pour alléger le tableau, les quatre vidéos sont désignées par V1 à V4, dans l'ordre présenté au tableau 3.2.

TABLE 3.7 – Réduction des détections par ByteTrack sur l'ensemble du corpus

Vidéo	Détections YOLO	Crops uniques	Réduction	Observation
V1	40	3	92,5 %	Trois trajectoires retenues ; trois fiches produites
V2	13	1	92,3 %	Une trajectoire stable correspondant au téléviseur
V3	23	2	91,3 %	Deux crops retenus, regroupés ensuite en une fiche par Gemini
V4	10	1*	90,0 %	Une trajectoire stable correspondant au four à micro-ondes
Total	**86**	**7**	**91,9 %**	—

* Le switch Cisco est visuellement présent dans `ma_video3`, mais il n'a pas généré de crop exploitable après ByteTrack. Sa détection est trop instable pour constituer une trajectoire fiable.

Note : le nombre de crops mesure la réduction opérée par ByteTrack, mais ne prouve ni l'exactitude de la catégorie détectée ni la présence de tous les équipements visibles. Ces deux aspects sont évalués séparément au chapitre 5 à l'aide des annotations manuelles et de l'analyse des fiches générées.

3.3.4 Organisation des données visuelles

Les données visuelles produites par le pipeline sont organisées selon l'arborescence suivante :

Listing 3.1 – Arborescence des données visuelles

```
backend/
|-- uploads/
|   |-- video_{timestamp}.mp4
|-- crops/
|   |-- track_{id}.jpg
|-- test_videos/
    |-- ma_video.mp4
    |-- ma_video1.mp4
    |-- ma_video2.mp4
    |-- ma_video3.mp4
```

Le tableau suivant récapitule le bilan global de la préparation des données visuelles :

Bilan de la préparation des données visuelles

Frames brutes totales	1 385 frames
Frames nettes ($\tau = 50$)	545 frames (39,4 %)
Keyframes sélectionnées	60 frames (4,3 % du total)
Détections YOLO-World	86 boîtes englobantes
Crops uniques (ByteTrack)	7 crops (réduction 91,9 %)
Réduction totale	de 1 385 frames à 7 crops (−99,5 %)

3.3.5 Constitution de la vérité terrain

Les comptages issus directement du pipeline ne suffisent pas à mesurer la précision d'un détecteur ou la cohérence d'un algorithme de suivi. Le calcul du mAP@50 exige une référence indépendante, annotée manuellement avant la comparaison avec les prédictions. Le calcul de l'IDF1 exige en plus des identifiants temporels stables pour chaque objet suivi.

Le protocole retenu utilise les 60 keyframes sélectionnées dans les quatre vidéos. Pour l'évaluation de la détection, chaque équipement visible doit être entouré par une boîte englobante et associé à une catégorie de référence. Un objet fortement occulté ou trop petit pour être identifié sans ambiguïté est marqué comme difficile et exclu du calcul principal, tout en étant conservé pour l'analyse qualitative.

Après relecture, l'annotation réalisée avec LabelImg contient 59 boîtes de référence réparties sur 57 images, les trois autres keyframes ne contenant aucun équipement ciblé. Deux annotations ambiguës ont été corrigées avant le calcul final : un climatiseur visible mais initialement absent de la vérité terrain et un réfrigérateur classé par erreur comme four à micro-ondes. Les fichiers ont ensuite été contrôlés automatiquement afin de vérifier le nombre de valeurs, les identifiants de classe et la normalisation des coordonnées. Aucune erreur structurelle n'a été relevée.

Cette annotation ne contient pas d'identifiants temporels. L'évaluation du suivi par IDF1 est donc laissée hors du périmètre expérimental actuel et nécessiterait une campagne complémentaire en mode suivi vidéo.

TABLE 3.8 – Règles d'annotation du corpus d'évaluation

Élément	Règle appliquée
Support annoté	60 keyframes, 59 boîtes de référence et 3 images négatives
Détection	Une boîte et une catégorie pour chaque équipement identifiable
Suivi	Identifiants temporels non annotés dans la campagne actuelle
Cas difficiles	Signalés séparément et exclus du score principal si l'identification humaine est ambiguë
Contrôle qualité	Relecture des boîtes, des catégories et des fichiers négatifs avant le calcul

Ce protocole rend l'exigence BNF1 calculable. Les résultats de mAP@50 sont présentés au chapitre d'évaluation. BNF2 reste non évalué, car l'annotation temporelle nécessaire au calcul de l'IDF1 n'a pas été réalisée.

3.4 Préparation de la base documentaire

Après la préparation des données visuelles, la seconde composante du système concerne les données documentaires. Contrairement aux données visuelles qui servent à identifier les équipements, ces données ont pour rôle d'enrichir leur description avec des caractéristiques techniques précises.

La base documentaire constitue le socle de connaissances du système RAG. Elle regroupe l'ensemble des fiches techniques des équipements susceptibles d'être rencontrés lors des audits SFM Technologies. Son rôle est de fournir à Gemini, avant chaque analyse visuelle, un contexte factuel ancré sur des données réelles — spécifications constructeurs, puissances nominales, tensions d'alimentation — que le modèle seul ne pourrait pas garantir sans risque d'hallucination.

3.4.1 Structure des fiches techniques

Chaque entrée de la base documentaire est modélisée comme une fiche technique structurée, dont les champs ont été définis en cohérence avec le modèle `Equipment` de SFM Technologies. Les champs retenus sont les suivants :

TABLE 3.9 – Structure d'une fiche technique dans la base RAG

Champ	Type	Description
`brand`	Texte	Marque du fabricant (ex. : Cisco, Daikin, Dell)
`model`	Texte	Référence commerciale du modèle
`designation`	Texte	Catégorie SFM (ex. : Climatisation, InfrastructureIT)
`power`	Entier (W)	Puissance nominale en watts
`voltage`	Entier (V)	Tension d'alimentation en volts
`cooling_capacity`	Entier (BTU)	Capacité de refroidissement (climatiseurs uniquement)
`description`	Texte	Description textuelle libre utilisée pour la génération des embeddings
`source`	Texte	Origine de la donnée (manuel, scraping, PDF)
`img_path`	Texte	Chemin vers l'image produit associée (si disponible)

Le champ `description` est le champ central pour la recherche par similarité textuelle. Il est généré automatiquement par concaténation des autres champs :

Listing 3.2 – Génération automatique du champ description

```python
def build_description(item: dict) -> str:
    parts = []
    if item.get("brand"):
        parts.append(item["brand"])
    if item.get("model"):
        parts.append(item["model"])
    if item.get("designation"):
        parts.append(item["designation"])
    if item.get("power"):
        parts.append(f"{item['power']}W")
    if item.get("voltage"):
        parts.append(f"{item['voltage']}V")
    if item.get("cooling_capacity"):
        parts.append(f"{item['cooling_capacity']} BTU")
    return " ".join(parts)
    # Exemple : "Cisco Catalyst InfrastructureIT 370W 220V"
```

3.4.2 Nettoyage et normalisation

Les données collectées depuis des sources hétérogènes — catalogue manuel, web scraping, PDF — présentent des incohérences qu'il convient de corriger avant indexation.

***Uniformisation des unités.* —** Les spécifications techniques peuvent être exprimées dans des unités différentes selon les sources. Les règles de normalisation suivantes ont été appliquées :

- **Puissance** : convertie systématiquement en **watts** (W). Les valeurs exprimées en kilowatts (kW) sont multipliées par 1 000 ;
- **Tension** : conservée en **volts** (V). Les plages (ex. : 100–240 V) sont remplacées par la valeur nominale européenne (220 V ou 230 V selon la norme EN 60038 [50]) ;
- **Capacité de refroidissement** : convertie en **BTU/h**. Les valeurs exprimées en kW thermiques sont multipliées par 3 412 ;
- **Marques** : normalisées en casse titre (ex. : `CISCO` → `Cisco`, `daikin` → `Daikin`).

***Suppression des doublons.* —** Un doublon est défini comme une entrée partageant le même couple (`brand`, `model`). La déduplication est réalisée par comparaison exacte de ces deux champs, en conservant l'entrée la plus complète (celle avec le plus grand nombre de champs non nuls).

***Nettoyage des textes.* —** Les textes issus de la collecte web et de l'extraction des PDF contiennent fréquemment des artefacts : balises HTML résiduelles, caractères spéciaux, espaces multiples, numéros de page. Un pipeline de nettoyage est appliqué :

Listing 3.3 – Pipeline de nettoyage des textes extraits

```python
import re

def clean_text(text: str) -> str:
    # Supprimer les balises HTML résiduelles
    text = re.sub(r'<[^>]+>', ' ', text)
    # Supprimer les caractères spéciaux non alphanumériques
    text = re.sub(r'[^\w\s\.,;\-\/\(\)°]', ' ', text)
    # Normaliser les espaces multiples
    text = re.sub(r'\s+', ' ', text)
    # Supprimer les espaces en début et fin
    return text.strip()
```

3.4.3 Organisation des documents

Les données documentaires sont organisées selon une arborescence claire distinguant les sources et les formats :

Listing 3.4 – Arborescence de la base documentaire

```
backend/
|-- knowledge_base/
    |-- equipment_catalog.json   # Catalogue manuel (10 fiches)
    |-- pdfs/                    # Fiches techniques PDF
    |   |-- (vide - module prêt)
    |-- images/                  # Images produits scrapées
        |-- cisco_0_{timestamp}.jpg
        |-- cisco_1_{timestamp}.jpg
        |-- cisco_2_{timestamp}.jpg
```

Le catalogue manuel `equipment_catalog.json` constitue le cœur de la base documentaire. Voici un exemple de fiche représentative :

Listing 3.5 – Exemple de fiche technique dans le catalogue JSON

```
{
  "brand":            "Cisco",
  "model":            "Catalyst 2960-X",
  "designation":      "InfrastructureIT",
  "power":            370,
  "voltage":          220,
  "cooling_capacity": null,
  "description":      "Cisco Catalyst 2960-X InfrastructureIT
                       370W 220V switch reseau entreprise",
  "source":           "manual",
  "img_path":         null
}
```

Le tableau suivant présente les 10 équipements du catalogue manuel, couvrant les catégories les plus fréquentes dans le parc SFM Technologies :

TABLE 3.10 – Contenu du catalogue manuel de dix fiches techniques

Désignation	Marque	Modèle	Puissance	Tension
InfrastructureIT	Cisco	Catalyst 2960-X	370 W	220 V
InfrastructureIT	Cisco	Catalyst 9200	800 W	220 V
AppareilInformatique	Dell	Latitude 5420	65 W	220 V
AppareilInformatique	HP	EliteBook 840	65 W	220 V
Climatisation	Daikin	FTXM35R	1 250 W	220 V
Climatisation	LG	S12EQ	1 100 W	220 V
Réfrigération	Samsung	RT38K5530S8	150 W	220 V
PériphériqueImpression	HP	LaserJet Pro M404	880 W	220 V
AppareilCuisine	Silvercrest	SEM 900 B2	900 W	220 V
AppareilInformatique	LG	27UK850	65 W	220 V

3.5 Vectorisation des données

Une fois les données visuelles et documentaires collectées, nettoyées et normalisées, elles doivent être transformées en représentations numériques exploitables par le moteur de recherche vectorielle. Cette transformation correspond à l'étape de vectorisation.

La vectorisation consiste à convertir les données brutes — textes et images — en vecteurs numériques appelés embeddings, permettant d'effectuer des recherches par similarité sémantique et visuelle dans Qdrant [10].

3.5.1 Génération des embeddings textuels

Les embeddings textuels sont générés à partir du champ `description` de chaque fiche technique par le modèle **sentence-transformers/all-MiniLM-L6-v2** [42].

***Caractéristiques du modèle.* —** Le choix de ce modèle repose sur ses performances en similarité sémantique, sa légèreté et sa rapidité d'inférence sur CPU. Le tableau suivant résume ses principales caractéristiques.

TABLE 3.11 – Caractéristiques du modèle all-MiniLM-L6-v2

Caractéristique	Valeur
Architecture	BERT distillé (MiniLM) — 6 couches
Dimension des embeddings	384 dimensions
Taille du modèle	90 MB
Longueur maximale d'entrée	256 tokens
Score sur STS Benchmark	78,9 (Spearman) [42]
Temps d'encodage (CPU)	< 10 ms par phrase
Licence	Apache 2.0 (gratuit, usage commercial autorisé)

***Processus d'encodage.* —** Pour chaque fiche technique de la base documentaire, le champ `description` est encodé en un vecteur de 384 dimensions. Ce vecteur est ensuite normalisé (norme L2 unitaire) avant d'être indexé dans Qdrant, conformément aux recommandations pour la recherche par similarité cosinus [42].

Listing 3.6 – Génération des embeddings textuels avec MiniLM

```
from sentence_transformers import SentenceTransformer

class RAGService:

    TEXT_MODEL = "sentence-transformers/all-MiniLM-L6-v2"

    def __init__(self):
        # Chargement du modèle (90 MB, CPU uniquement)
        self.text_embedder = SentenceTransformer(
            self.TEXT_MODEL
        )

```

```
    def encode_text(self, description: str) -> list:
        """
        Encode une description textuelle en vecteur
        de 384 dimensions normalisé L2.
        """
        vector = self.text_embedder.encode(
            description,
            normalize_embeddings=True  # Norme L2 = 1
        )
        return vector.tolist()
```

3.5.2 Génération des embeddings visuels

Les embeddings visuels sont générés par le modèle **CLIP ViT-B/32** [22] de OpenAI, qui produit des vecteurs de 512 dimensions dans un espace sémantique vision-langage partagé entre images et textes encodés par **CLIP lui-même**.

Cette propriété est exploitée pour la recherche visuelle (comparaison crop ↔ images de la base) et l'identification *zero-shot* (comparaison crop ↔ labels textuels encodés par CLIP).

À noter que cet espace vectoriel CLIP est distinct de celui de MiniLM-L6-v2 (384 dimensions) : les embeddings visuels CLIP et les embeddings textuels MiniLM ne sont pas comparables directement et sont stockés dans deux collections Qdrant séparées.

***Caractéristiques du modèle.* —** Le choix de CLIP ViT-B/32 repose sur sa capacité à projeter les données visuelles dans un espace sémantique compatible avec les représentations textuelles. Le tableau suivant présente les principales caractéristiques techniques du modèle retenu.

TABLE 3.12 – Caractéristiques du modèle CLIP ViT-B/32

Caractéristique	**Valeur**
Architecture	Vision Transformer ViT-B/32 [51]
Dimension des embeddings	512 dimensions
Taille du modèle	338 MB (poids ViT + encodeur textuel)
Résolution d'entrée	224×224 pixels
Score *zero-shot* ImageNet	63,2 % top-1 [22]
Temps d'encodage image (CPU)	< 300 ms par image
Licence	MIT (gratuit, usage commercial autorisé)

***Encodage des images de produits.* —** Les images de produits récupérées lors de la collecte web — notamment les trois images Cisco — sont encodées par CLIP et stockées dans la collection `equipment_images` de Qdrant. Lors de l'analyse d'un crop d'équipement, ce même crop est encodé par CLIP et comparé aux images de la base par similarité cosinus.

Listing 3.7 – Génération des embeddings visuels avec CLIP

```
import clip
import torch
from PIL import Image

class RAGService:
```

```
    CLIP_MODEL = "ViT-B/32"

    def __init__(self):
        # Chargement CLIP (338 MB, CPU)
        self.clip_model, self.clip_preprocess = \
            clip.load(self.CLIP_MODEL, device="cpu")

    def encode_image(self, image_path: str) -> list:
        """
        Encode une image en vecteur de 512 dimensions
        normalisé L2 via CLIP ViT-B/32.
        """
        image = self.clip_preprocess(
            Image.open(image_path).convert("RGB")
        ).unsqueeze(0)

        with torch.no_grad():
            vector = self.clip_model.encode_image(image)
            # Normalisation L2
            vector = vector / vector.norm(
                dim=-1, keepdim=True
            )

        return vector.squeeze().tolist()
```

3.5.3 Stratégie de recherche multimodale

La stratégie de recherche dans Qdrant combine deux modalités complémentaires pour maximiser la pertinence du contexte fourni à Gemini. Elle est organisée en trois niveaux successifs :

Niveau 1 — Recherche visuelle (prioritaire). — Le crop de l'équipement est encodé par CLIP et comparé aux images de produits indexées. Si un résultat dépasse le seuil de similarité cosinus $\theta_{img} = 0,6$, le contexte correspondant est retourné directement. Ce niveau est prioritaire car une correspondance visuelle exacte fournit le contexte le plus fiable.

Niveau 2 — Identification zero-shot par CLIP (repli). — Si aucune image similaire n'est trouvée, CLIP est utilisé en mode *zero-shot* pour identifier la catégorie la plus probable parmi un ensemble de labels prédéfinis. Le crop est comparé à des descriptions textuelles encodées (*ex.* : *"laptop computer"*, *"refrigerator"*, *"network switch"*) et le label avec le score de similarité le plus élevé est retenu comme requête textuelle.

Niveau 3 — Recherche textuelle de repli. — Le label identifié au niveau 2 est encodé par MiniLM et comparé aux embeddings textuels de la base. Les 3 entrées les plus similaires sont retournées comme contexte pour Gemini.

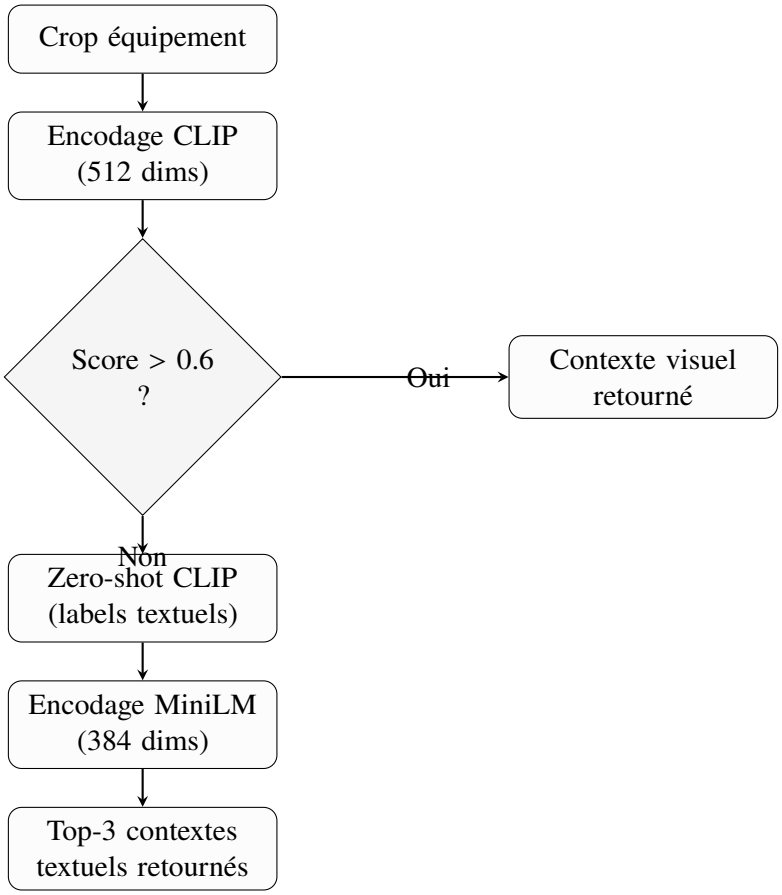


FIGURE 3.1 – Stratégie de recherche multimodale à trois niveaux

Listing 3.8 – Implémentation de la recherche multimodale

```python
def search_by_image(
    self, crop_path: str, top_k: int = 3
) -> tuple[list, list]:
    """
    Recherche multimodale à 3 niveaux.
    Retourne : (contextes textuels, chemins images).
    """
    # Niveau 1 : recherche visuelle CLIP
    img_vector = self.encode_image(crop_path)
    hits = self.client.query_points(
        collection_name=self.IMAGE_COLLECTION,
        query=img_vector,
        limit=top_k
    ).points

    if hits and hits[0].score > 0.6:
        logger.info(
            f"RAG found {len(hits)} matches "
            f"({len(hits)} images)"
        )
        return (
            [h.payload for h in hits],
            [h.payload.get("img_path") for h in hits]
        )
```

```
25
26      # Niveau 2 : identification zero-shot CLIP
27      logger.info("No image match - fallback to text search")
28      best_label = self._clip_zero_shot(
29          crop_path,
30          labels=[
31              "laptop computer", "desktop computer",
32              "air conditioner", "refrigerator",
33              "network switch", "printer",
34              "microwave", "television"
35          ]
36      )
37      logger.info(
38          f"CLIP identified: '{best_label}' "
39          f"(score: {score:.2f})"
40      )
41
42      # Niveau 3 : recherche textuelle MiniLM
43      return self._search_by_clip_text(best_label), []
```

Afin d'observer le comportement réel de la stratégie de recherche multimodale, les journaux de l'exécution corrigée ont été analysés sur les sept crops issus du corpus de validation.

TABLE 3.13 – Résultats de la recherche multimodale par crop sur le corpus complet

Vidéo / Crop	Niveau	Label CLIP	Score	Résultat RAG
ma_video — crop 1	Niv. 2+3	laptop computer	0,40	3 contextes textuels
ma_video — crop 2	Niv. 2+3	refrigerator	0,28	3 contextes textuels
ma_video — crop 3	Niv. 2+3	laptop computer	0,46	3 contextes textuels
ma_video1 — crop 1	Niv. 2+3	laptop computer	0,68	3 contextes textuels
ma_video2 — crop 1	Niv. 2+3	refrigerator	0,51	3 contextes textuels
ma_video2 — crop 2	Niv. 2+3	refrigerator	0,51	3 contextes textuels
ma_video3 — crop 1	Niv. 2+3	laptop computer	0,48	3 contextes textuels
Total	**7 crops** — aucun appariement d'image ; recherche textuelle de repli pour tous les crops.			

Les résultats montrent que le mécanisme de repli fonctionne, mais aussi que les labels CLIP restent parfois ambigus. En particulier, plusieurs crops sont rapprochés de *laptop computer* alors que les vidéos contiennent des équipements de catégories différentes. La pertinence métier des contextes retournés doit donc être vérifiée avant de les considérer comme exacts.

3.5.4 Stockage dans Qdrant

Les embeddings générés sont indexés dans deux collections Qdrant distinctes [52], correspondant aux deux modalités de recherche.

Qdrant permet également d'associer plusieurs vecteurs nommés à un même point. Cette fonctionnalité n'est pas utilisée dans le prototype actuel, car les vecteurs MiniLM (384 dimensions) et CLIP (512 dimensions) appartiennent à des espaces différents et font l'objet de requêtes distinctes. Deux collections séparées rendent cette séparation explicite et correspondent au code présenté dans le listing 3.9.

Listing 3.9 – Création des collections Qdrant

***Configuration des collections.* —**

```python
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance
)

class RAGService:

    TEXT_COLLECTION  = "equipment_text"
    IMAGE_COLLECTION = "equipment_images"

    def _create_collections(self):
        # Collection texte - MiniLM (384 dims)
        self.client.create_collection(
            collection_name=self.TEXT_COLLECTION,
            vectors_config=VectorParams(
                size=384,
                distance=Distance.COSINE
            )
        )
        logger.info(
            f"Collection created: {self.TEXT_COLLECTION}"
        )

        # Collection images - CLIP (512 dims)
        self.client.create_collection(
            collection_name=self.IMAGE_COLLECTION,
            vectors_config=VectorParams(
                size=512,
                distance=Distance.COSINE
            )
        )
        logger.info(
            f"Collection created: {self.IMAGE_COLLECTION}"
        )
```

***Structure d'un point Qdrant.* —** Chaque point stocké dans Qdrant est composé de trois éléments [52] :

1. un **identifiant unique** (UUID généré automatiquement) ;
2. un **vecteur** (384 ou 512 dimensions selon la collection) ;
3. un **payload** (métadonnées JSON associées au vecteur).

Afin de faciliter la recherche et la restitution des informations pertinentes, chaque vecteur indexé dans Qdrant est associé à un ensemble de métadonnées stockées dans un payload. Le tableau suivant décrit la structure des payloads pour les deux collections utilisées.

TABLE 3.14 – Structure du payload des deux collections Qdrant

Collection	Champs du payload	Exemple
equipment_text	brand	"Cisco"
	model	"Catalyst 2960-X"
	designation	"InfrastructureIT"
	power	370
	voltage	220
equipment_images	brand	"Cisco"
	img_path	"knowledge_base/images/cisco_0.jpg"
	description	"Cisco switch réseau entreprise"

Bilan de l'indexation. — Après la création des collections et l'insertion des vecteurs, un état récapitulatif de la base Qdrant permet d'évaluer le volume de données indexées ainsi que les ressources mobilisées.

État des collections Qdrant après indexation

Collection equipment_text	11 points indexés
Collection equipment_images	0 point indexé
Dimension vecteurs texte	384 dimensions (MiniLM)
Dimension vecteurs image	512 dimensions (CLIP)
Métrique de distance	Similarité cosinus
Mode de fonctionnement	In-memory (Windows 11)
Temps d'indexation total	22,0 secondes
RAM consommée	< 50 MB

À l'issue des étapes de collecte, de préparation et de vectorisation, il devient possible d'évaluer la qualité globale des données obtenues ainsi que leur adéquation avec les besoins du système proposé.

3.6 Synthèse et bilan de la préparation des données

Cette section synthétise l'ensemble des résultats obtenus lors de la préparation des données, en présentant les statistiques globales du corpus et les données finales exploitées par le prototype.

3.6.1 Statistiques du corpus

Données visuelles. — Le tableau suivant présente le bilan complet du pipeline de préparation des données visuelles, depuis les frames brutes jusqu'aux crops finaux transmis à Gemini :

TABLE 3.15 – Bilan du pipeline de préparation des données visuelles

Étape	Entrée	Sortie	Réduction
Filtrage Laplacien ($\tau = 50$)	1 385 frames	545 frames	−60,6 %
Sélection keyframes	545 frames	60 keyframes	−89,0 %
Détection YOLO-World	60 keyframes	86 détections	N/A
Déduplication ByteTrack [8]	86 détections	7 crops	−91,9 %
Pipeline complet	**1 385 frames**	**7 crops**	**−99,5 %**

La figure suivante illustre graphiquement cette réduction progressive du volume de données à chaque étape du pipeline :

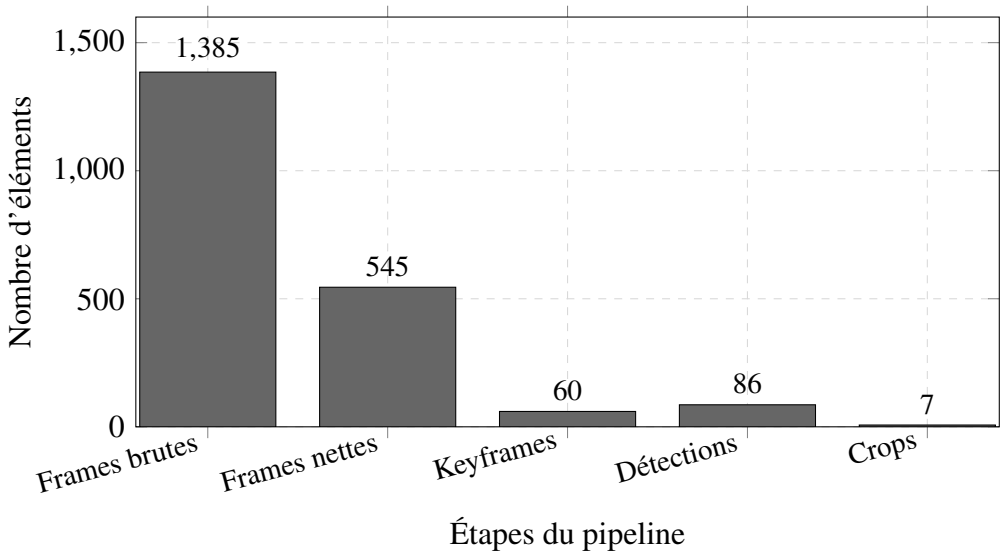


FIGURE 3.2 – Réduction progressive du volume de données à chaque étape du pipeline

TABLE 3.16 – État réel des données documentaires disponibles lors de la dernière exécution

Source	Entrées texte	Images indexées	Observation
Index Qdrant chargé par le backend	11	0	Valeurs retournées par `/api/stats`
Fichiers Cisco présents sur disque	—	0	6 fichiers, correspondant à 3 images dupliquées, non indexées
Total	**11**	**0**	**Recherche textuelle active ; recherche visuelle indisponible**

***Données documentaires.* —** La présence de fichiers image dans le répertoire de connaissances ne signifie donc pas qu'ils sont disponibles dans l'index vectoriel. Les journaux confirment qu'aucun appariement visuel n'est trouvé et que le service bascule vers la recherche textuelle pour chaque crop.

TABLE 3.17 – Empreinte mémoire des composantes de préparation

Composante	RAM	Remarque
Modèle MiniLM-L6-v2	≈ 90 MB	Chargé une fois au démarrage
Modèle CLIP ViT-B/32	≈ 338 MB	Chargé une fois au démarrage
Qdrant en mémoire (11 points texte)	< 50 MB	Rechargé à chaque démarrage
YOLO-World (yolov8x-worldv2)	≈ 200 MB	Chargé une fois au démarrage
Sous-total modèles et Qdrant	≈ **678 MB**	Périmètre propre aux composantes IA/RAG
FastAPI et n8n	≈ 250 MB	Services applicatifs comptabilisés au chapitre 4
Total local du prototype	≈ **928 MB**	Arrondi à 930 MB dans la vue de déploiement

Empreinte mémoire du système. —

3.6.2 Données finales retenues

À l'issue du processus de préparation, le prototype dispose du corpus visuel et de la base documentaire décrits ci-dessous. Cette synthèse porte uniquement sur les données produites ou indexées. Les résultats de détection, la conformité des fiches et les tests de bout en bout relèvent du chapitre d'évaluation.

Données finales exploitées par le système

Données visuelles

Corpus de test	4 vidéos (52,1 s, 1 385 frames)
Keyframes par vidéo	15 (paramètre configurable)
Crops par vidéo	1 à 3 (selon les équipements présents)
Format des crops	JPEG, 768×768 px maximum

Base documentaire RAG

Fiches texte indexées	11 entrées
Images indexées	0
Fichiers image sur disque	6 fichiers, soit 3 images dupliquées
Temps d'indexation	22,0 secondes lors de la dernière exécution

Limites identifiées et perspectives. — La préparation des données a mis en évidence trois limites qui constituent des axes d'amélioration pour les déploiements futurs :

1. **Base RAG limitée à 11 entrées textuelles** : le blocage HTTP 403/404 des principaux constructeurs (Dell, HP, Daikin, LG, Canon) a réduit la couverture de la base documentaire. L'intégration de PDF constructeurs téléchargés manuellement permettrait d'enrichir significativement cette base ;
2. **Qdrant en mémoire** : ce mode implique que la base RAG est rechargée à chaque démarrage du serveur (22,0 secondes lors de la dernière mesure). La migration vers le mode persistant de Qdrant [53], supporté sous Linux, éliminerait ce délai ;
3. **Corpus de test restreint** : les quatre vidéos de test ont été filmées dans le même environnement. Des vidéos filmées sur des sites clients réels SFM, dans des conditions

d'éclairage et d'encombrement variées, permettraient de valider la robustesse du système dans des conditions plus représentatives.

Conclusion

Ce chapitre a d'abord décrit les données brutes et les limites observées dans le corpus, puis présenté les traitements retenus pour y répondre. La préparation des données visuelles repose sur un pipeline en quatre étapes — filtrage Laplacien, sélection de keyframes, détection YOLO-World [3] et déduplication ByteTrack — qui réduit le volume de données de **99,5 %**, passant de 1 385 frames brutes à 7 crops. La base documentaire effectivement chargée contient 11 entrées textuelles et aucune image indexée. Elle fournit donc un contexte complémentaire à Gemini, mais sa couverture et sa pertinence restent à élargir et à valider expérimentalement.

Le protocole d'annotation défini dans ce chapitre fournit par ailleurs la vérité terrain nécessaire au calcul du mAP@50. L'IDF1 reste non mesurable sans annotation temporelle des identités. Les données préparées et indexées constituent désormais les entrées du pipeline Vision-RAG. Le chapitre suivant présente son architecture logicielle et les interactions entre les composants de détection, de recherche multimodale et de génération assistée par contexte.

Chapitre 4

Conception et architecture du système

Introduction

Les choix technologiques ayant été justifiés au chapitre 2 et les données préparées au chapitre 3, ce chapitre est consacré à la **conception architecturale** du système Vision-RAG. Il décrit l'organisation des composants logiciels, leurs interactions et les décisions de conception qui ont guidé le développement.

Note méthodologique : ce chapitre présente exclusivement les décisions de conception et les paramètres configurables du système (τ, nombre de keyframes, classes YOLO). Les mesures expérimentales (temps de traitement, consommation RAM, taux de réduction) sont présentées au chapitre suivant.

Ce chapitre s'articule autour de six axes : la vue fonctionnelle du système Vision-RAG, l'architecture globale, la conception détaillée de chaque module du pipeline Vision-RAG, les diagrammes dynamiques et statiques, la modélisation des données, et enfin le déploiement.

4.1 Vue fonctionnelle du système

La vue fonctionnelle, les acteurs et les cas d'utilisation ont été définis dans l'analyse des besoins au chapitre 1. Le diagramme de la figure 1.4 montre notamment que le lancement d'une analyse constitue le cas d'utilisation central : il déclenche les traitements internes d'extraction, de détection, de suivi, de recherche contextuelle et de génération des fiches.

Cette distinction entre objectifs utilisateurs et traitements internes sert de base à la conception présentée dans la suite du chapitre. Les composants techniques ne sont donc pas ajoutés comme acteurs ou cas d'utilisation autonomes ; ils sont modélisés dans les vues d'architecture et les diagrammes dynamiques.

4.2 Architecture globale du système

L'architecture du système Vision-RAG repose sur une organisation modulaire visant à assurer la séparation des responsabilités entre les différents composants logiciels. Cette section présente la structure générale du système, en identifiant les principales couches fonctionnelles, leurs composants respectifs ainsi que les interactions qui les relient. Cette vue d'ensemble constitue le cadre de référence pour la description détaillée des modules présentée dans les sections suivantes.

4.2.1 Présentation des composants

Le système Vision-RAG adopte une **architecture trois couches** séparant strictement la présentation, le traitement métier et le stockage des données.

TABLE 4.1 – Composants principaux du système Vision-RAG

Couche	Composant	Responsabilité
Présentation	Application Flutter	Capture vidéo, envoi, affichage et correction des fiches Equipment
	Backend FastAPI	Orchestration du pipeline Vision-RAG, exposition des points d'accès REST
Métier	Pipeline Vision-RAG	Extraction keyframes, détection YOLO-World, déduplication ByteTrack
	Module RAG (`RAGService`)	Recherche multimodale CLIP + MiniLM dans Qdrant
	Module Gemini (`GeminiService`)	Invocation API Gemini, génération fiches JSON
	n8n (externe)	Automatisation post-analyse, persistance Supabase, notifications
Données	Qdrant (en mémoire)	Base vectorielle temporaire (RAG runtime)
	Supabase (PostgreSQL)	Persistance long terme des fiches validées

Note architecturale — statut de n8n

n8n est un **service d'automatisation externe** au backend FastAPI. Il ne fait pas partie du pipeline Vision-RAG et n'est pas instancié par le code Python. Il prend en charge les traitements *post-analyse* : réception des fiches validées via webhook, persistance dans Supabase et envoi de notifications FCM. Il est donc :

- absent du diagramme de classes (service externe non instancié par le backend) ;
- présent dans le schéma d'architecture globale (flux de données) ;
- présent dans le diagramme de déploiement (composant déployé localement).

4.2.2 Schéma d'architecture globale

La figure 4.1 présente le **schéma d'architecture logicielle** du système Vision-RAG. Il illustre les flux de données entre les différentes couches et distingue clairement les composants internes du backend des services externes.

4.3 Conception détaillée des modules

Après avoir présenté l'architecture globale du système, cette section décrit en détail les différents modules qui composent le pipeline Vision-RAG. Chaque module est analysé selon son rôle, son fonctionnement interne et son intégration avec les autres composants, afin de mettre en évidence les choix de conception retenus et leur contribution au fonctionnement global de l'application.

4.3.1 Module de traitement vidéo

Le module de traitement vidéo, implémenté dans `VideoProcessor`, transforme une séquence vidéo brute en un ensemble de frames nettes et diversifiées exploitables par le pipeline Vision-RAG. Il opère en deux étapes séquentielles.

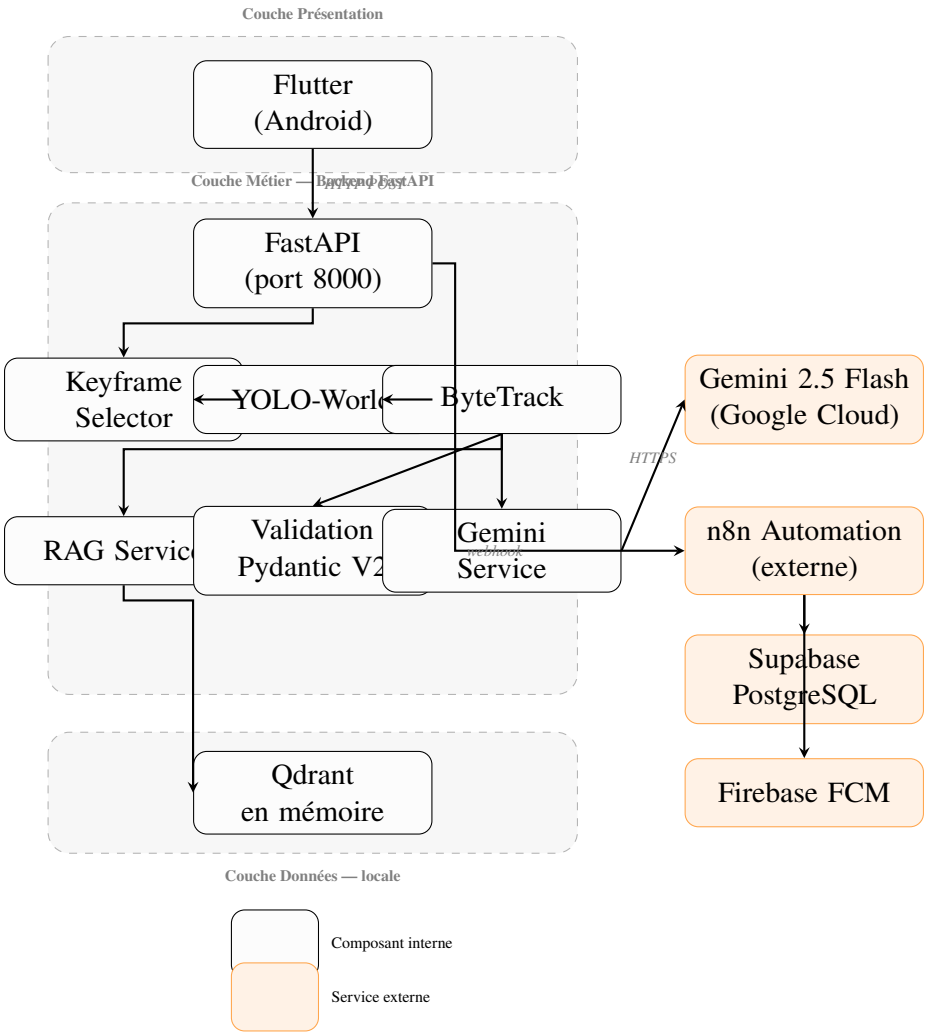


FIGURE 4.1 – Architecture logicielle du système Vision-RAG

Filtrage par qualité. — La netteté de chaque frame est mesurée par la variance du Laplacien. Toute frame dont la variance est inférieure au seuil $\tau = 50$ est rejetée. Ce seuil a été calibré **empiriquement** : une valeur initiale $\tau = 100$, fréquemment utilisée comme valeur de référence dans les travaux exploitant la variance du Laplacien [48], a été abaissée après observation que des équipements en arrière-plan étaient manqués avec cette valeur. Les performances associées sont détaillées au chapitre suivant.

Sélection de diversité. — Parmi les frames nettes retenues, un filtre de diversité par corrélation d'histogrammes HSV élimine les frames redondantes. Le nombre maximal de keyframes est paramétrable (`max_keyframes` = 15 par défaut).

4.3.2 Module de détection

Le module de détection regroupe `YOLOService` et `TrackingService`. Il transforme les keyframes en crops (images recadrées) uniques représentatifs de chaque équipement.

YOLO-World zero-shot et correspondance des classes. — `YOLOService` configure explicitement `YOLOWorld` avec **34 prompts textuels** en français et en anglais couvrant les **8 catégories** du modèle Equipment SFM. Chaque catégorie est couverte par plusieurs variantes lexicales pour maximiser la détection *zero-shot* :

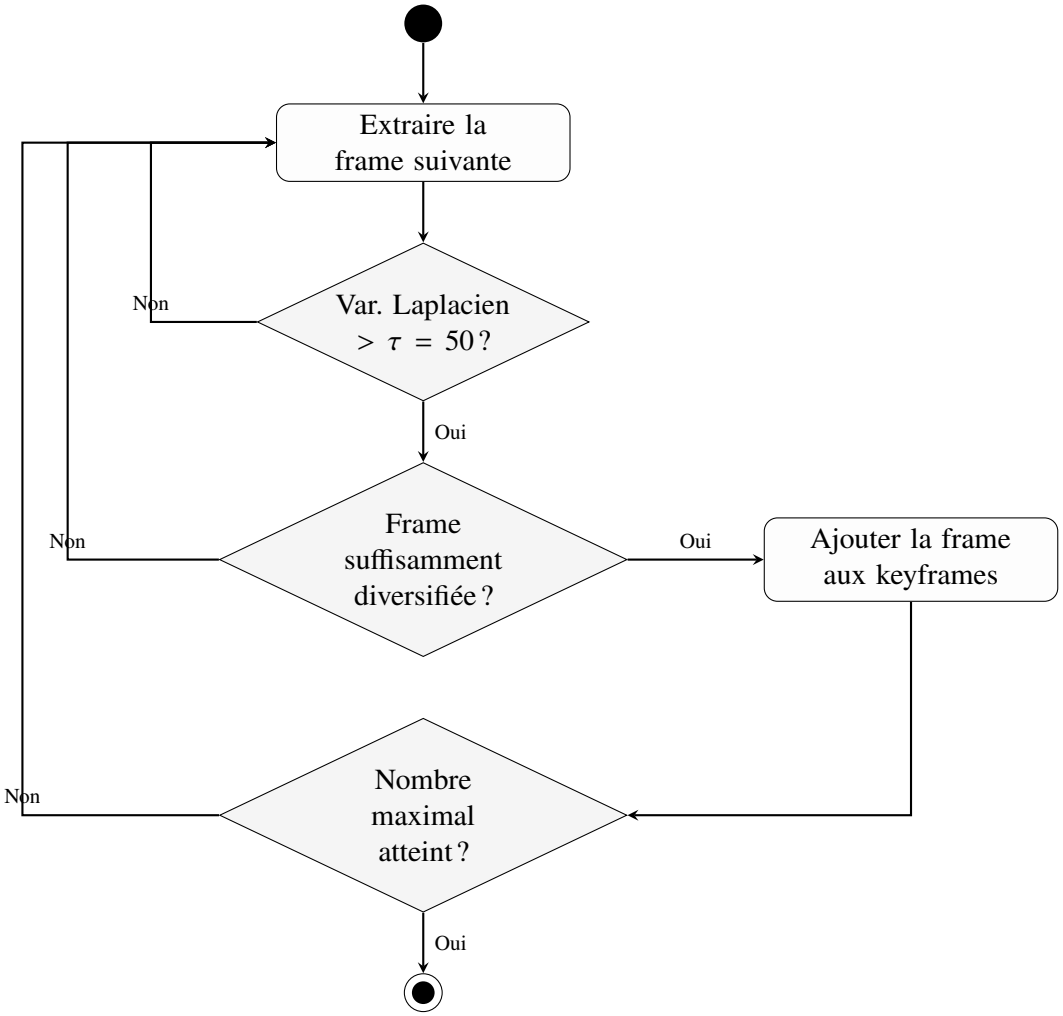


FIGURE 4.2 – Diagramme d'activités du module de traitement vidéo

TABLE 4.2 – Correspondance entre les classes YOLO-World et les catégories SFM

Catégorie SFM	**Classes YOLO-World configurées**
AppareilInformatique	computer, ordinateur, laptop
Climatisation	air conditioner, climatiseur
Réfrigération	refrigerator, réfrigérateur, fridge, freezer
AppareilCuisine	kitchen appliance, appareil de cuisine, microwave, microwave oven, coffee machine, oven
PériphériqueImpression	printer, imprimante
InfrastructureIT	server, serveur informatique, router, routeur, switch, industrial controller, PLC, electrical panel, tableau électrique
Éclairage	lighting, éclairage, light fixture
Autres	faucet, robinet, tap, television, projector

ByteTrack et sélection de la meilleure vue. — `TrackingService` applique l'algorithme ByteTrack sur les détections de toutes les keyframes. Pour chaque *track ID*, le crop de surface maximale est sélectionné :

$$\text{best_shot}(k) = \arg\max_{f \in \mathcal{F}_k} \left[(x_2^{k,f} - x_1^{k,f}) \cdot (y_2^{k,f} - y_1^{k,f}) \right]$$

Ce mécanisme garantit qu'un équipement présent dans plusieurs frames consécutives ne

génère qu'un seul crop unique, éliminant les doublons avant l'invocation de l'API Gemini. Les valeurs expérimentales de réduction obtenues — nombre de détections, crops produits, taux de réduction — sont détaillées au chapitre suivant.

4.3.3 Module RAG (`RAGService`)

Le module RAG (`RAGService`) centralise la logique de récupération contextuelle. Il assure l'enrichissement de chaque crop avec des spécifications techniques issues de la base documentaire, avant toute invocation de l'API Gemini.

Initialisation automatique. — L'index RAG est reconstruit automatiquement au démarrage du serveur via le mécanisme `lifespan` de FastAPI. Ce mécanisme garantit que la base vectorielle Qdrant est disponible dès la première requête, sans intervention manuelle. Le temps de reconstruction est présenté au chapitre suivant.

Stratégie multimodale. — La recherche suit trois niveaux successifs : recherche visuelle CLIP (niveau 1, prioritaire), identification *zero-shot* par CLIP (niveau 2, repli), et recherche textuelle MiniLM (niveau 3). Cette stratégie est détaillée au chapitre 3.

Limites identifiées. — Le module RAG présente trois limites identifiées et gérées :

- **Latence sur CPU** : l'encodage CLIP d'un crop introduit une latence sur CPU, acceptable dans le contexte du projet mais susceptible de constituer un goulot d'étranglement en cas de traitement parallèle massif. Les mesures correspondantes sont détaillées au chapitre suivant ;
- **Qdrant vide au démarrage** : si la reconstruction de l'index échoue (réseau indisponible, sources bloquées), le système bascule en mode dégradé : `GeminiService` invoque l'API Gemini sans contexte RAG et toutes les fiches Equipment sont marquées `needsReview = true` ;
- **Mode hors connexion** : l'analyse via l'API Gemini nécessite une connexion Internet. En l'absence de connectivité, seules les étapes YOLO-World et ByteTrack sont exécutables localement, produisant des crops sans fiches Equipment associées.

4.3.4 Module Gemini (`GeminiService`)

Le module Gemini (`GeminiService`) orchestre les invocations de l'API Gemini 2.5 Flash. À partir d'un crop et de son contexte RAG, il produit une fiche Equipment structurée en JSON, conforme au modèle SFM.

Stratégie de prompt engineering. — Le prompt est construit dynamiquement à partir de quatre éléments :

1. **Schéma JSON strict** : listes fermées des désignations et statuts, générées **automatiquement** depuis les enums Pydantic, garantissant la cohérence entre le prompt et le modèle de données ;
2. **Contexte RAG** : spécifications techniques récupérées dans Qdrant, injectées avant l'image pour ancrer la réponse sur des données réelles ;
3. **Règles d'abstention** : instruction explicite de retourner `null` pour tout champ non clairement visible ;
4. **Exemples few-shot** : trois exemples représentatifs (Daikin, Dell et Cisco) illustrant le format de fiche Equipment attendu.

Gestion des erreurs. — En cas de quota dépassé (erreur HTTP 429), un mécanisme de nouvelle tentative automatique est déclenché. Les délais d'attente et le comportement de repli sont décrits

dans le tableau suivant :

TABLE 4.3 – Mécanisme de nouvelle tentative après une erreur HTTP 429

Tentative	Délai	Action	Solution de repli
1	0 s	Invocation initiale	—
2	35 s	Nouvelle tentative	—
3	70 s	Nouvelle tentative	`needsReview = true`

4.4 Diagrammes dynamiques

Après avoir présenté la structure statique du système, il est nécessaire d'analyser son comportement dynamique. Cette section décrit les interactions temporelles entre les différents composants du pipeline Vision-RAG afin d'illustrer le déroulement d'une analyse complète, depuis la capture de la vidéo jusqu'à la production des fiches Equipment.

4.4.1 Diagramme de séquence global

Le diagramme de séquence ci-dessous modélise les interactions temporelles entre tous les acteurs et composants lors d'une analyse complète. Le pipeline Vision-RAG est décomposé en ses composants réels pour plus de précision. Les durées expérimentales associées à chaque phase sont présentées au chapitre suivant.

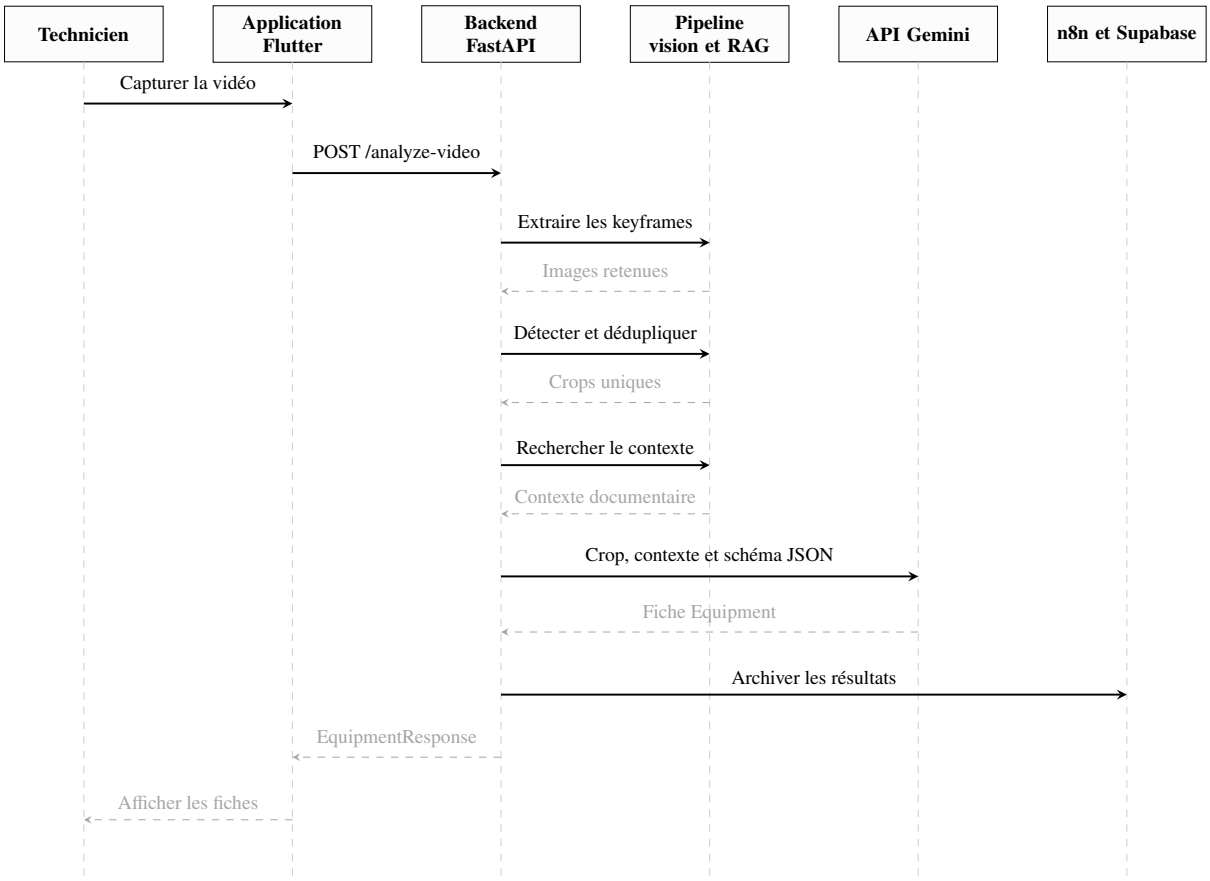


FIGURE 4.3 – Diagramme de séquence du pipeline Vision-RAG complet. Les durées expérimentales sont présentées au chapitre suivant.

4.5 Diagrammes statiques

Les diagrammes statiques constituent une représentation structurelle du système en décrivant les différentes classes qui le composent ainsi que les relations existant entre elles. Cette modélisation permet d'expliciter l'organisation interne du backend Vision-RAG, la répartition des responsabilités entre les composants métiers et les échanges de données, tout en servant de référence pour l'implémentation présentée dans les sections suivantes.

4.5.1 Diagramme de classes métier

Le diagramme suivant représente les classes **internes au backend**, en distinguant les services métier, les entités de domaine et les objets de transfert (DTO). Les services externes (API Gemini, n8n) ne sont pas modélisés ici car ils ne sont pas instanciés par le code Python du backend.
*Note UML : les relations représentées entre `VisionRAGAPI` et les services sont des **dépendances** au sens large (le contrôleur délègue aux services). La notation composition est utilisée pour exprimer la relation de contrôle, conformément aux conventions simplifiées couramment admises dans les rapports d'ingénierie.*

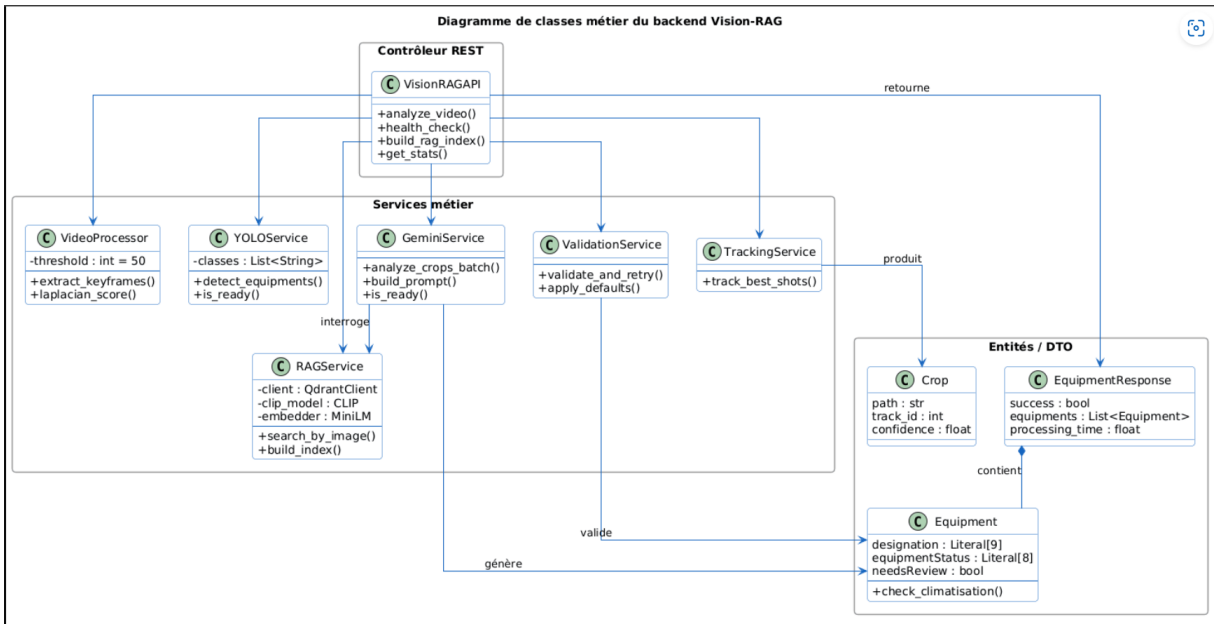


FIGURE 4.4 – Diagramme de classes métier du backend Vision-RAG

4.5.2 Diagramme de composants

Ce diagramme complète le précédent en représentant les interactions entre le pipeline Vision-RAG interne et les services externes.

4.6 Modélisation des données

Cette section présente la modélisation des données du système Vision-RAG. Elle décrit l'architecture de stockage adoptée ainsi que les principaux modèles de données utilisés pour représenter, valider et persister les informations manipulées tout au long du pipeline de traitement. Les choix retenus visent à assurer la cohérence, l'intégrité et la disponibilité des données au sein de l'application.

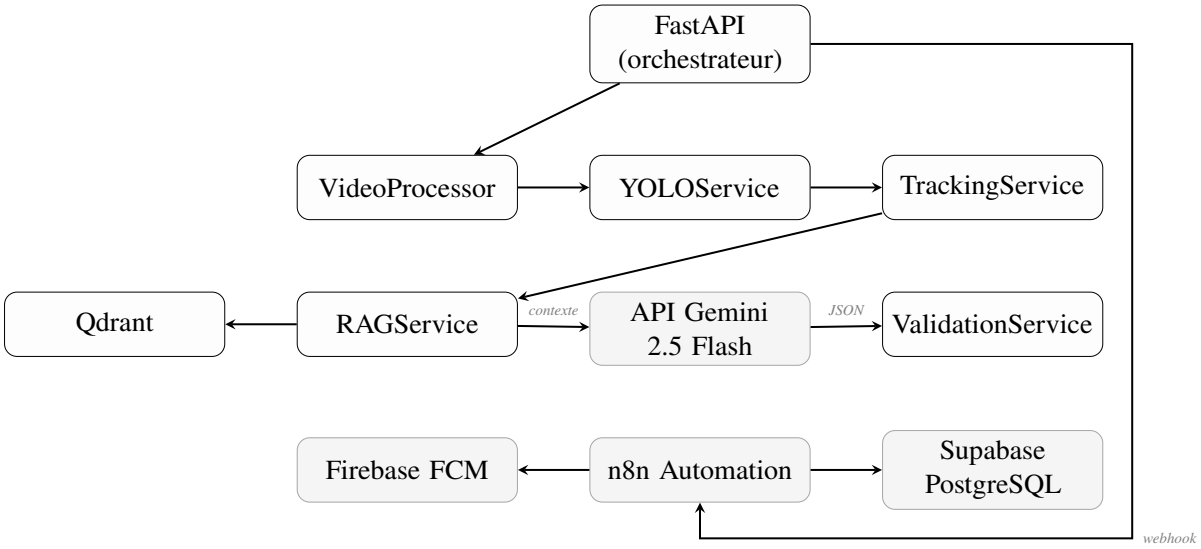


FIGURE 4.5 – Diagramme des composants du pipeline Vision-RAG et des services externes

4.6.1 Architecture des données

Le système Vision-RAG utilise trois mécanismes de stockage distincts et **complémentaires**, chacun répondant à un besoin précis et non interchangeables :

TABLE 4.4 – Justification des trois mécanismes de stockage

Besoin	**Technologie**	**Justification**
Base vectorielle	Qdrant en mémoire	Stockage temporaire des embeddings RAG pendant l'exécution. Réinitialisé à chaque démarrage du serveur. Non persistant par choix : garantit la cohérence des vecteurs sans accumulation de données obsolètes. Le temps de reconstruction est présenté au chapitre suivant.
Base métier	Supabase (PostgreSQL)	Persistance long terme des fiches Equipment validées. Accessible via API REST depuis n8n et depuis l'application Flutter.
Base interne n8n	SQLite	Base interne à n8n uniquement. Stocke les workflows, credentials et historique d'exécution. Non accessible par les autres composants du système.

4.6.2 Modèle Equipment (Pydantic V2)

Le modèle **Equipment** constitue la **source unique de vérité** (*Single Source of Truth*, SSOT) : défini une seule fois dans `models/equipment.py`, il est réutilisé pour la validation FastAPI, la génération automatique du prompt Gemini et la sérialisation vers Supabase.

TABLE 4.5 – Champs du modèle Equipment et contraintes

Champ	Type	Contrainte
`id`	`str`	UUID v4, auto-généré
`designation`	`Literal`	9 valeurs autorisées, obligatoire
`equipmentStatus`	`Literal`	8 valeurs autorisées, obligatoire
`brand`	`Optional[str]`	`null` si non visible dans le crop
`power`	`Optional[int]`	En watts, `null` si non visible
`temperature`	`Optional[float]`	Obligatoire si `Climatisation`; valeur par défaut : 22,0
`coolingCapacity`	`Optional[int]`	Obligatoire si `Climatisation`; valeur par défaut : 9 000
`needsReview`	`bool`	`true` si > 50 % des champs sont `null` ou règle Climatisation appliquée

Le modèle complet repose sur un validateur Pydantic V2 (`@model_validator`) afin de contrôler la cohérence des champs avant l'envoi de la réponse API.

4.6.3 Structure des réponses API

TABLE 4.6 – Champs de la réponse `POST /api/analyze-video`

Champ	Type	Description
`success`	`bool`	`true` si analyse terminée sans erreur
`video_id`	`str`	Identifiant horodaté unique
`location`	`str`	Localisation fournie par Flutter
`equipments`	`List[Equipment]`	Fiches Equipment générées et validées
`processing_time`	`float`	Temps total en secondes
`metadata. keyframes_extracted`	`int`	Keyframes sélectionnées
`metadata. unique_crops`	`int`	Crops après ByteTrack
`metadata. valid_equipments`	`int`	Fiches Equipment sans `needsReview`
`metadata. needs_review`	`int`	Fiches Equipment avec `needsReview = true`

4.7 Diagramme de déploiement

Cette section présente le déploiement sous deux vues complémentaires : une vue logique décrivant les interactions entre composants, et une vue physique précisant les ressources matérielles estimées.
Cette architecture correspond à l'environnement de développement et de validation du prototype. Un déploiement en production nécessiterait une containerisation (Docker) et une orchestration adaptée.

4.7.1 Vue logique — interactions entre composants

Cette sous-section présente la vue logique du système Vision-RAG, en mettant en évidence les interactions entre les différents composants logiciels impliqués dans le traitement d'une

analyse. Elle illustre les échanges de données entre l'application mobile, le backend, les services de stockage et les services cloud, afin de montrer la répartition des responsabilités et les flux de communication au sein de l'architecture.

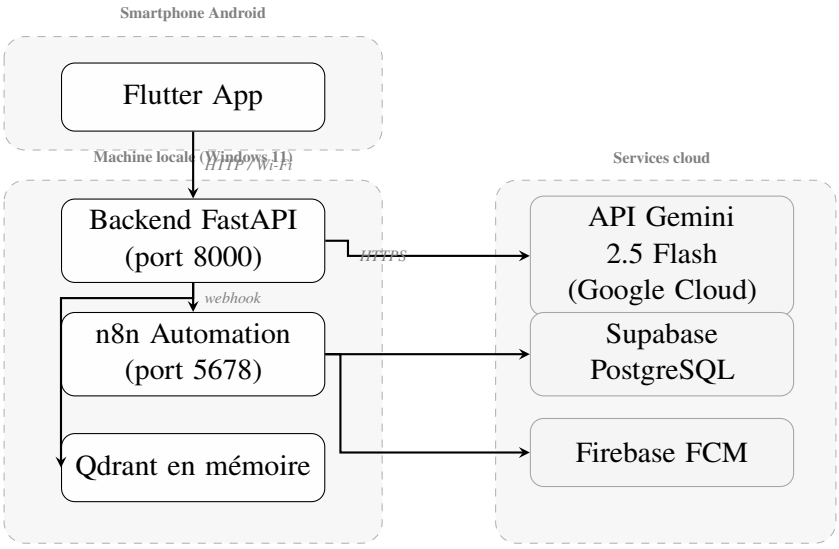


FIGURE 4.6 – Vue logique de l'environnement de développement et de validation

4.7.2 Vue physique et ressources matérielles

Au-delà de l'organisation logique des composants, il est nécessaire d'évaluer les ressources matérielles requises pour exécuter le prototype. Cette sous-section présente la répartition des principaux composants sur les différents nœuds de déploiement ainsi qu'une estimation de leur consommation mémoire, afin d'apprécier les exigences matérielles de la solution et sa compatibilité avec l'environnement de développement retenu.

TABLE 4.7 – Ressources mémoire estimées par composant

Composant	**Nœud**	**RAM estimée**	**Remarque**
FastAPI + services	Machine locale	~ 50 MB	Orchestrateur léger
YOLO-World	Machine locale	~ 200 MB	Chargé une fois au démarrage
CLIP + MiniLM	Machine locale	~ 430 MB	Chargés une fois au démarrage
Qdrant en mémoire	Machine locale	< 50 MB	Reconstruit au démarrage
n8n	Machine locale	~ 200 MB	Service externe d'automatisation
API Gemini 2.5 Flash	Google Cloud	0 MB local	Invocation HTTPS
Supabase + FCM	Cloud externe	0 MB local	Services managés
Total local (estimé)	Machine locale	~ **930 MB***	Compatible 4 GB RAM disponibles

* *Valeur estimée à partir des tailles connues des modèles et des services. Elle correspond au sous-total IA/RAG d'environ 678 MB présenté au chapitre 3, auquel s'ajoutent environ 50 MB pour FastAPI et 200 MB pour n8n, soit 928 MB arrondis à 930 MB. La mesure précise en conditions réelles est présentée au chapitre suivant.*

4.8 Glossaire des termes techniques

TABLE 4.8 – Glossaire des termes du pipeline Vision-RAG

Terme	Définition
Frame	Image individuelle issue d'une séquence vidéo.
Keyframe	Frame sélectionnée par le filtre de netteté (variance du Laplacien $> \tau = 50$) et le critère de diversité HSV.
Crop	Image recadrée extraite par l'algorithme ByteTrack, correspondant au meilleur plan d'un équipement tracké (surface maximale).
Track ID	Identifiant stable assigné par ByteTrack à un objet détecté à travers plusieurs frames consécutives.
Embedding	Représentation vectorielle continue d'un texte ou d'une image dans un espace de haute dimension, utilisée pour mesurer leur similarité sémantique.
Zero-shot	Capacité d'un modèle à reconnaître une classe sans entraînement spécifique sur cette classe, uniquement à partir d'une description textuelle.
Fiche Equipment	Structure JSON produite par le pipeline Vision-RAG pour chaque équipement détecté, conforme au modèle Pydantic V2 SFM.
needsReview	Indicateur booléen positionné à `true` quand une fiche Equipment nécessite une vérification manuelle.
SSOT	*Single Source of Truth* : principe selon lequel le modèle Equipment est défini une seule fois et réutilisé partout dans le système.
DTO	*Data Transfer Object* : objet de transport de données entre deux couches logicielles (`Crop`, `EquipmentResponse`).

L'architecture proposée répond aux besoins du prototype développé. Néanmoins, certaines limites demeurent, notamment la dépendance à une connexion Internet pour l'invocation de l'API Gemini et les performances contraintes d'une exécution exclusivement sur CPU. L'architecture reste néanmoins suffisamment modulaire pour permettre le remplacement futur des modèles IA ou de la base vectorielle sans modifier les autres composants du système.

Conclusion

Ce chapitre a présenté l'architecture complète du système Vision-RAG selon une progression rigoureuse : vue fonctionnelle, schéma d'architecture logicielle, conception détaillée des quatre modules du pipeline Vision-RAG, modélisation des données et déploiement.

Les **cinq diagrammes UML** produits — cas d'utilisation, activités, séquence, classes et déploiement — offrent une vision complète du système, complétés par un schéma d'architecture logicielle et un diagramme de composants illustrant les flux entre le pipeline Vision-RAG et les services externes. Les responsabilités de chaque composant ont été clairement définies : n8n assure l'automatisation des traitements post-analyse, Qdrant le stockage vectoriel temporaire et Supabase la persistance des fiches Equipment. La vue physique du déploiement confirme la compatibilité de l'ensemble avec les contraintes matérielles du projet, avec une charge mémoire estimée à $\approx$ **930 MB**, bien en deçà des 4 GB disponibles sur la machine de développement.

Le chapitre suivant présente l'implémentation concrète de ces composants, les résultats de validation obtenus et les perspectives d'évolution du système.

Chapitre 5

Résultats expérimentaux et validation

Introduction

Après avoir présenté la conception du système Vision-RAG, ce chapitre est consacré à son évaluation expérimentale. L'objectif n'est pas seulement de vérifier que les différents modules fonctionnent séparément, mais aussi d'analyser leur comportement lorsqu'ils sont intégrés dans une même chaîne de traitement, depuis la vidéo d'entrée jusqu'à la génération d'une fiche `Equipment` exploitable.

L'évaluation suit le déroulement réel du pipeline développé : extraction des keyframes, détection des équipements, suivi et déduplication, recherche contextuelle par RAG, analyse multimodale, validation des données et contrôle fonctionnel par l'utilisateur. Cette organisation permet de mettre en évidence les apports de chaque composant, les limites observées et les conditions dans lesquelles le prototype fournit les meilleurs résultats.

5.1 Protocole expérimental

Le protocole expérimental définit les conditions dans lesquelles le prototype a été testé. Il précise les objectifs de l'évaluation, les vidéos utilisées, les critères retenus ainsi que la méthode de mesure appliquée aux résultats.

5.1.1 Objectifs de l'évaluation

L'évaluation poursuit cinq objectifs principaux :

— vérifier la capacité du système à extraire des images pertinentes à partir de vidéos courtes prises par smartphone ;
— mesurer la qualité de la détection et de la déduplication des équipements visibles dans la scène ;
— analyser l'apport du module RAG dans la génération des fiches `Equipment`, notamment pour les champs techniques ;
— vérifier la conformité structurelle et le comportement de bout en bout du prototype ;
— valider l'utilisabilité du prototype dans des scénarios proches d'un audit énergétique réel.

Ces objectifs permettent d'évaluer le système à la fois comme une chaîne de traitement automatique et comme un outil d'aide au travail du technicien.

5.1.2 Niveaux d'évaluation

Trois niveaux complémentaires sont distingués afin d'éviter de confondre la performance des modèles avec le bon fonctionnement de l'application.

TABLE 5.1 – Niveaux d'évaluation du prototype

Niveau	Mesures principales	Question évaluée
Modèles de vision	mAP@50, précision, rappel, IDF1	Les objets sont-ils correctement détectés et suivis par rapport à la vérité terrain ?
Fiches générées	Champs corrects, fiches acceptées, corrigées ou rejetées	Les informations proposées sont-elles exactes et utiles au technicien ?
Intégration système	Requêtes terminées, délai, conformité JSON, erreurs	Le pipeline complet produit-il une réponse exploitable sans défaillance ?

Un test d'intégration réussi ne constitue donc pas, à lui seul, une preuve de précision du détecteur ou du suivi. Inversement, une bonne détection ne garantit pas que la fiche finale respecte le schéma métier.

5.1.3 Jeu de vidéos de test

Les expérimentations sont réalisées à partir de courtes vidéos représentant différents environnements rencontrés lors d'un audit énergétique. Les vidéos sont filmées avec un smartphone afin de rester proches des conditions d'usage visées par le projet.

TABLE 5.2 – Description du jeu de vidéos utilisé pour l'évaluation

Vidéo	Durée	Résolution	Équipements visibles
V1	7,3 s	640×360	Ordinateur Dell, climatiseur Daikin, switch Cisco
V2	20,2 s	360×640	Téléviseur LG
V3	12,5 s	360×640	Réfrigérateur Samsung, four à micro-ondes, switch Cisco
V4	12,1 s	640×360	Four à micro-ondes SILVERCREST, switch Cisco

Ce corpus totalise 52,1 secondes et 1 385 frames. Sa taille limitée ne permet pas de généraliser les résultats à tous les bâtiments tertiaires ; elle convient en revanche à une première validation contrôlée du prototype. Les identifiants V1 à V4 désignent respectivement `ma_video.mp4`, `ma_video1.mp4`, `ma_video2.mp4` et `ma_video3.mp4`.

5.1.4 Critères d'évaluation

Les résultats sont évalués selon des critères liés à la détection, à la qualité des fiches générées et à la validation fonctionnelle du prototype.

TABLE 5.3 – Critères retenus pour l'évaluation du prototype

Critère	Description
mAP@50	Précision moyenne de la détection pour un seuil d'intersection sur union de 0,50.
Précision et rappel	Proportion de détections correctes et proportion d'objets de référence effectivement retrouvés.
IDF1	Cohérence des identités attribuées à un même équipement au fil des keyframes.
Conformité structurelle	Respect du schéma `Equipment` et des contraintes Pydantic.
Exactitude des champs	Proportion de champs corrects, absents ou corrigés après comparaison à une référence vérifiée.
Validation humaine	Répartition des fiches acceptées directement, corrigées ou rejetées par le technicien.
Temps de traitement	Durée de chaque étape et durée totale entre l'envoi de la vidéo et la réponse finale.

5.1.5 Méthode de mesure

La vérité terrain définie au chapitre 3 repose sur l'annotation manuelle de 60 keyframes, comprenant 59 boîtes de référence et trois images négatives. Une prédiction est considérée comme correcte si sa catégorie correspond à la référence et si l'intersection sur union entre les deux boîtes est supérieure ou égale à 0,50. Ces correspondances permettent de calculer la précision, le rappel, le score F1 et le mAP@50. Les annotations LabelImg ne comportant pas d'identifiants de trajectoire, elles ne permettent pas de calculer l'IDF1 de ByteTrack ; cette mesure nécessiterait une seconde annotation temporelle des identités au niveau de chaque vidéo.

Les fiches `Equipment` sont évaluées séparément, champ par champ, à partir des informations visibles et des documents techniques vérifiés. Chaque champ est classé comme correct, absent, incorrect ou non vérifiable. Une information non vérifiable ne doit pas être comptée comme correcte par défaut.

Les temps de traitement sont relevés au niveau des principales étapes du pipeline. Lorsque l'appel à un service externe intervient, notamment pour Gemini, le temps mesuré inclut le délai de réponse de l'API.

5.1.6 Statut des mesures

Les tests de bout en bout déjà exécutés démontrent que le pipeline peut recevoir une vidéo et produire une réponse JSON. Les prédictions ont été générées le 4 août 2026, puis les métriques ont été recalculées le 7 août 2026 après la relecture et la correction de deux annotations. L'IDF1 n'est en revanche pas rapporté, faute d'identifiants de trajectoire dans la vérité terrain. Cette séparation évite d'attribuer à un modèle un score calculé à partir de critères binaires de disponibilité ou de conformité applicative.

Lors de la nouvelle exécution de validation transmise après la correction de YOLO-World, les quatre vidéos ont atteint un état final et six fiches ont été produites. Cinq ont satisfait directement les contrôles structurels et une a été marquée `needsReview`. Ce résultat mesure le comportement de bout en bout et la conformité des objets JSON ; il ne mesure ni la précision de détection ni la qualité du suivi. L'ancien score agrégé sur 100, fondé sur quatre critères binaires, n'est donc pas retenu comme métrique scientifique. Pour la même raison, le verdict automatique « système validé pour production » est interprété comme la réussite de la suite d'intégration et non comme une qualification industrielle.

5.2 Environnement expérimental

Cette section présente l'environnement matériel et logiciel utilisé pour les tests. Ces informations sont importantes pour interpréter les temps de traitement, en particulier les étapes de vision par ordinateur et

les appels à des services externes.

5.2.1 Configuration matérielle

TABLE 5.4 – Configuration matérielle utilisée pendant les tests

Élément	Configuration
Ordinateur	Ordinateur portable sous Windows 11
Processeur	Intel Core i7-1165G7 de 11e génération, 2,80 GHz
Mémoire RAM	16 Go (16 892 956 672 octets détectés)
Carte graphique	Intel Iris Xe intégrée ; inférence réalisée sur CPU
Système d'exploitation	Windows 11

5.2.2 Configuration logicielle

TABLE 5.5 – Configuration logicielle utilisée pendant les tests

Composant	Version ou service utilisé
Python	3.11
FastAPI	0.111, avec Uvicorn 0.30
Flutter	3.44.1, Dart 3.8
YOLO-World	Ultralytics 8.4.39
ByteTrack	supervision 0.24
Gemini	Gemini 2.5 Flash, bibliothèque google-generativeai 0.8.3
Qdrant	qdrant-client 1.12, mode local en mémoire
Supabase	Service cloud
n8n	2.8.4, avec Node.js 20.x

5.3 Évaluation du pipeline Vision-RAG

L'évaluation du pipeline suit les étapes successives de la chaîne de traitement. Cette présentation permet de comprendre l'effet de chaque module sur le résultat final.

5.3.1 Extraction des keyframes

La première étape consiste à extraire des frames de la vidéo, puis à conserver les images suffisamment nettes à l'aide de la variance du Laplacien. Cette sélection réduit le nombre d'images envoyées aux étapes suivantes tout en préservant les frames les plus exploitables.

TABLE 5.6 – Résultats de l'extraction et de la sélection des keyframes

Vidéo	Frames extraites	Frames retenues	Taux retenu	Observation
V1	177	15	8,5 %	Séquence courte au format paysage
V2	601	15	2,5 %	Séquence la plus longue du corpus
V3	284	15	5,3 %	Vidéo portrait
V4	323	15	4,6 %	Présence de détections instables

L'évaluation a été réalisée sur les 60 keyframes annotées, au seuil de confiance de 0,25 configuré dans le backend et avec un seuil d'IoU de 0,50 pour l'appariement à la vérité terrain. Le mAP@50 est moyenné sur les cinq classes effectivement présentes dans les annotations. Les classes `computer_monitor` et `network_switch`, absentes de ce corpus annoté, ne sont pas incluses dans cette moyenne.

La figure 5.1 présente cinq exemples retenus parmi les annotations manuelles. Ils couvrent les cinq catégories effectivement représentées dans la vérité terrain utilisée pour le calcul des métriques.

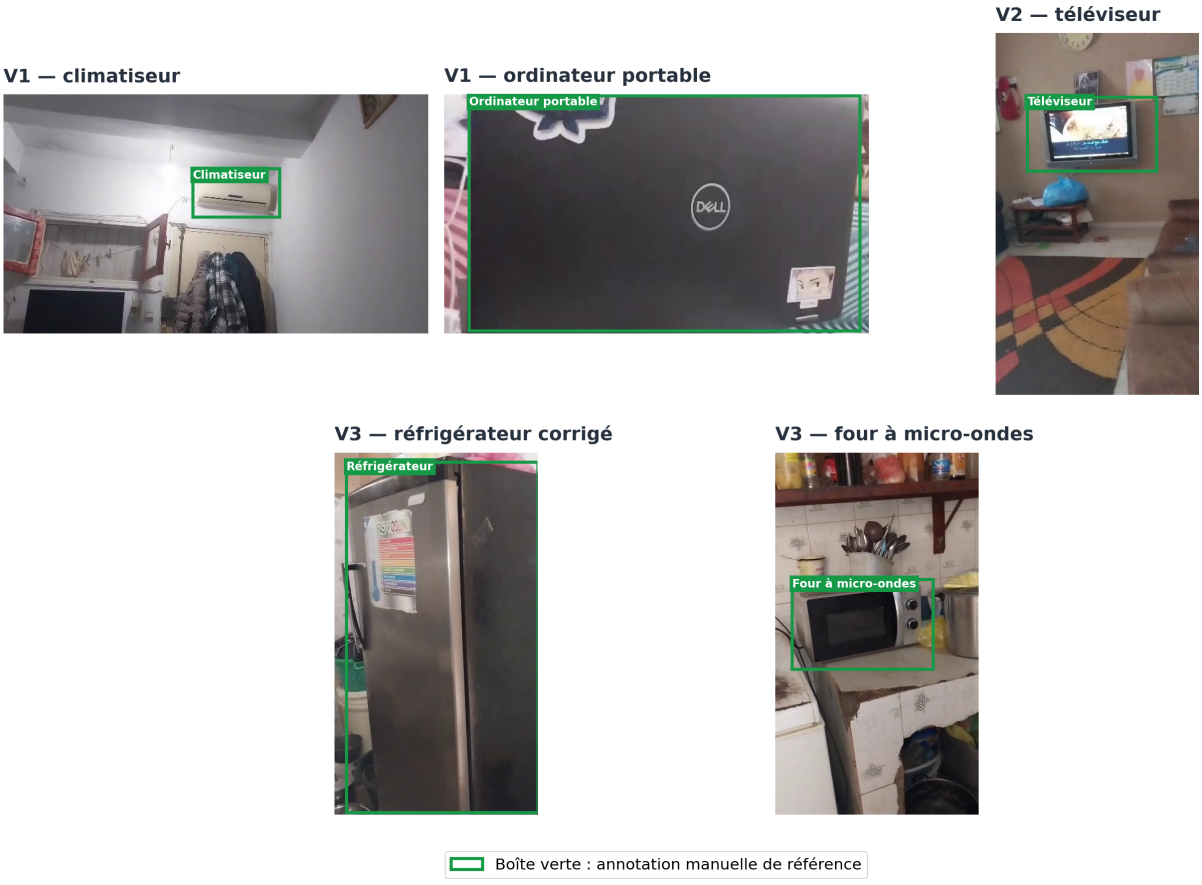


FIGURE 5.1 – Exemples de keyframes annotées constituant la vérité terrain

TABLE 5.7 – Métriques de détection YOLO-World

Métrique	Résultat	Seuil attendu
Précision micro	63,0 %	Mesure descriptive
Rappel micro	78,0 %	Mesure descriptive
Score F1 micro	69,7 %	Mesure descriptive
mAP@50	74,9 %	BNF1 : au moins 70 %

Les 73 prédictions appartenant aux sept classes de référence se répartissent en 46 vrais positifs, 27 faux positifs et 13 faux négatifs. La relecture de la vérité terrain a permis de reclasser correctement trois correspondances : les deux équipements visibles dans `V1_004` et le réfrigérateur de `V3_010`. Les prédictions du modèle n'ont pas été modifiées entre les deux calculs.

5.3.2 Détection YOLO-World

Les keyframes retenues sont analysées par YOLO-World afin de localiser les équipements ciblés. L'intérêt de ce modèle est de permettre une détection *open-vocabulary*, adaptée aux catégories d'équipements rencontrées dans le projet.

Une première exécution de l'évaluateur a révélé que le service retournait les classes COCO génériques, telles que *person* ou *sports ball*, au lieu du vocabulaire configuré. Le chargement a été corrigé en utilisant explicitement la classe `YOLOWorld` et le fichier `yolov8x-worldv2.pt`. La liste des 34 prompts personnalisés a ensuite été vérifiée avant de relancer l'évaluation. Les prédictions ci-dessous correspondent à cette exécution corrigée du 4 août 2026. Leur comparaison avec la vérité terrain relue a été recalculée le 7 août 2026, sans nouvelle inférence du modèle.

TABLE 5.8 – Résultats de détection par type d'équipement

Équipement	Réf.	Préd.	VP	FP	FN	Observation
Climatiseur	10	10	10	0	0	Toutes les occurrences sont correctement détectées
Ordinateur portable	5	3	3	0	2	Précision élevée, mais deux objets manqués
Écran informatique	0	0	0	0	0	Classe absente des keyframes annotées
Switch réseau	0	0	0	0	0	Classe absente des keyframes annotées
Téléviseur	14	30	14	16	0	Tous les objets retrouvés, mais surdétection importante
Réfrigérateur	17	9	6	3	11	Rappel encore faible et onze objets manqués
Four à micro-ondes	13	21	13	8	0	Toutes les références retrouvées, avec huit faux positifs

TABLE 5.9 – Métriques YOLO-World par classe représentée

Classe	Précision	Rappel	F1	AP@50
Climatiseur	100,0 %	100,0 %	100,0 %	100,0 %
Ordinateur portable	100,0 %	60,0 %	75,0 %	60,0 %
Téléviseur	46,7 %	100,0 %	63,6 %	96,9 %
Réfrigérateur	66,7 %	35,3 %	46,2 %	35,3 %
Four à micro-ondes	61,9 %	100,0 %	76,5 %	82,3 %

Le climatiseur constitue la classe la mieux maîtrisée, avec dix détections correctes sur dix références. Le téléviseur conserve un rappel de 100 %, mais sa précision de 46,7 % révèle encore 16 faux positifs. Le réfrigérateur reste la principale faiblesse avec un rappel de 35,3 %, malgré l'amélioration apportée par la correction de l'annotation de `V3_010`. Les sorties associées à des prompts hors du périmètre annoté (panneau électrique, luminaire, projecteur, imprimante et four) ont été conservées dans les journaux, mais exclues des métriques des sept classes de référence.

La figure 5.2 complète ces résultats numériques par trois cas représentatifs. Le vrai positif montre un recouvrement presque total entre la boîte prédite et la référence. Le faux positif illustre la tendance du modèle à confondre certaines zones sombres avec un téléviseur. Le faux négatif correspond à un ordinateur portable présent dans la vérité terrain mais absent des prédictions.

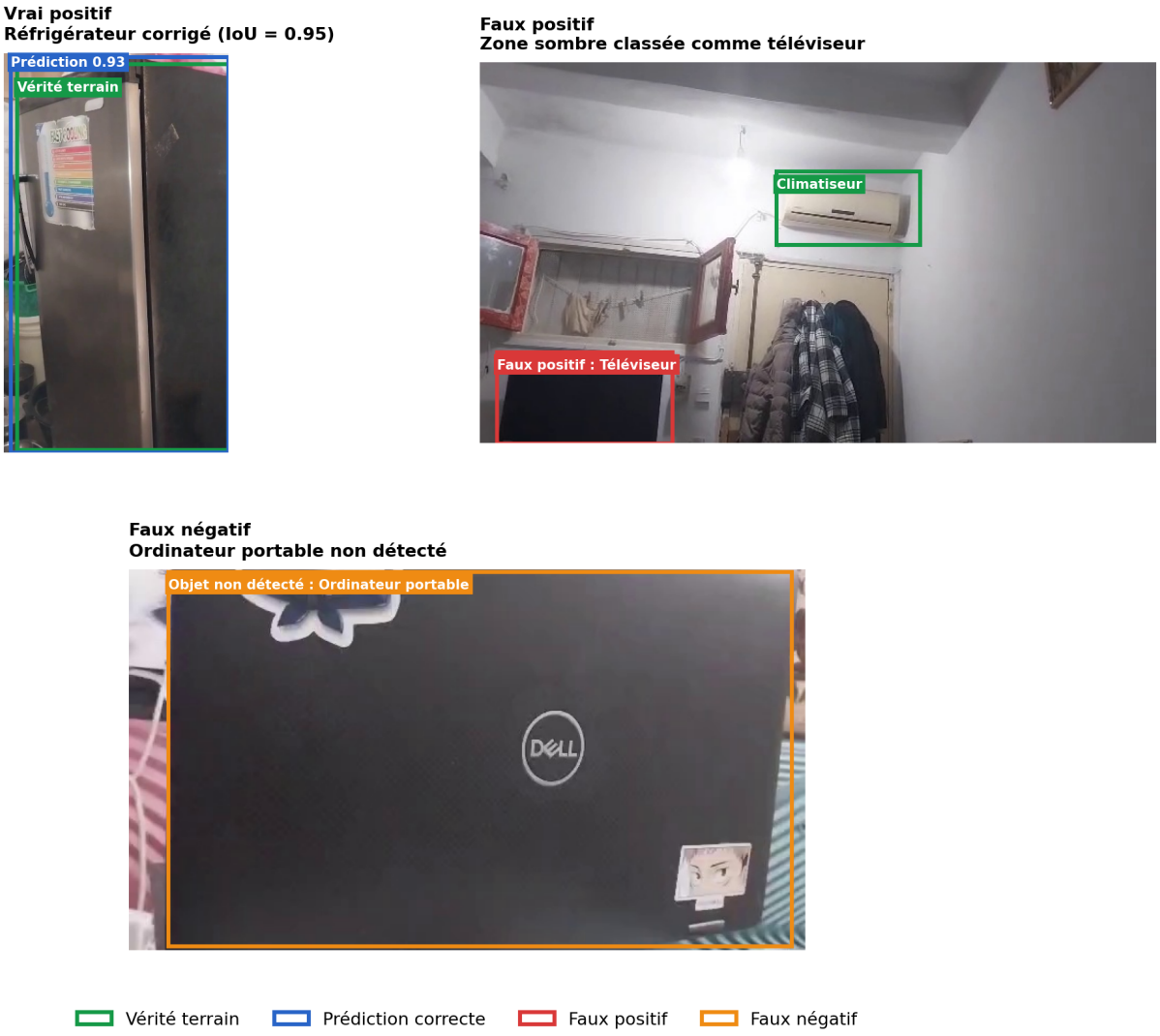


FIGURE 5.2 – Comparaison qualitative de la vérité terrain et des prédictions : vrai positif, faux positif et faux négatif

5.3.3 Déduplication ByteTrack

ByteTrack est utilisé pour regrouper les détections correspondant au même équipement lorsqu'il apparaît sur plusieurs frames. Cette étape évite de générer plusieurs fiches pour un seul appareil.

TABLE 5.10 – Effet de la déduplication sur le nombre de crops retenus

Vidéo	Détections brutes	Tracks retenus	Crops finaux	Commentaire
V1	40	3	3	Trois équipements conservés
V2	13	1	1	Un téléviseur conservé
V3	23	2	2	Deux trajectoires produisent deux crops ; Gemini les regroupe en une fiche
V4	10	1	1	Le switch instable n'est pas retenu

La réduction du nombre de détections montre l'effet opérationnel de la déduplication, mais elle ne

suffit pas à évaluer la cohérence des identités. L'annotation LabelImg réalisée pour la détection ne contient pas d'identifiants temporels. L'IDF1 ne peut donc pas être calculé avec la vérité terrain actuelle.

TABLE 5.11 – Métrique de suivi ByteTrack

Métrique	Résultat	Seuil attendu
IDF1	Non mesurable avec les annotations actuelles	BNF2 non évalué

5.3.4 Recherche contextuelle RAG

Le module RAG intervient après l'extraction des crops d'équipements. Il permet de retrouver des informations proches dans la base documentaire et de fournir à Gemini un contexte plus précis pour la génération des fiches.

TABLE 5.12 – Qualité des informations retrouvées par le module RAG

Vidéo	Contexte trouvé	Score CLIP	Observation
V1	3 résultats textuels par crop	0,40 ; 0,28 ; 0,46	Aucun appariement d'image ; labels *laptop* ou *refrigerator*
V2	3 résultats textuels	0,68	Repli textuel après absence d'appariement visuel
V3	3 résultats textuels	0,51	Label *refrigerator*, à confirmer par référence métier
V4	3 résultats textuels	0,48	Label *laptop computer*, ambigu pour la scène observée

Le point d'accès `/api/stats` indique 11 entrées textuelles et aucune image indexée. Les six fichiers Cisco présents sur disque correspondent à trois images dupliquées, mais ne sont pas chargés dans la collection visuelle. Le RAG fonctionne donc actuellement par recherche textuelle de repli. Ces observations ne suffisent pas à démontrer un gain de précision. L'étude d'ablation avec et sans RAG, présentée plus loin dans ce chapitre, confirme cette limite sur V1.

5.3.5 Génération des fiches Equipment

À partir du crop de l'équipement et du contexte fourni par le RAG, Gemini génère une fiche structurée conforme au modèle `Equipment`. L'évaluation porte sur la présence et la justesse des champs principaux.

TABLE 5.13 – Évaluation des champs des fiches `Equipment`

Champ	Correct	À corriger	Remarques
Désignation	Non évalué	Non évalué	Absence de référence champ par champ validée par un technicien
Catégorie	Non évalué	Non évalué	Évaluation métier distincte de la conformité Pydantic
Marque	Non évalué	Non évalué	Certaines marques sont visibles, mais aucune grille formelle n'a été remplie
État de fonctionnement	Non vérifiable	Non vérifiable	Une image fixe ne suffit pas à confirmer l'état réel
Spécifications techniques	Non évalué	Non évalué	Informations souvent absentes de l'image et du corpus RAG

5.3.6 Conformité structurelle avec Pydantic

La validation Pydantic constitue la dernière étape automatique du pipeline. Elle vérifie que les fiches produites respectent le schéma attendu et signale les cas nécessitant une revue humaine. Une fiche conforme au schéma peut néanmoins contenir une information métier incorrecte : cette vérification ne doit donc pas être présentée comme une mesure de précision du modèle.

TABLE 5.14 – Résultats de la validation automatique des fiches

Vidéo	Fiches générées	Fiches valides	Revue requise	Cause principale
Vidéo 1	3	2	1	Une fiche marquée `needsReview`
Vidéo 2	1	1	0	Fiche structurellement valide
Vidéo 3	1	1	0	Fiche structurellement valide
Vidéo 4	1	1	0	Fiche structurellement valide

Au total, cinq fiches sur six, soit 83,3 %, passent directement les contrôles structurels. Ce taux mesure la conformité au schéma et non l'exactitude métier des valeurs générées.

5.4 Évaluation des performances

Les performances sont évaluées principalement à travers le temps de traitement. Les mesures CPU ou mémoire ne sont pas détaillées ici, sauf si elles sont relevées de manière fiable pendant les expérimentations.

5.4.1 Temps par étape

TABLE 5.15 – Temps moyen de traitement par étape du pipeline

Étape	Temps moyen	Commentaire
Extraction des keyframes	0,85 s	Moyenne des quatre vidéos
Détection YOLO-World	8,80 s	Étape dominante sur CPU
Déduplication ByteTrack	0,03 s	Coût négligeable après détection
Recherche RAG et analyse Gemini	7,06 s	Mesure combinée disponible dans les journaux
Validation Pydantic	$< 0,01$ s	Validation locale du schéma `Equipment`
Temps interne du pipeline	16,74 s	Moyenne mesurée par le backend

5.4.2 Temps global par vidéo

TABLE 5.16 – Temps global de traitement par vidéo

Vidéo	Durée vidéo	Temps total	Observation
Vidéo 1	7,3 s	19,25 s	3 crops et 40 détections brutes
Vidéo 2	20,2 s	15,68 s	1 crop et 13 détections brutes
Vidéo 3	12,5 s	15,56 s	2 crops et 23 détections brutes
Vidéo 4	12,1 s	16,48 s	1 crop et 10 détections brutes

5.4.3 Analyse des performances

La détection YOLO-World représente environ 52,5 % du temps interne moyen. La recherche RAG combinée à Gemini représente 42,2 %, et l'extraction des keyframes 5,1 %. ByteTrack et Pydantic ont un coût négligeable. L'optimisation prioritaire concerne donc l'inférence YOLO sur CPU, par réduction raisonnée du nombre de keyframes, utilisation d'un modèle plus léger ou ajout d'une accélération matérielle. Le délai Gemini reste dépendant du réseau et du quota du service externe.

La suite de tests mesure par ailleurs un temps de réponse externe moyen de 21,4 s sur les quatre vidéos, contre 16,74 s pour l'instrumentation interne du pipeline. L'écart moyen de 4,66 s inclut notamment l'envoi de la requête, les échanges HTTP et la tentative de webhook n8n. Lors de cette exécution, n8n n'était pas démarré : chaque notification a donc échoué après un délai d'attente, sans empêcher le backend de retourner une réponse HTTP 200.

5.4.4 Test de robustesse et effet du quota externe

Un premier test séquentiel de robustesse a été lancé avec cinq keyframes par requête. Les quinze premières requêtes ont reçu une réponse, puis le test a été interrompu pendant la seizième. Le tableau distingue les exécutions normales des réponses dégradées par le quota Gemini.

TABLE 5.17 – Résultats du premier test séquentiel de robustesse

Requêtes	Nombre	Temps moyen	Interprétation
1 à 12	12	26,46 s	Analyses terminées avant saturation du quota
13 à 15	3	107,07 s	Réessais après erreurs HTTP 429 ; certaines réponses ne contiennent aucune fiche
Ensemble	15	42,58 s	Backend toujours disponible, mais service métier dégradé
16	1	Non terminée	Interruption manuelle après dépassement du délai attendu

Le mécanisme de reprise traite également une réponse JSON Gemini mal formée : une nouvelle tentative a alors permis de terminer l'analyse. En revanche, les erreurs de quota persistantes conduisent, après trois tentatives, à une réponse HTTP 200 sans équipement. Ce premier essai ne validait donc pas le critère BNF4 de 95 % de succès sur vingt requêtes.

Le test a été répété le 9 août 2026 avec le même script et vingt requêtes consécutives. Les vingt analyses ont terminé sans erreur, sans réponse HTTP 429 et sans redémarrage du backend. Les temps s'étendent de 11,5 à 23,0 s, avec une moyenne de 14,0 s, une médiane de 13,4 s et un 95e centile de 16,2 s.

TABLE 5.18 – Résultats de la répétition du test de robustesse le 9 août 2026

Indicateur	Résultat	Interprétation
Requêtes terminées	20/20	Taux de succès de 100 %
Échecs et erreurs HTTP 429	0	Aucun dépassement de quota observé
Temps moyen	$14,0 \pm 2,4$ s	Variabilité limitée, avec une requête à 23,0 s
Médiane	13,4 s	La moitié des requêtes termine en moins de 13,4 s
Étendue	11,5 à 23,0 s	Toutes les réponses restent sous le seuil BNF3 de 60 s

Cette répétition valide BNF4 et BNF7 dans les conditions du test. Elle montre que le problème de quota observé lors du premier essai ne s'est pas reproduit. Elle ne supprime toutefois pas la dépendance à un service externe : la surveillance des erreurs HTTP 429, un mécanisme de temporisation et un mode dégradé restent nécessaires pour un usage régulier.

5.5 Validation fonctionnelle

La validation fonctionnelle reprend les quatre vidéos réellement utilisées, sans leur attribuer a posteriori des scénarios qui ne correspondent pas à leur contenu. Elle vérifie que chaque requête atteint un état final, que les crops attendus sont produits et qu'une fiche peut être affichée puis soumise à la validation humaine.

5.5.1 Résultats par vidéo

TABLE 5.19 – Validation fonctionnelle sur le corpus réel

Vidéo	Équipements visibles	Crops retenus	Observation
V1	Ordinateur Dell, climatiseur Daikin, switch Cisco	3	Trois crops sont retenus ; leur identité est vérifiée à l'étape suivante
V2	Téléviseur LG	1	Le téléviseur est conservé après déduplication
V3	Réfrigérateur Samsung, four à micro-ondes, switch Cisco	2	Deux crops sont retenus, puis regroupés en une fiche par Gemini
V4	Four à micro-ondes SILVERCREST, switch Cisco	1	Le switch apparaît trop brièvement pour former une trajectoire fiable

Ces résultats décrivent le comportement observé du pipeline. Le fait qu'un crop soit produit ne signifie pas automatiquement que sa classe ou sa fiche est correcte ; ces deux aspects sont évalués séparément par les métriques de vision et par la validation des champs.

5.5.2 Tests des interfaces et cas limites

TABLE 5.20 – Tests fonctionnels complémentaires du backend

Test	Condition	Résultat
Fichier absent	Requête sans champ vidéo	HTTP 422
Localisation vide	Vidéo valide, localisation vide et 5 keyframes	HTTP 200, une fiche à réviser
Statistiques RAG	Appel de `/api/stats`	11 entrées texte, 0 image
Concurrence	Deux analyses lancées en parallèle	Deux réponses HTTP 200
Webhook n8n	Workflow volontairement arrêté pendant cette exécution	Quatre avertissements non bloquants ; réponses HTTP maintenues

Le test concurrent confirme que le serveur accepte deux requêtes, mais il ne constitue pas un test de charge. Dans la nouvelle exécution, les deux requêtes parallèles ont obtenu une réponse HTTP 200. Une réponse JSON Gemini mal formée a également été récupérée à la deuxième tentative. L'arrêt volontaire de n8n explique les avertissements de connexion et ne remet pas en cause la validation directe antérieure du workflow, de l'insertion Supabase et de la génération du rapport HTML.

5.5.3 Validation des fiches par le technicien

La validation humaine constitue une étape importante du prototype. Le système ne remplace pas totalement le technicien : il prépare une fiche structurée, puis laisse à l'utilisateur la possibilité de confirmer ou de corriger les informations proposées.

TABLE 5.21 – Résultats de validation humaine des fiches générées

Vidéo	Acceptées directement	Corrigées	Rejetées	Champs modifiés
V1	Non mesuré	Non mesuré	Non mesuré	Aucune campagne formelle de validation humaine
V2	Non mesuré	Non mesuré	Non mesuré	Aucune campagne formelle de validation humaine
V3	Non mesuré	Non mesuré	Non mesuré	Aucune campagne formelle de validation humaine
V4	Non mesuré	Non mesuré	Non mesuré	Aucune campagne formelle de validation humaine

5.6 Impact du module RAG

L'apport du RAG constitue l'un des points les plus importants de l'évaluation. Cette section compare les fiches générées avec et sans contexte documentaire afin de mesurer l'effet de la recherche contextuelle sur la qualité des champs produits.

5.6.1 Résultats sans RAG

Sans contexte RAG, le modèle multimodal s'appuie principalement sur l'image de l'équipement et sur les instructions du prompt. Cette configuration permet d'évaluer la capacité du modèle à identifier visuellement les équipements, mais elle limite l'accès aux informations techniques spécifiques. Trois exécutions ont été réalisées sur la vidéo V1 le 7 août 2026. Gemini seul produit trois fiches à chaque essai, en 36,8 s, 35,6 s et 39,1 s, soit une moyenne de $37,2 \pm 1,8$ s. Les désignations restent cohérentes et la marque Dell est reconnue pour l'ordinateur portable, mais les champs `model`, `power` et `voltage` restent absents dans les neuf fiches produites.

5.6.2 Résultats avec RAG

Avec le RAG, le prompt transmis au modèle est enrichi par des informations issues de la base documentaire. Cette approche vise à améliorer la précision des champs tels que la marque, le modèle, la

puissance ou les caractéristiques techniques. Sur la même vidéo et avec les mêmes paramètres, le pipeline complet produit également trois fiches lors de chaque exécution. Les temps obtenus sont de 46,5 s, 43,8 s et 43,3 s, soit une moyenne de $44,5 \pm 1,7$ s. Le climatiseur est systématiquement marqué pour revue, tandis que l'écran et l'ordinateur portable passent directement la validation structurelle. Aucun des trois essais ne renseigne cependant le modèle, la puissance ou la tension.

5.6.3 Analyse comparative

TABLE 5.22 – Temps des trois exécutions avec et sans contexte RAG

Exécution	Sans RAG	Avec RAG	Écart
1	36,8 s	46,5 s	+9,7 s
2	35,6 s	43,8 s	+8,2 s
3	39,1 s	43,3 s	+4,2 s
Moyenne	$37,2 \pm 1,8$ **s**	$44,5 \pm 1,7$ **s**	**+7,4 s, soit +19,8 %**

TABLE 5.23 – Comparaison agrégée des fiches générées avec et sans contexte RAG

Critère	Sans RAG	Avec RAG	Analyse
Équipements produits	3 par essai	3 par essai	Nombre stable sur les trois répétitions
Désignation	9 fiches cohérentes	9 fiches cohérentes	Valeurs identiques
Marque	Dell reconnu 3/3	Dell reconnu 3/3	Aucun apport observé
Modèle, puissance, tension	27 valeurs nulles	27 valeurs nulles	Aucun enrichissement technique
État de fonctionnement	Valeurs variables	Valeurs variables	5 correspondances sur 9 ; quatre différences non vérifiables
Confiance de l'écran	90 % en moyenne	81,7 % en moyenne	Baisse moyenne de 8,3 points avec RAG
Temps total moyen	$37,2 \pm 1,8$ s	$44,5 \pm 1,7$ s	Surcoût moyen de 7,4 s, soit 19,8 %
Marques à confiance < 50 %	0 sur 3 essais	0 sur 3 essais	Aucun cas selon l'indicateur du script

Sur les six champs comparés pour chacune des trois fiches et des trois répétitions, 50 valeurs sur 54 sont identiques, soit 92,6 %. Les quatre différences concernent uniquement l'état de fonctionnement. Ce champ varie également d'une exécution à l'autre pour une même configuration. Comme il ne peut pas être déduit avec certitude d'une image fixe, ces variations ne peuvent pas être interprétées comme une amélioration sans validation par le technicien.

Cette comparaison repose toujours sur une seule vidéo, mais les trois répétitions réduisent l'influence d'un appel isolé à Gemini et permettent d'observer la variabilité des réponses. Elle demeure une étude d'ablation exploratoire plutôt qu'une preuve statistique de supériorité d'une configuration.

Le RAG n'apporte donc, sur ces essais du 7 août 2026, aucun gain démontré sur les champs techniques. Il augmente le temps moyen de 19,8 % et diminue de 8,3 points la confiance moyenne associée à l'écran. Ce résultat est cohérent avec l'état actuel de la base, limitée à 11 entrées textuelles et sans image indexée. Il ne remet pas en cause le principe du RAG, mais montre que son intérêt dépend de la qualité, de la couverture et de la pertinence du corpus documentaire.

L'indicateur d'« hallucination » utilisé par le script compte uniquement les marques renseignées avec une confiance inférieure à 50 %. Il vaut zéro dans les deux configurations pour les trois exécutions, mais ne constitue pas une mesure complète des hallucinations. De même, `needsReview` n'est pas comparé, car le point d'accès sans RAG ne retourne pas ce champ dans ce test.

5.7 Discussion

Cette section synthétise les résultats obtenus et met en perspective les points forts et les limites du prototype.

5.7.1 Forces du système

Les principaux points forts observés du système sont les suivants :

— une chaîne de traitement complète allant de la vidéo à une fiche structurée ;
— une réduction du travail manuel de recensement des équipements ;
— une meilleure standardisation des fiches grâce au modèle `Equipment` ;
— un détecteur *zero-shot* atteignant un mAP@50 de 74,9 % sur les cinq classes représentées dans le corpus ;
— une robustesse applicative face à une localisation vide, une requête invalide, une réponse Gemini mal formée, deux requêtes parallèles et vingt requêtes consécutives terminées sans erreur ;
— une validation humaine conservée pour sécuriser les résultats avant leur exploitation.

5.7.2 Limites observées

Les résultats montrent une surdétection importante du téléviseur, avec 16 faux positifs, ainsi qu'un rappel limité à 35,3 % pour le réfrigérateur. Les classes écran informatique et switch réseau ne sont pas représentées dans les keyframes annotées et ne peuvent donc pas être évaluées. L'IDF1 reste indisponible faute d'identifiants de trajectoire dans la vérité terrain. Le RAG est limité à 11 entrées textuelles sans image indexée et n'a démontré aucun gain technique au cours des trois répétitions de l'étude d'ablation. Enfin, les fiches n'ont pas fait l'objet d'une campagne formelle de validation champ par champ par un technicien.

5.7.3 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées : enrichissement de la base documentaire, ajout de nouvelles catégories d'équipements, amélioration du prompt Gemini, optimisation du nombre de keyframes, conservation d'un historique des corrections effectuées par les techniciens et réutilisation de ces corrections pour améliorer progressivement le système.

Conclusion

Ce chapitre a évalué le prototype Vision-RAG depuis l'extraction des keyframes jusqu'à la production des fiches structurées. Après correction du chargement de YOLO-World, le détecteur atteint un mAP@50 de 74,9 %, une précision micro de 63,0 % et un rappel micro de 78,0 % sur 59 objets annotés. La suite d'intégration termine les quatre vidéos avec un temps externe moyen de 21,4 s, cinq fiches structurellement valides et une fiche à réviser. La répétition du test de robustesse termine par ailleurs vingt requêtes consécutives sur vingt, sans erreur HTTP 429, en 14,0 s en moyenne.

Ces résultats valident la faisabilité technique du prototype, mais mettent en évidence des limites de précision par classe, de suivi annoté, de couverture du RAG et de validation métier. Le système doit donc être présenté comme un outil d'assistance nécessitant une revue humaine, et non comme une solution autonome déjà qualifiée pour un déploiement industriel.

Chapitre 6

Mise en œuvre et déploiement du prototype

Introduction

Après la conception de l'architecture et l'évaluation expérimentale du pipeline Vision-RAG, ce chapitre présente sa mise en œuvre sous la forme d'un prototype opérationnel. L'objectif est de décrire la manière dont les différents composants ont été intégrés, depuis l'application Android jusqu'à l'archivage des fiches dans Supabase.

Le terme *déploiement* désigne ici un environnement de démonstration et de validation. Le backend, Qdrant et n8n s'exécutent sur une machine locale sous Windows 11, tandis que Gemini et Supabase sont consommés comme des services cloud. Il ne s'agit donc pas encore d'un déploiement de production accessible publiquement. Cette distinction est importante, car plusieurs mécanismes attendus dans un système industriel — authentification, terminaison HTTPS, adresse réseau stable et supervision continue — restent à mettre en place.

Le chapitre présente successivement le périmètre fonctionnel atteint, l'implémentation du backend, l'application mobile, l'automatisation avec n8n, la procédure d'exécution, les vérifications opérationnelles ainsi que les limites du déploiement actuel.

6.1 Périmètre fonctionnel du prototype

La version finale du prototype couvre le parcours principal d'un audit : sélection d'une vidéo, transmission au backend, analyse des équipements, affichage des fiches, correction manuelle et archivage. Le tableau 6.1 confronte les besoins fonctionnels définis au chapitre 1 à l'état réel de l'implémentation.

TABLE 6.1 – État d'implémentation des besoins fonctionnels

ID	Réalisation dans le prototype	Statut
BF1	Capture d'une vidéo ou sélection depuis la galerie Android	Implémenté
BF2	Envoi multipart vers le point d'accès `/api/analyze-video`	Implémenté
BF3	Extraction et sélection des keyframes par `VideoProcessor`	Implémenté
BF4	Détection *zero-shot* par `YOLOService`	Implémenté
BF5	Suivi, déduplication et sélection du meilleur crop par `TrackingService`	Implémenté
BF6	Recherche textuelle active ; recherche visuelle prévue mais sans image indexée	Partiellement validé
BF7	Génération des fiches par `GeminiService`	Implémenté
BF8	Modification manuelle depuis l'écran d'édition Flutter	Implémenté
BF9	Archivage dans Supabase via le workflow n8n	Implémenté
BF10	Webhook post-analyse opérationnel ; notification FCM non validée	Partiellement validé
BF11	Production d'un rapport HTML récapitulatif par n8n	Implémenté
BF12	Export vers la GMAO de SFM Technologies	Non implémenté

Le prototype valide ainsi le flux métier principal sans prétendre couvrir l'ensemble des fonctions d'une solution industrielle. L'export GMAO a été maintenu hors périmètre, car aucun connecteur ni contrat d'API cible n'était disponible pendant le projet. De même, la notification mobile est préparée dans le workflow, mais sa validation nécessite encore la configuration complète de Firebase Cloud Messaging.

La conformité fonctionnelle ne suffit toutefois pas à conclure que toutes les exigences de qualité définies au chapitre 1 sont satisfaites. Le tableau 6.2 rapproche donc chaque besoin non fonctionnel des preuves effectivement disponibles à la fin du projet. Le statut *non validé* ne signifie pas nécessairement que le système échoue, mais qu'aucun essai conforme au protocole prévu ne permet encore de conclure.

TABLE 6.2 – Matrice de conformité des besoins non fonctionnels

ID	Exigence	Résultat et justification	Statut
BNF1	mAP@50 ≥ 0,70	mAP@50 de 74,9 % sur 60 keyframes et 59 objets annotés après contrôle de la vérité terrain.	Conforme
BNF2	IDF1 ≥ 70 %	Les annotations ne contiennent pas d'identifiants temporels de trajectoire ; l'IDF1 ne peut pas être calculé.	Non évalué
BNF3	Analyse en moins de 60 s	Les quatre vidéos, toutes inférieures à 30 s, sont terminées en 21,4 s en moyenne côté client.	Conforme sur le corpus testé
BNF4	Au moins 95 % de succès sur 20 requêtes	Le test du 9 août 2026 termine 20 requêtes consécutives sur 20, sans erreur HTTP 429, soit un taux de succès de 100 %.	Conforme
BNF5	Séparation en six services	Les responsabilités sont réparties entre les services vidéo, détection, suivi, RAG, analyse multimodale et validation.	Conforme
BNF6	Communications HTTPS	Les échanges entre Flutter et FastAPI utilisent HTTP sur le réseau Wi-Fi local de démonstration.	Non conforme dans le prototype
BNF7	Dix demandes successives sans redémarrage	Les 20 requêtes du test de robustesse ont été terminées sans redémarrage du backend.	Conforme
BNF8	Résultats accessibles en trois actions	Le parcours permet de choisir ou capturer une vidéo, lancer l'analyse puis consulter les fiches ; aucune étude d'utilisabilité formelle n'a toutefois été menée.	Partiellement validé

Cette matrice distingue ainsi les résultats mesurés des objectifs encore à valider. Les priorités avant une mise en production sont l'activation de HTTPS, la poursuite des essais de charge dans des conditions plus variées, l'annotation temporelle nécessaire à l'IDF1 et une évaluation d'utilisabilité avec des techniciens.

6.2 Environnement d'implémentation

L'environnement retenu privilégie une installation légère, compatible avec la machine disponible. Aucun GPU dédié n'est requis : l'inférence YOLO-World est réalisée sur CPU, tandis que l'analyse multimodale est déléguée à l'API Gemini.

TABLE 6.3 – Environnement logiciel du prototype déployé

Composant	Version	Rôle
Python	3.11	Langage du backend
FastAPI / Uvicorn	0.111 / 0.30	API REST et serveur ASGI
Pydantic	2.9	Modèle `Equipment` et validation
Ultralytics	8.4.39	Exécution de YOLO-World
supervision	0.24	Intégration de ByteTrack
OpenCV	4.10	Lecture et préparation des vidéos
sentence-transformers	3.0	Embeddings MiniLM de 384 dimensions
CLIP ViT-B/32	1.0	Embeddings visuels de 512 dimensions
qdrant-client	1.12	Index vectoriel local en mémoire
google-generativeai	0.8.3	Accès à Gemini 2.5 Flash
Flutter / Dart	3.44.1 / 3.8	Application mobile Android
n8n / Node.js	2.8.4 / 20.x	Automatisation post-analyse
Supabase	Service cloud	Persistance PostgreSQL

La machine locale fonctionne sous Windows 11, avec un processeur Intel Core i7-1165G7 et 16 Go de mémoire vive. Le téléphone Android et la machine doivent être connectés au même réseau Wi-Fi. Cette architecture a été choisie pour la validation du prototype ; elle ne garantit ni haute disponibilité ni accès distant sécurisé.

6.3 Implémentation du backend FastAPI

Le backend constitue le point central du système. Il reçoit la vidéo, orchestre les services du pipeline, construit la réponse JSON et transmet le résultat à n8n. Le fichier `main.py` initialise les modèles au démarrage grâce au cycle de vie de FastAPI. Cette initialisation évite de recharger YOLO-World, CLIP, MiniLM et la base RAG à chaque requête.

6.3.1 Organisation des services

L'implémentation suit la séparation des responsabilités définie au chapitre 4. Chaque étape principale est isolée dans un service spécialisé.

TABLE 6.4 – Services du backend et responsabilités associées

Service	Responsabilité
`VideoProcessor`	Contrôle de la vidéo, filtrage Laplacien avec $\tau = 50$ et sélection des keyframes par diversité HSV
`YOLOService`	Chargement explicite de `YOLOWorld`, configuration des 34 prompts textuels et détection *zero-shot*
`TrackingService`	Association ByteTrack, suppression des doublons et sélection du meilleur crop
`RAGService`	Chargement de Qdrant, embeddings MiniLM et CLIP, puis récupération du contexte documentaire
`GeminiService`	Construction du prompt multimodal, appel à Gemini 2.5 Flash et reprise sur erreur jusqu'à trois tentatives
`ValidationService`	Application des valeurs par défaut, validation Pydantic et positionnement de `needsReview`
`ScrapingService`	Collecte complémentaire de pages web et extraction de documents PDF

Cette organisation facilite le diagnostic des erreurs. Une modification du filtre vidéo ou du prompt Gemini peut être réalisée sans modifier le point d'accès HTTP ni le code de persistance.

6.3.2 Traitement d'une requête d'analyse

L'application envoie une requête HTTP `POST` contenant la vidéo et la localisation. Le backend sauvegarde temporairement le fichier, exécute les six étapes du pipeline, construit les métadonnées de traitement, puis retourne les fiches au téléphone. L'envoi vers n8n est effectué après la génération de la réponse métier.

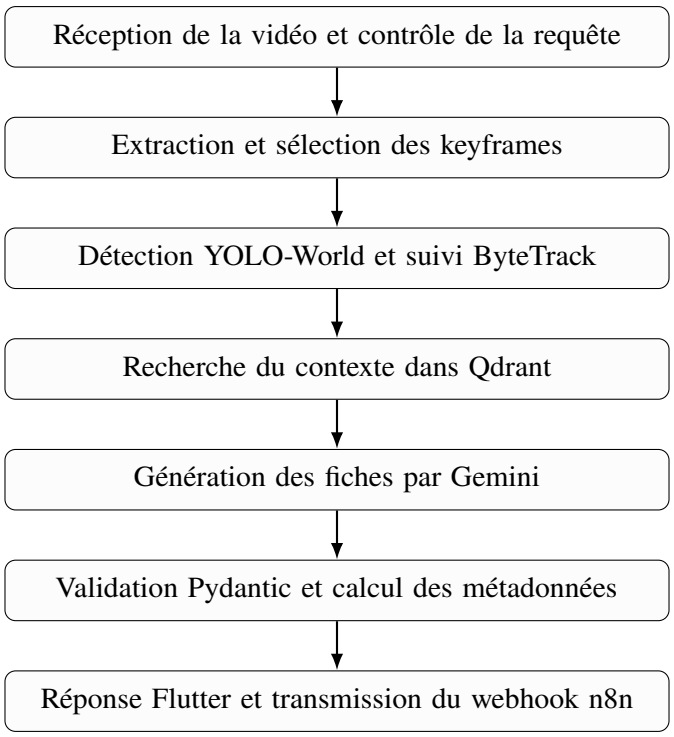


FIGURE 6.1 – Enchaînement opérationnel d'une analyse vidéo

6.3.3 Validation et gestion des erreurs

Le modèle Pydantic constitue la source unique de vérité entre le backend et l'application. Lorsque Gemini omet un champ nullable, le service de validation applique une valeur par défaut compatible avec le schéma. Une fiche conforme à Pydantic n'est toutefois pas nécessairement exacte sur le plan métier ; le champ `needsReview` et l'écran d'édition maintiennent donc l'utilisateur dans la boucle de décision.

Les erreurs d'entrée sont renvoyées sous forme de réponses HTTP explicites. Une requête sans fichier est rejetée avec le code 422. Les appels à Gemini disposent d'un mécanisme de reprise limité à trois tentatives. Les objets `datetime` sont convertis par un sérialiseur dédié avant l'envoi du webhook afin de garantir la compatibilité JSON.

6.4 Application mobile Flutter

L'application Flutter constitue l'interface destinée au technicien. Elle a été testée sur un téléphone Android réel connecté au même réseau Wi-Fi que le backend. Son organisation privilégie un parcours linéaire afin de réduire le nombre d'actions nécessaires entre la capture et la consultation des résultats.

TABLE 6.5 – Écrans principaux de l'application Flutter

Écran	Fonction assurée
`HomeScreen`	Accueil, vérification de la disponibilité du serveur et saisie de la localisation
`CameraScreen`	Capture d'une vidéo ou sélection depuis la galerie
`LoadingScreen`	Affichage d'une progression animée représentant les principales étapes du traitement
`ResultsScreen`	Présentation des fiches générées, des statuts et des indicateurs de confiance
`EditScreen`	Correction des listes de valeurs et des champs textuels avant validation
`EquipmentCard`	Composant réutilisable pour l'affichage synthétique d'un équipement

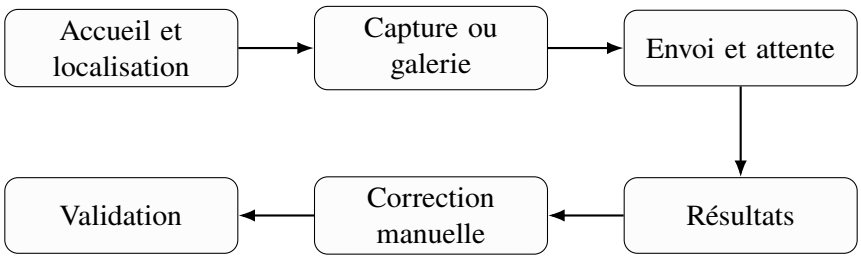
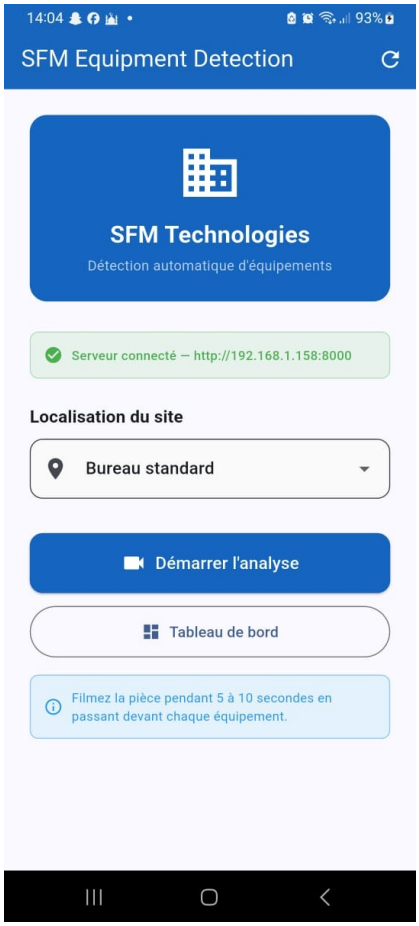


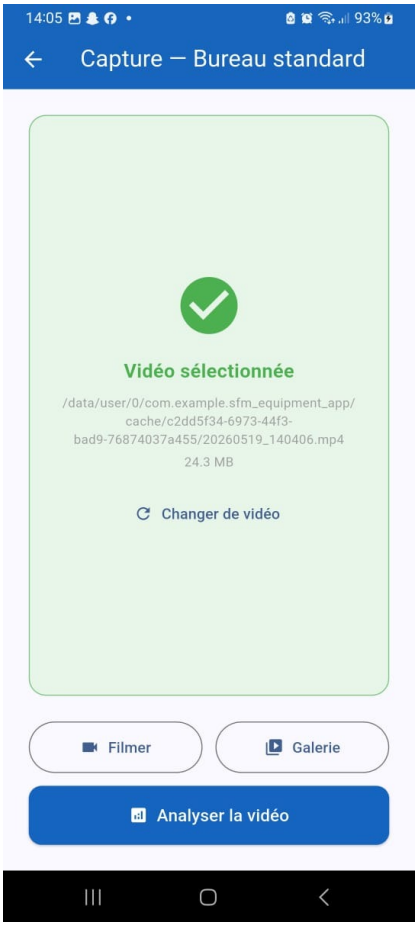
FIGURE 6.2 – Parcours principal dans l'application mobile

Les figures 6.3 et 6.4 présentent les écrans observés sur le téléphone Android utilisé pendant la validation. Elles complètent le diagramme fonctionnel par des captures de l'application réellement exécutée.

La gestion de l'adresse du backend demeure manuelle. L'URL de base est mise à jour lorsque l'adresse IP attribuée au poste change. Cette contrainte est acceptable pour la démonstration, mais elle devra être remplacée par une configuration externalisée ou un nom DNS dans une version déployée.



(a) Accueil, localisation et disponibilité du serveur



(b) Vidéo sélectionnée avant le lancement de l'analyse

FIGURE 6.3 – Capture ou sélection d'une vidéo dans l'application Flutter

6.5 Automatisation avec n8n et persistance Supabase

Le traitement Vision-RAG reste entièrement orchestré par FastAPI. n8n intervient après l'analyse afin de découpler la persistance et la génération du rapport des services d'intelligence artificielle. Le backend transmet un webhook contenant l'identifiant de la vidéo, la localisation, le temps de traitement, les fiches et les métadonnées de la session.

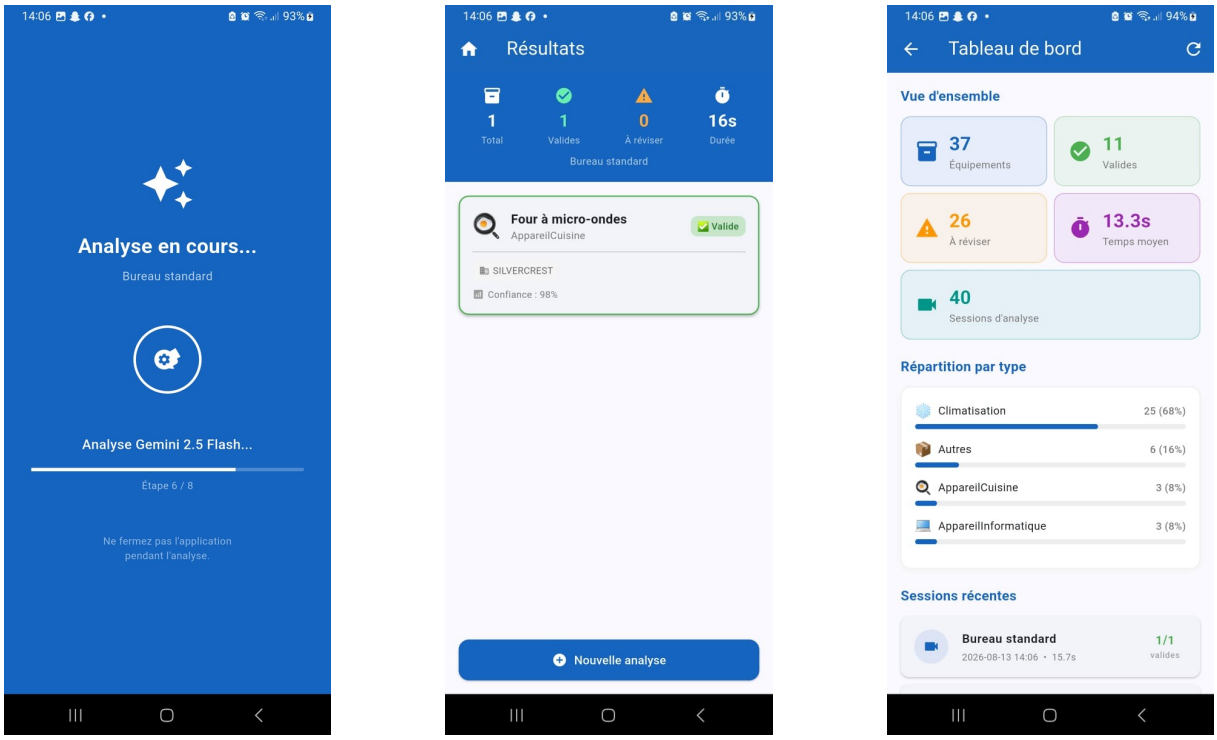
6.5.1 Workflow post-analyse

Le workflow comporte cinq opérations principales :

1. réception du webhook envoyé par FastAPI ;
2. transformation et typage des données dans un nœud JavaScript ;
3. création de la session dans la table `analysis_sessions` ;
4. insertion des fiches dans la table `equipments` ;
5. fusion des résultats et génération du rapport HTML.

6.5.2 Modèle de persistance

Supabase héberge deux tables principales. La table `analysis_sessions` conserve les informations globales d'une exécution : vidéo, localisation, durée et compteurs. La table `equipments` contient les



(a) Progression de l'analyse multimodale

(b) Fiche d'équipement générée

(c) Synthèse des analyses dans le tableau de bord

FIGURE 6.4 – Analyse, résultat et suivi des sessions dans l'application mobile

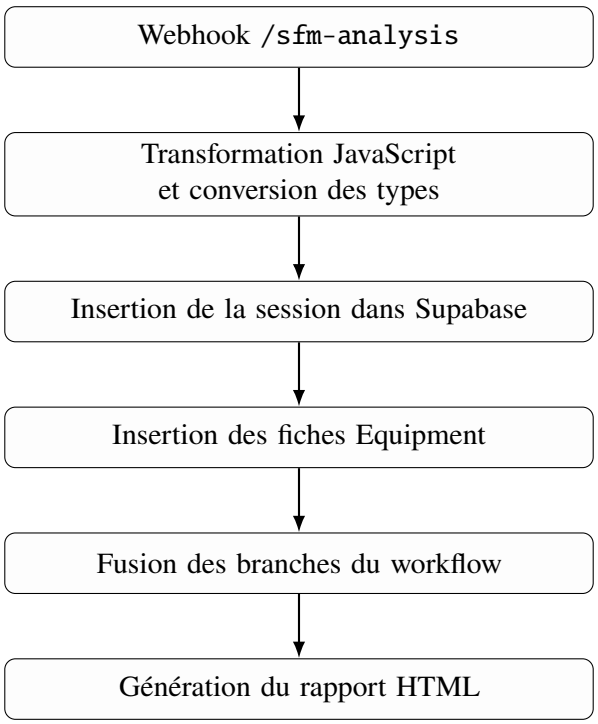


FIGURE 6.5 – Workflow n8n mis en œuvre après l'analyse

fiches individuelles et peut être reliée à la session correspondante. Les clés d'accès et l'identifiant réel du projet Supabase ne sont pas reproduits dans le rapport.

La génération HTML fournit un livrable immédiatement consultable sans imposer la création d'un moteur de rapports supplémentaire. Elle constitue une preuve de faisabilité ; la normalisation graphique et l'export PDF pourront être ajoutés dans une version ultérieure.

6.6 Installation et exécution du prototype

Le lancement nécessite trois processus : le backend, n8n et l'application Flutter. Les secrets tels que la clé Gemini et les identifiants Supabase doivent être placés dans des variables d'environnement et ne doivent pas être intégrés au code source.

6.6.1 Démarrage du backend

Listing 6.1 – Démarrage du backend sous Windows

```
cd backend
venv\Scripts\activate
python main.py
```

Le serveur écoute sur `0.0.0.0:8000`, ce qui autorise les connexions du téléphone sur le réseau local. Une règle du pare-feu Windows doit permettre les connexions entrantes vers le port 8000.

6.6.2 Démarrage de n8n

Listing 6.2 – Démarrage du service d'automatisation

```
n8n start
```

L'interface d'administration est disponible localement sur le port 5678. Le workflow doit être activé afin que son point d'accès de production accepte les webhooks envoyés par FastAPI.

6.6.3 Lancement de l'application Android

Listing 6.3 – Installation de l'application sur le téléphone

```
cd sfm_equipment_app
flutter pub get
flutter run
```

L'adresse de base configurée dans `ApiService` doit prendre la forme `http://IP-PC:8000`. Le téléphone et le poste doivent utiliser le même réseau Wi-Fi. L'usage de HTTP en réseau local reste limité à la démonstration ; une mise en production doit obligatoirement utiliser HTTPS.

6.7 Vérification opérationnelle du déploiement

Les performances détaillées sont analysées au chapitre 5. La présente section se limite à vérifier que chaque composant peut être exécuté et communiquer avec les autres dans l'environnement retenu.

TABLE 6.6 – Vérifications opérationnelles réalisées

Composant	Vérification	Résultat
Backend FastAPI	Nouvelle suite automatisée après correction de YOLO-World	4/4 vidéos terminées ; moyenne de 21,4 s
Gestion des erreurs	Envoi d'une requête sans fichier	HTTP 422 obtenu
Concurrence	Envoi de deux requêtes en parallèle	Deux réponses HTTP 200
Application Flutter	Analyse de V4 depuis un téléphone Android réel	Fiche affichée et modifiable
n8n	Service arrêté volontairement pendant la nouvelle suite	Échecs non bloquants ; validation antérieure conservée
Supabase	Insertion d'une session et d'un équipement de test	Insertion confirmée à 18 :14 :22
Rapport HTML	Génération du rapport pour la requête de test	Génération confirmée à 18 :14 :23

Ces vérifications confirment l'intégration technique du prototype, mais ne doivent pas être confondues avec les métriques de précision de la détection ou du suivi. En particulier, le score interne sur 100 de la suite automatisée repose sur des critères binaires d'intégration et n'est pas retenu comme mesure scientifique de performance.

Un premier test par injection manuelle d'un corps JSON avait validé la chaîne n8n–Supabase–rapport HTML. Une exécution antérieure avec n8n actif avait aussi validé la transmission automatique depuis FastAPI. Pendant la nouvelle suite, n8n n'a volontairement pas été démarré : les quatre tentatives ont produit un avertissement non bloquant, tandis que les analyses ont continué à retourner une réponse HTTP 200. Ce comportement confirme que l'indisponibilité du workflow post-analyse ne bloque pas le traitement principal.

6.8 Incidents rencontrés et stabilisation

La mise en œuvre a nécessité plusieurs corrections touchant les trois niveaux du système. Leur documentation permet de montrer le caractère itératif du développement et facilite la maintenance future.

TABLE 6.7 – Principaux incidents corrigés pendant l'intégration

Incident	Cause identifiée	Correction
Statut d'équipement nul	Champ omis par Gemini	Valeur par défaut dans `ValidationService`
Réponse JSON impossible à sérialiser	Présence d'objets `datetime`	Sérialiseur JSON dédié
Aucun crop après suivi	Deux instances de tracker utilisées	Utilisation d'une instance unique de ByteTrack
Réponse Gemini tronquée	Limite de sortie insuffisante	Limite portée à 2048 tokens
Classes COCO retournées au lieu des prompts métier	Chargement du modèle avec la classe générique `YOLO`	Utilisation explicite de `YOLOWorld` et du fichier `yolov8x-worldv2.pt`
Four à micro-ondes non détecté	Prompts YOLO incomplets	Ajout de *microwave* et *microwave oven*
Climatiseur manqué	Seuil de netteté trop strict	Abaissement de τ de 100 à 50
RAG vide au premier appel	Index Qdrant non chargé	Initialisation dans le cycle de vie FastAPI
Erreur de liste Flutter	Valeur absente du menu déroulant	Alignement de la valeur initiale sur les options
Backend inaccessible	Adresse locale ou pare-feu incorrect	Mise à jour de l'URL et ouverture du port 8000
Paramètres n8n invalides	Expressions JSON et types incorrects	Sérialisation explicite et conversions numériques

6.9 Limites du déploiement actuel

Le prototype démontre la faisabilité de l'approche, mais plusieurs limites empêchent encore son exploitation comme service de production.

TABLE 6.8 – Limites du prototype et évolutions nécessaires

Limite	Impact	Évolution proposée
Qdrant en mémoire	Reconstruction de l'index après chaque redémarrage	Stockage persistant sur une machine Linux
Adresse IP dynamique	Modification manuelle de l'URL Flutter	DNS local ou configuration externalisée
Exécution CPU	Temps plus élevé sur les vidéos longues	GPU local ou service d'inférence distant
Dépendance à Gemini	Indisponibilité hors connexion et quotas API	Cache, limitation des appels et mode dégradé
HTTP sur réseau local	Communications non chiffrées	Reverse proxy avec certificat TLS
Authentification absente	Aucun contrôle d'accès utilisateur	Authentification JWT, gestion des rôles et quota quotidien par utilisateur
FCM non validé	Notification mobile non garantie	Configuration Firebase et tests sur plusieurs appareils
Export GMAO absent	Pas d'intégration au système métier final	Définition du contrat d'échange avec SFM Technologies
Corpus et base RAG limités	Couverture réduite des équipements	Enrichissement continu par documents constructeurs validés

6.10 Perspective de mise en production

Une mise en production ne consisterait pas uniquement à déplacer le prototype sur un serveur. Elle nécessiterait une évolution progressive de l'architecture :

1. externaliser les paramètres et les secrets dans un gestionnaire de configuration sécurisé ;
2. conteneuriser FastAPI, n8n et Qdrant afin de garantir la reproductibilité du déploiement ;
3. activer la persistance Qdrant et sauvegarder régulièrement les données Supabase ;
4. placer le backend derrière un reverse proxy HTTPS avec authentification et limitation du débit ;
5. ajouter un compte utilisateur et limiter chaque compte à trois analyses par jour, avec contrôle du quota par le backend ;
6. mettre en place une file de tâches pour absorber plusieurs analyses simultanées sans bloquer l'API ;
7. ajouter des journaux centralisés, des métriques de supervision et des alertes ;
8. valider le connecteur GMAO et les notifications sur l'infrastructure réelle de SFM Technologies.

La limitation à trois analyses quotidiennes doit être appliquée côté serveur, et non uniquement dans l'interface Flutter. Après authentification, le backend identifierait l'utilisateur à partir de son jeton, compterait dans Supabase les analyses terminées pendant la journée et refuserait une quatrième demande avec une réponse explicite. L'application afficherait le nombre d'analyses restantes et la date de réinitialisation du quota. Cette règle relève d'un choix de gestion des ressources et pourra être rendue configurable selon le profil de l'utilisateur.

Docker, évoqué au chapitre 2, relève donc de cette perspective et non de l'environnement actuellement utilisé. La priorité immédiate reste la stabilisation des métriques expérimentales et l'élargissement du corpus avant toute ouverture à des utilisateurs externes.

Conclusion

Ce chapitre a présenté la mise en œuvre du prototype Vision-RAG et son déploiement dans un environnement local connecté à des services cloud. Le backend FastAPI orchestre six étapes de traitement spécialisées, tandis que l'application Flutter permet au technicien de capturer une vidéo, de consulter les fiches générées et de les corriger. Le workflow n8n assure les opérations post-analyse, notamment la persistance dans Supabase et la génération d'un rapport HTML.

Les vérifications réalisées montrent que les composants communiquent dans les conditions prévues pour la démonstration. Le déploiement demeure néanmoins celui d'un prototype : l'authentification, HTTPS, la persistance Qdrant, la notification FCM et l'export GMAO doivent encore être finalisés avant une mise en production. Ces limites ne remettent pas en cause la faisabilité de la solution, mais définissent clairement les travaux nécessaires pour passer d'une preuve de concept à un outil exploitable à plus grande échelle.

Conclusion générale et perspectives

Ce projet de fin d'études avait pour objectif de concevoir un prototype capable d'assister un technicien dans le recensement d'équipements électriques à partir de courtes vidéos filmées par smartphone. La solution développée couvre le parcours allant de la sélection de la vidéo jusqu'à la production d'une fiche `Equipment` structurée, modifiable depuis une application Flutter et transmissible vers un workflow d'automatisation.

L'architecture mise en œuvre combine plusieurs approches complémentaires. La sélection des keyframes par OpenCV réduit le volume d'images à analyser. YOLO-World localise les équipements selon un vocabulaire configurable, tandis que ByteTrack regroupe les détections répétées et sélectionne les crops les plus exploitables. Le module RAG interroge Qdrant afin de fournir un contexte documentaire à Gemini 2.5 Flash. Enfin, Pydantic contrôle la structure des objets JSON avant leur affichage et leur éventuelle transmission à n8n.

L'évaluation expérimentale repose sur quatre vidéos totalisant 1 385 frames. Les 60 keyframes retenues ont été annotées manuellement et comprennent 59 objets de référence ainsi que trois images négatives. Après correction du chargement de YOLO-World et relecture de la vérité terrain, le détecteur obtient un mAP@50 de 74,9 %, une précision micro de 63,0 %, un rappel micro de 78,0 % et un score F1 micro de 69,7 %. Les performances varient toutefois fortement selon les catégories. Le climatiseur atteint un AP@50 de 100 %, alors que le réfrigérateur reste la classe la plus difficile avec un rappel de 35,3 %. La surdétection du téléviseur constitue également une source importante de faux positifs.

La nouvelle suite d'intégration termine les quatre vidéos avec six fiches générées, dont cinq passent directement les contrôles structurels et une est marquée pour revue. Le temps interne moyen du pipeline est de 16,74 secondes, contre 21,4 secondes observées côté client. Les tests complémentaires valident le rejet d'une requête sans fichier, le traitement d'une localisation vide et l'exécution de deux requêtes en parallèle. Une réponse Gemini mal formée a été récupérée grâce au mécanisme de nouvelle tentative. Le test de robustesse répété le 9 août 2026 termine également 20 requêtes consécutives sur 20, sans erreur HTTP 429 ni redémarrage, en 14,0 secondes en moyenne. L'arrêt de n8n pendant la dernière suite n'a pas bloqué les réponses du backend, ce qui confirme le caractère non bloquant du traitement post-analyse.

Ces résultats valident la faisabilité technique de l'approche, mais ne constituent pas une qualification industrielle. Les annotations actuelles ne contiennent pas d'identifiants temporels et ne permettent donc pas de calculer l'IDF1 de ByteTrack. Les classes écran informatique et switch réseau ne sont pas représentées dans la vérité terrain. La base RAG ne contient que 11 entrées textuelles et aucune image indexée ; l'étude d'ablation n'a montré aucun enrichissement technique mesurable au cours des trois répétitions réalisées sur la vidéo V1, tout en augmentant le temps moyen de 19,8 %. Enfin, les fiches générées n'ont pas encore fait l'objet d'une campagne formelle de validation champ par champ par un technicien.

Les perspectives prioritaires consistent à élargir le corpus vidéo et à équilibrer les catégories, enrichir la base documentaire avec des fiches constructeurs vérifiées, indexer réellement les images de référence et ajuster les prompts ou les seuils de détection pour réduire les faux positifs. Une annotation temporelle permettrait d'évaluer quantitativement ByteTrack. La mise en production nécessiterait en outre une infrastructure persistante et sécurisée, avec HTTPS, authentification, supervision, file de tâches, persistance Qdrant et intégration validée avec la GMAO de SFM Technologies.

En définitive, Vision-RAG doit être considéré comme un outil d'assistance au technicien. Il automatise une partie importante du recensement et standardise la restitution des résultats, tout en maintenant une validation humaine pour les cas ambigus et les informations techniques non vérifiables.

Bibliographie

[1] P. Chapman *et al.*, “Crisp-dm 1.0 step-by-step data mining guide,” 2000.

[2] A. Zareian, K. Rosa, D. Hu, and S.-F. Chang, “Open-vocabulary object detection using captions,” *CVPR*, 2021.

[3] T. Cheng *et al.*, “Yolo-world : Real-time open-vocabulary object detection,” *arXiv preprint arXiv :2401.17270*, 2024.

[4] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *ICIP*, 2016.

[5] International Organization for Standardization, “Iso 50002-1 :2025 — energy audits — part 1 : General requirements with guidance for use,” International Standard ISO 50002-1 :2025, ISO, 2025.

[6] International Organization for Standardization, “Iso 50002-2 :2025 — energy audits — part 2 : Guidance for conducting an energy audit using iso 50002-1 in buildings,” International Standard ISO 50002-2 :2025, ISO, 2025.

[7] International Energy Agency, “Energy efficiency 2025 — buildings,” 2025. Consulté le 9 août 2026.

[8] Y. Zhang *et al.*, “Bytetrack : Multi-object tracking by associating every detection box,” *ECCV*, 2022.

[9] Google, “Gemini technical report,” 2024.

[10] Qdrant, “Qdrant vector database.” `https://qdrant.tech`, 2025.

[11] Google, “Flutter documentation.” `https://flutter.dev`, 2025.

[12] n8n, “n8n workflow automation.” `https://n8n.io`, 2025.

[13] T.-Y. Lin *et al.*, “Microsoft coco : Common objects in context,” in *ECCV*, 2014.

[14] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, “Mot16 : A benchmark for multi-object tracking,” *arXiv preprint arXiv :1603.00831*, 2016.

[15] R. Szeliski, *Computer Vision : Algorithms and Applications*. Springer, 2 ed., 2022.

[16] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[17] H. Bay, T. Tuytelaars, and L. Van Gool, “Surf : Speeded up robust features,” in *European Conference on Computer Vision*, pp. 404–417, Springer, 2006.

[18] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[19] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, “Object detection in 20 years : A survey,” *Proceedings of the IEEE*, 2023.

[20] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The pascal visual object classes (voc) challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[21] A. Gupta, P. Dollar, and R. Girshick, “Lvis : A dataset for large vocabulary instance segmentation,” in *CVPR*, 2019.

[22] A. Radford, J. W. Kim, C. Hallacy, *et al.*, “Learning transferable visual models from natural language supervision,” in *ICML*, 2021.

[23] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *CVPR*, 2014.

[24] R. Girshick, “Fast r-cnn,” in *ICCV*, 2015.

[25] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn : Towards real-time object detection with region proposal networks,” in *NeurIPS*, 2015.

[26] W. Liu, D. Anguelov, D. Erhan, *et al.*, “Ssd : Single shot multibox detector,” in *ECCV*, 2016.

[27] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once : Unified, real-time object detection,” in *CVPR*, 2016.

[28] S. Liu *et al.*, “Grounding dino : Marrying dino with grounded pre-training for open-set object detection,” *arXiv preprint arXiv :2303.05499*, 2023.

[29] Ultralytics, “Yolov8 model performance documentation,” 2026. Consulté le 22 juillet 2026.

[30] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *ICIP*, 2017.

[31] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.

[32] Y. Du *et al.*, “An overview of vision-language pre-trained models,” *arXiv preprint arXiv :2202.09061*, 2022.

[33] OpenAI, “Gpt-4v(ision) system card,” 2023.

[34] Google DeepMind, “Gemini : A family of highly capable multimodal models,” 2023.

[35] Anthropic, “Claude 3.5 sonnet model card addendum,” 2024.

[36] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” 2023.

[37] OpenAI, “Api pricing,” 2026. Consulté le 22 juillet 2026.

[38] Google, “Gemini developer api pricing,” 2026. Consulté le 22 juillet 2026.

[39] Anthropic, “Claude platform pricing,” 2026. Consulté le 22 juillet 2026.

[40] L. Huang *et al.*, “A survey on hallucination in large language models,” *arXiv preprint arXiv :2311.05232*, 2023.

[41] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, 2020.

[42] N. Reimers and I. Gurevych, “Sentence-bert : Sentence embeddings using siamese bert-networks,” *arXiv preprint arXiv :1908.10084*, 2019.

[43] Pinecone, “Pinecone documentation,” 2024.

[44] Chroma, “Chroma documentation,” 2024.

[45] Weaviate, “Weaviate documentation,” 2024.

[46] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, 2019.

[47] Qdrant Team, “Qdrant vector database documentation,” 2023.

[48] S. Pertuz, D. Puig, and M. A. Garcia, “Analysis of focus measure operators for shape-from-focus,” *Pattern Recognition*, vol. 46, no. 5, pp. 1415–1432, 2013.

[49] I. Koprinska and S. Carrato, “Temporal video segmentation : A survey,” in *Proceedings of IEEE Workshop on Multimedia Signal Processing*, pp. 85–90, 2001.

[50] International Electrotechnical Commission, “Iec 60038 : Standard voltages,” 2009.

[51] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, “An image is worth 16x16 words : Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.

[52] Qdrant Team, “Qdrant vector database documentation,” 2024.

[53] Qdrant Team, “Qdrant storage and deployment modes,” 2024.

Annexe A

Éléments techniques complémentaires

Cette annexe rassemble les informations pratiques nécessaires pour retrouver les données expérimentales et reproduire les principales vérifications du prototype. Elle complète les chapitres d'évaluation et de déploiement sans reprendre les développements déjà présentés dans le corps du rapport.

A.1 Correspondance des vidéos expérimentales

Les identifiants abrégés utilisés dans les tableaux du chapitre 5 correspondent aux fichiers suivants.

TABLE A.1 – Correspondance entre les identifiants et les fichiers vidéo

ID	Fichier	Durée	Contenu principal
V1	`ma_video.mp4`	7,3 s	Ordinateur, climatiseur et switch
V2	`ma_video1.mp4`	20,2 s	Téléviseur
V3	`ma_video2.mp4`	12,5 s	Réfrigérateur, four à micro-ondes et switch
V4	`ma_video3.mp4`	12,1 s	Four à micro-ondes et switch

A.2 Procédure de reproduction

La reproduction des essais suppose que les dépendances du backend sont déjà installées et que les variables d'environnement contenant les accès à Gemini sont configurées. Les secrets ne doivent pas être enregistrés dans les scripts ni reproduits dans le rapport.

1. démarrer le backend FastAPI et attendre le chargement de YOLO-World, CLIP, MiniLM et de l'index Qdrant ;
2. vérifier la disponibilité du serveur sur le port 8000 ;
3. exécuter les quatre vidéos avec les mêmes seuils de netteté, de confiance et d'intersection sur union que ceux indiqués dans les chapitres 3 et 5 ;
4. conserver séparément les temps internes du backend et le temps total observé par le client ;
5. pour l'étude d'ablation, exécuter successivement les configurations avec et sans contexte RAG sur la même vidéo et répéter chaque configuration trois fois ;
6. archiver les réponses JSON et les journaux afin de pouvoir expliquer toute différence entre deux exécutions de Gemini.

La comparaison RAG présentée dans ce rapport a été réalisée avec la commande suivante depuis l'environnement du backend :

Listing A.1 – Lancement de l'étude comparative du RAG

```
python test_rag_comparison.py
```

A.3 Checklist de validation d'une exécution

Une exécution est considérée comme exploitable pour l'analyse lorsque les éléments suivants peuvent être vérifiés.

TABLE A.2 – Checklist minimale de validation expérimentale

N°	Vérification	Preuve à conserver
1	Le fichier vidéo est accepté et les keyframes sont produites.	Journal du traitement vidéo
2	Les détections YOLO-World sont associées aux classes configurées.	Fichier de prédictions ou image annotée
3	Les crops finaux ne contiennent pas de doublons manifestes.	Répertoire des crops et identifiants de suivi
4	La réponse respecte le modèle Pydantic `Equipment`.	Réponse JSON et résultat de validation
5	Les fiches incertaines portent l'indicateur `needsReview`.	Valeur du champ dans la réponse
6	Une indisponibilité de n8n ne bloque pas la réponse principale.	Code HTTP et avertissement non bloquant
7	Les temps des principales étapes et le temps total sont enregistrés.	Journal horodaté de l'exécution

Cette checklist facilite la répétition des essais et évite de confondre une réponse techniquement conforme avec une prédiction exacte sur le plan métier.